Engineering with AI

Beyond Vibe Coding

Yunus Kiran

First edition / 2026

About this edition

A practical comparison of BMAD Method, GSD, AWS AI-DLC, and AI Unified Process for developers, technical leads, and architects. The book follows decisions, context, and verification through a synthetic multi-tenant approval service.

The method accounts describe dated source snapshots. The case is a qualitative walkthrough. A separate TypeScript policy model has ten passing local checks. The source notes identify the inspected revisions and the limits of the evidence.

The manuscript text was drafted and revised using generative AI under the direction of Yunus Kiran. Technical, source, similarity and editorial checks in the publication report were performed by an AI assistant. Independent human review has not been completed. The cover was designed by an AI assistant using original programmatic typography and geometric artwork.

The methods and product names belong to their respective owners. Their use identifies the subjects discussed. This publication is independently prepared and carries no endorsement from the method maintainers or platform vendors. No third-party logo or screenshot is used on the cover.

Read chapters 1-3 for the comparison vocabulary, 4-7 for the four methods, 8-10 for specification policy, comparison and selection, and 11-14 for the case, assurance, diagnosis and future evaluation.

Contents

1. The Evolution of AI-Assisted Development 4

2. Beyond Vibe Coding 14

3. Understanding AI-Driven Methodologies 25

4. BMAD Method - Deep Dive 41

5. GSD - Deep Dive 59

6. AWS AI-DLC - Deep Dive 77

7. AI Unified Process - Deep Dive 96

8. Spec-Driven Development as an Approach 114

9. Comparing the Methodologies 129

10. Choosing the Right Methodology 146

11. Real-World Case Study: Building a SaaS 161

12. Testing, Security & Production Readiness 189

13. Common Failures and Anti-Patterns 207

14. The Future of AI-Native Engineering 219

Source notes 228

1. The Evolution of AI-Assisted Development

A bounded history, with dates

A developer asks an assistant to add an approval button. The assistant changes a component, adds an endpoint and writes a test. The screen looks convincing. Before accepting the change, someone still has to answer a less visible question: who is allowed to approve whose request?

That question is the starting point of this book. Producing an implementation and establishing that the implementation is acceptable are different kinds of work. An AI system may participate in both. A team needs a way to connect them.

Four selected sources help frame the change in scope. On June 29, 2021, GitHub announced the technical preview of Copilot, describing assistance that proposed lines and functions using the surrounding code. This is one historical marker for a particular product, rather than a founding date for AI-assisted programming. GitHub's announcement

In October 2022, Yao and colleagues introduced ReAct, which interleaved reasoning with actions that obtained information from an external environment. Its tasks included question answering and interactive decision making. This supplies an example of a research pattern for acting and observing, not an organization-level software delivery process. ReAct, first-version metadata and abstract

The 2024 SWE-agent paper described a system for navigating repositories, editing files and executing programs through an interface designed for a language-model agent. It gives us a concrete software-engineering example of delegated actions and feedback. Its benchmark results do not measure the four frameworks compared in this book. SWE-agent, first-version metadata and abstract

On July 31, 2025, AWS introduced its AI-Driven Development Life Cycle in an article that described AI preparing plans and artifacts across inception, construction and operations, with people validating consequential decisions. That article supplies a primary statement of the proposed lifecycle. Its claims

about faster delivery remain the publisher's claims. AWS's original AI-DLC article

These anchors concern a product preview, research patterns and a proposed lifecycle. They are different units of evidence, not consecutive generations of one system. They do not establish that ReAct caused the frameworks' development, or that every team passed through the same stages. Code suggestions, delegated changes and coordinated workflows can coexist in one project.

The useful historical question is therefore specific: what work does a source say the system undertakes, under what conditions, and what remains for someone else to decide? A feature announcement can establish that a vendor described a capability at a particular time. It cannot, by itself, establish widespread adoption, a productivity improvement or the disappearance of a human responsibility.

We will use this selective history to distinguish changes in the boundary of work delegated to a system. The founding histories of the four frameworks and broader adoption trends need separate evidence; these anchors do not establish either.

From assistance to delegated work

Consider three instructions a developer might give an AI system:

1. Complete this validation expression.
2. Add the request-approval behavior and show the changes.
3. Investigate the requirement, propose an implementation plan, make the change and assemble evidence for acceptance.

These are illustrative instructions, not records of executed sessions. Each delegates a different boundary of work. The first leaves the surrounding decisions with the developer. The second gives the system room to choose a design. The third also delegates parts of investigation, planning and verification.

As that boundary expands, more assumptions can enter the result. An approval endpoint needs a rule for the actor, the target request and the current state. A plan needs a view of which behavior is already agreed and

which behavior remains undecided. A verification report needs a reason to treat its checks as relevant to the requirement.

For this book, we will describe delegation through those concrete responsibilities rather than assign a single autonomy label to an entire project. Who may edit files? Who may settle an ambiguous business rule? Who may accept a test failure? Who may merge or release? Different answers can apply to the same assistant in different tasks.

A methodology becomes relevant when it gives these activities a repeatable arrangement: how work starts, what records carry decisions, when people intervene, how checks are performed and how changes are reconciled. The editor, terminal or coding host is part of the execution environment. It can enable or restrict that arrangement, but its presence does not tell us which arrangement the team follows.

That distinction defines the comparison in the rest of the book. BMAD Method, the selected GSD lineage, AWS AI-DLC and AI Unified Process (AIUP) are the four framework subjects. Spec-Driven Development (SDD) is a cross-cutting approach whose meaning and artifact policies require separate examination. A tool can implement pieces of several approaches. Counting it as another peer methodology would obscure the question we are trying to answer.

Follow the responsibility as the work changes shape

Imagine asking an assistant to explain a failing lookup in an existing repository. The immediate output is an explanation that an engineer can test against the code. If the engineer instead asks a tool-using agent to investigate and repair it, the expected work may now include choosing files, running commands, interpreting results and changing a query. The business intent is the same, but the agent has more opportunities to act on an incorrect assumption before someone inspects it.

The engineering record needs to match that wider assignment. “Fix the lookup” gives the agent little basis for deciding whether to alter a shared query or add a check at one route. “A tenant B actor must be unable to retrieve a known A request, while A's authorized read must still work” gives the assignment a testable boundary. A record can also state that the approved change concerns a local repository and that release is a separate decision.

With that information, the next engineer can tell where the assignment ends and which decision needs a person.

This example illustrates a distinction that applies throughout the book. Assistance can reduce the work of discovering and editing. Delegation asks the team to decide which judgments a tool may make along the way. The review question changes from “is this suggested line right?” to “did the work preserve the conditions we agreed to, and what evidence covers that claim?” Both kinds of work can occur in the same session.

Responsibilities that need owners

Our running example is a small vendor-access approval service. A requester asks for a vendor to receive temporary access; an eligible person in the same customer organization decides the request. The service records the decision. It does not provision access in another system. This is the book's synthetic case, with requirements defined in the case specification.

Suppose the requester also holds an administrator role. May that person approve the request? In this case, the answer is no. That is a product rule, CS-05. A framework may help elicit the rule or carry it into a plan. A generated class cannot establish the organization's desired rule merely by implementing one interpretation.

We will use an explicit ownership record when discussing this case:

Responsibility	Decision the case needs	Proposed owner in the exercise
Product intent	Self-approval stays forbidden when roles overlap	Author acting as product decision owner
Technical design	Where and how the chosen implementation enforces the rule	Implementing engineer, with technical review
Acceptance	Which positive and negative observations would establish the agreed behavior	Acceptance reviewer, independent of the implementation output
Change control	Whether a new approval policy applies to existing requests	Product decision owner, recorded before the change episode

Responsibility	Decision the case needs	Proposed owner in the exercise
Operation	What evidence would justify running and supporting the service	Assigned operator and release owner in a future operational exercise

These are proposed duties, not a completed staffing record. One person may hold several duties, but the record should expose that overlap. An AI-generated review does not create an independent human reviewer.

The ownership question matters even when an answer appears obvious. If an assistant infers that administrators can do everything, the resulting code may be internally consistent and still violate CS-05. If it infers that a new policy applies retroactively, a technically successful migration may change promises already made to users. Those are hypothetical failure paths we can investigate, not incidents observed in this project.

Describe authority before choosing a workflow

A practical delegation brief can fit in a few sentences. The engineer authorizes investigation of a reproducible defect in a local branch. The assistant may inspect source, edit the affected implementation and run the existing checks. The product owner retains the decision about externally visible behavior where the current contract is ambiguous. The release owner retains the decision about using the resulting artifact in a stateful environment. The brief also identifies the observation that would make the investigation useful.

This division can change during work. An investigation may reveal that the reported defect comes from a deliberate rule. A local repair may expose a migration need. The contributor should record that discovery where it affects the next action, along with the owner of any new decision. In this way, escalation becomes a specific request to settle a condition rather than a broad instruction to approve everything again.

Consider a generated recommendation to remove a validation guard. The implementation might become simpler, and the reported example might begin to pass. The delegation brief gives the reviewer a way to ask whether removing that guard changes an accepted boundary. The engineer can then inspect its callers and associated requirements before accepting the patch.

The quality of that review depends on a retrievable rule and a relevant observation.

A maintenance change as a running question

Begin with a deliberately small change: repair an approval path that lets a requester approve their own item when they also have the approver role. This is an illustrative maintenance question. The repository does not yet contain that implementation or an observed defect.

The visible output might be a conditional check and a regression test. To assess it, we would first identify the decision being repaired. The intended rule is independent of which role labels the requester holds. The negative example starts with a pending request and an overlapping-role requester trying to approve it. A separate positive example starts with an equivalent pending request and an eligible, different approver in the same tenant. Keeping the starting state comparable prevents a terminal-state rejection from masquerading as evidence of the self-approval rule.

Next comes the boundary of the repair. Does the application expose one decision route or several? Is role membership checked when the decision is made? Could a foreign-tenant request be found through a direct identifier? These are questions for source inspection and the eventual acceptance contract. We do not know their answers before a baseline exists.

Finally, there is the evidence boundary. A test named `prevents self-approval` is a useful clue, but its assertion must exercise the forbidden action. A passing check of a hidden button would establish something about that interface; it would leave the server's decision behavior unresolved. In the planned case, acceptance includes business-interface checks for the rule.

The same maintenance request gives us several distinct artifacts: an agreed rule, a bounded change, a rationale for the chosen location and observations from relevant checks. A methodology can prescribe how those artifacts relate. Its value for this task remains a question to evaluate, including the effort required to prepare and maintain them.

This small example also prevents process size from becoming a proxy for rigor. A short repair may need a short record. A policy change affecting concurrent decisions and existing requests needs different investigation. We

will ask whether each selected framework can express the appropriate amount of work, while retaining the obligations that matter for the change.

Make the first maintenance question observable

Suppose a maintainer sees a report that an actor in B can retrieve a request owned by A. The first useful record identifies the observed path and the intended behavior. If the report contains only a screenshot of an error, the maintainer still needs a known A request, an eligible A identity, a B identity and a way to exercise the same request under both. A reproduction establishes what happened at a particular boundary before anyone proposes a repair.

Now consider what an agent could infer from the repository. It may find a data model with a tenant field and a helper used by most routes. Those are useful clues. They cannot settle the intended error or disclosure behavior if the owner never made that choice. A reviewer can authorize a narrow change after resolving the relevant part of the contract. Any broader decision about membership lifecycle or privilege inheritance can be handled separately.

Follow the change back to the original report after editing. If the direct lookup is corrected, inspect other consumers of the changed helper and see whether permitted access still works. Record which paths were exercised and what remains open. This small narrative provides the thread for later chapters: define a boundary, keep the decision available, and collect evidence at the place where the behavior can be observed.

What changed, what is uncertain

The central hypothesis of this book is that making intent, decision rights and acceptance evidence explicit can make AI-driven engineering easier to inspect and reproduce. It is a hypothesis to examine through documented mechanisms and bounded exercises. It is not an established performance advantage for any named framework.

Documentation can show that a workflow calls for a plan or review. It cannot show that the operator read the plan, that the review found the relevant defect or that the additional step justified its cost. A later execution record would need to establish those observations for its own conditions. Even a

successful synthetic exercise would leave questions about teams, larger systems and longer maintenance periods.

We will therefore keep three kinds of statement distinct. A source may **prescribe** an activity. A recorded execution may **show** what happened in one run. An editorial argument may **recommend** a choice under declared constraints. Readers should be able to tell which kind they are reading without reconstructing the research process.

The evidence can challenge the thesis. If a framework adds artifacts that no one uses to make or check a decision, that is a cost worth recording. If a lightweight path preserves the necessary rule with less work, it deserves serious consideration. If a sophisticated review chain misses the relevant boundary, its existence cannot stand in for acceptance evidence.

We have not run the four-way case study. There is no measured winner, productivity multiplier or production-readiness result in this introduction. The comparison begins with questions whose answers could change our choices.

Notice which part of engineering became cheaper to repeat

Consider a team investigating a fault in a reporting service. A suggested patch can be produced quickly, but the team still has to find out which customers encountered the fault, whether the report is relied on for reconciliation and which historical data the fix should affect. Those decisions involve evidence the repository may only partly contain. If a contributor can generate several plausible patches, the team needs an efficient way to distinguish their assumptions.

One investigation begins with a paired fixture. Reproduce an incorrect total and a correct total that could be broken by the same change. Ask the assistant to identify the query and its callers, then record the observations that led to a proposed repair. The team can discard two patches and retain the investigation because the accepted behavior and reproduction still apply. The useful intermediate product is an account another engineer can inspect without starting over.

This is why the book keeps returning to revision and context. In an ordinary review, two diffs with identical code may look equivalent. If one was checked against an old fixture, while the other has a current observation from the path

the customer uses, they support different decisions. The distinction also applies to documents. A policy written yesterday may still govern the work if it remains accepted, while a test from an hour ago may already be stale because a query changed.

For an organization experimenting with AI assistance, a modest first question is whether a second engineer can recover the evidence for a change. Give them the accepted intent and the repository, and see whether they can find the decisive observation and its scope. The result can inform how the team writes handoffs. Productivity across projects would require a different study and a broader body of evidence.

Read the next success claim with one question

When someone reports that an agent completed a change, ask which decision the evidence lets a reviewer make. A generated patch may be ready for inspection. A local test may justify continued integration. A reviewed boundary observation may justify accepting a specific behavior. A release decision may require a wider packet. The answer should identify the candidate and the observation.

Use this question while reading the method accounts. Locate the work each arrangement organizes and the point at which a person interprets its output. The same question will connect the documentary descriptions to the case, the assurance chapter and the final discussion of future evaluation.

Questions for the rest of the book

The next chapter develops a practical response to development driven mainly by successive plausible outputs. Chapter 3 establishes the vocabulary used to compare processes. The four method chapters then examine selected, dated profiles through their own artifacts and decision paths. Chapter 8 gives Spec-Driven Development its cross-cutting treatment.

Chapters 9 and 10 have different jobs. The comparison chapter asks what the evidence supports. The selection chapter asks what a team should do under stated constraints, including when a narrow practice or small repair path is enough. The case chapter is where planned exercises must become reproducible records or remain explicitly labeled walkthroughs.

Across those chapters, keep the approval button in mind. A convincing screen gives us something to inspect. The engineering work is to establish which decisions produced it, which behavior it owes its users, and what evidence supports accepting it.

2. Beyond Vibe Coding

Define the behavior behind the label

An assistant produces a change. You try it, describe what looks wrong and ask for another version. That conversation can be a productive way to explore an unfamiliar problem. A prototype may reveal that a proposed interaction is confusing, that a requirement is incomplete or that the idea itself is unnecessary. Those are useful discoveries even if the first implementation is discarded.

The difficulty begins when a plausible result becomes an accepted change without a clear account of the decisions and evidence behind it. For this book, **vibe coding** names a working pattern: implementation advances through conversational intent while decisions, constraints and acceptance evidence are not consistently preserved. The definition describes behavior we can inspect. It does not classify a person, an editor or every use of conversation in development.

The relevant boundary is acceptance. What was acceptable as an experiment may need additional work before someone relies on it. A prototype's author might know that all the data are fictional and that a button bypasses several checks. A later maintainer, encountering the same code in a service, may not have that context. The engineering task is to decide what can be carried forward, what must change and which assumptions need a durable record.

Imagine that our vendor-access application has a history view. During exploration, a developer asks for a list that makes the screen easy to inspect. The assistant supplies synthetic events and a convenient route to reach them. The next request is to connect real business records. At that point, tenant scope and visibility are material decisions. The prototype's convenience does not establish the intended access rule.

We can examine this transition without judging whether the first conversation was disciplined enough. Ask what the next person can establish from the repository and the change record. Can they identify the behavior being requested? Can they distinguish a product decision from an implementation convenience? Can they find observations that bear on acceptance? A record that answers these questions may be compact.

Conversational coding and engineering discipline can coexist. A developer can discover a requirement in dialogue, summarize the decision in an issue, implement a constrained change and attach relevant checks. Conversely, an elaborate process can produce many files while leaving the decisive rule unresolved. The book evaluates the connection among intent, work and evidence.

That connection also sets a limit on the title's promise. Moving beyond the working pattern does not mean replacing every conversation with a document. It means making the accepted commitment understandable and reviewable. We will examine whether named frameworks help establish that connection, what they cost and where a smaller arrangement is enough.

From intent to an accepted change

Take this illustrative request: “Let a tenant member inspect the decision history for a vendor-access request.” In the common case, history must remain tenant-scoped and successful business transitions must have durable audit records. The request is therefore connected to CS-03 and CS-10 in the case specification. It does not authorize an application framework, a new persistence design or a production deployment.

Begin by separating what is agreed from what the sentence leaves open. The tenant boundary is part of the accepted case direction. The detailed disclosure policy and interface contract are still unresolved. A missing record and a foreign-tenant identifier need defined treatment before an executable acceptance check can assert the exact response. Choosing that response during implementation would turn an unrecorded assumption into product behavior.

The first useful output is a decision about the work's boundary. Which history is being exposed? Does this increment read existing audit events, or must it also establish the write path? Which successful transitions are included? What does the exercise exclude? Questions about operational log retention or real identity federation belong outside this small increment unless the common scope is explicitly changed.

Repository investigation supplies a different kind of answer. In an established application, the implementing engineer can inspect how identity is established, how requests are located and where audit events are stored.

Those observations describe the current implementation. They do not automatically settle the desired visibility rule. If an existing query crosses tenant boundaries, matching it would preserve the wrong behavior for this case.

Next, record the decisions needed before coding proceeds. Some can be resolved from current project guidance; some require the product owner. A useful question includes the evidence already found and the choice that remains. “The current reader accepts a request identifier without tenant scope; should history follow the case's tenant-local rule?” is more actionable than a general invitation to discuss security. In our case, the approved rule supplies the answer; the detailed interface still needs a contract.

Implementation then becomes a bounded attempt to realize that intent. Its record should identify the relevant changes and important deviations. If the assistant discovers that audit events do not survive restart, the scope of a read-only history change may no longer describe the work. That finding calls for a visible decision: repair the persistence behavior, split the work or stop the increment with the dependency unresolved. Quietly expanding the change would make effort and acceptance harder to interpret.

Acceptance needs an observation suited to each obligation. An own-tenant history read addresses useful behavior. A foreign-tenant read addresses the isolation boundary. A restart exercise addresses persistence. These observations establish different things; one cannot stand in for the others. Their fixture conditions, commands and outputs must be recorded when the exercise is actually executed.

A reviewer also needs to know the limits of the evidence. If only the reader has changed, existing audit-write checks may remain relevant, but their results must come from the baseline and environment being reviewed. A result from another revision cannot silently certify this change. If a check was not run, the record should say so and explain what acceptance remains unresolved.

Finally, someone makes the acceptance decision. That person needs the agreed intent, implementation boundary, relevant observations and open issues. “The assistant finished” describes an event in the workflow. Acceptance is a decision about whether the work meets the agreed conditions. The same person may implement and review a tiny change, but the record

should make that overlap visible rather than imply an independent assessment.

This sequence can happen within a short issue and a small pull request. Its usefulness depends on the decisions it preserves and the questions its evidence answers. The named framework comes later; first establish the work that any chosen process must support.

The smallest useful engineering record

A compact record needs enough information to distinguish an intended result from a plausible approximation. For a small change, five entries are a useful starting point: intent, constraints, change boundary, acceptance basis and disposition. The entries can live together in an issue or review description. They need not become five separate files.

Here is an original teaching example for the history increment. It is a planning record, not an executed run or a frozen interface contract:

Entry	Proposed content
Intent	A member can inspect the permitted decision history of a request in their own tenant
Constraints	Preserve tenant isolation and existing request decisions; expose no other tenant's history; retain durable audit obligations
Change boundary	Identify the history reader and its authorization/data-access path after repository inspection; do not add provisioning, billing or unrelated interface changes
Acceptance basis	Own-tenant positive read, foreign-tenant direct-identifier negative read, and relevant persistence/regression checks under the agreed disclosure contract
Disposition	Interface details unresolved; no implementation or acceptance checks executed

The last row matters. It prevents a well-formed record from looking like completed work. After execution, that row could link to real observations and an acceptance decision. Until then, the document describes obligations. A future report must preserve the difference.

Each entry should carry a decision rather than a generic assurance. “Follow best practices” leaves the reviewer to invent the applicable rules. “History reads use the authenticated tenant boundary, including direct identifiers” identifies behavior to inspect. “All tests pass” omits which tests, on which revision, and what they establish. A link to a particular command/output record is more informative when execution exists.

The acceptance basis should also expose why a check could fail for the wrong reason. A foreign-tenant read cannot establish isolation if the request identifier is absent from every tenant. The fixture needs a real record in tenant A, a permitted A reader and a B reader attempting the corresponding access. The disclosure contract determines the allowed response; the check must still distinguish isolation from an unusable reader.

Apply the same care to self-approval. An already completed request might reject every new decision, masking an implementation that would allow its requester to approve a pending item. The negative example must begin with a pending request and a requester who otherwise has the required role. A separate equivalent fixture establishes that an eligible different person can approve. These are proposed design conditions, not tests we have run.

The change boundary is equally important. An assistant may propose useful cleanup while inspecting the reader. A reviewer needs to decide whether that work belongs in this increment. Deferring it can preserve a clear acceptance question. Including it may be justified if the intended behavior cannot be implemented without it. Record the reason so the diff remains understandable.

Keep the record close to the work that consumes it. If a reviewer must reconstruct the accepted rule from a long chat, a link to the chat is not an adequate handoff by itself. Summarize the consequential decision and cite the supporting discussion where useful. If the rule already lives in a maintained requirement, link it and describe the change rather than create a second version with different wording.

During review, use the record in both directions. From intent, ask where the behavior is realized and observed. From the diff, ask which requirement or necessary supporting change explains each material addition. A modified reader justified by tenant isolation has a different status from an unrelated persistence rewrite. Both might be valid work, but their reasons and acceptance boundaries should be explicit.

A compact record can evolve as facts emerge. Preserve the initial uncertainty and its resolution where they explain a consequential choice. For a routine detail, a short final rationale may be enough. For a scope change that affects acceptance or evaluation, a dated decision and explicit deviation are more useful than silently replacing the old request.

We will use this record as a minimum question set, not a required template for every method. A native story, phase plan or use-case record may carry the same information differently. Chapter 3 establishes how to compare those arrangements without requiring identical documents.

A practical record can also name its stopping point. If investigation depends on an unresolved disclosure rule, record that dependency and the owner who can resolve it. The next contributor can continue the authorized investigation while keeping the dependent implementation decision open.

When ceremony costs more than it helps

Now reduce the problem. Assume an existing, reviewed interface has a navigation link labeled “View history.” A small copy change asks for “View decision history.” The destination, visibility policy and underlying query must remain unchanged. This is a hypothetical interface repair; the book's application and optional UI have not been built.

The intent is clear under those assumptions. The implementing engineer should first confirm that the change is confined to presentation and identify any related accessible name or interaction expectation. If inspection reveals that the link also changes a route or permission condition, the initial description is incomplete and needs revision.

For the bounded copy change, a short issue, constrained diff and relevant navigation check could carry the necessary record. A new product vision, domain model and architecture review would need a concrete reason. Requiring them merely because a framework can generate them would add activities without identifying another decision to make.

BMAD's inspected guidance explicitly allows trivial edits to bypass its process, while offering a bounded Build path for other small changes. That is documentary evidence that at least this source profile allows proportionate use; it is not a measured estimate of effort saved. Small-change guidance, lines 15-23

The counterexample tests the book's thesis. If the useful decision record already exists and the change preserves it, additional ceremony may supply little new information. We should be willing to record that result in a future exercise. A framework does not earn an advantage because it generated more artifacts for the same accepted behavior.

The comparison must count the whole relevant effort. A fast initial edit followed by a long investigation of unintended changes is different from a fast accepted repair. An extensive setup used for many later changes is also different from a setup repeated for each one. State what cost is being allocated and which later work the observation includes.

One practical check is to ask what each proposed activity will decide. If a new architecture document would merely repeat an unchanged route and query boundary, reuse the accepted context. If a review of the label reveals an unexpected navigation change, inspect and resolve that change. The activity earns its place through the uncertainty it addresses, rather than through a standard document count.

An evaluation record can distinguish one-time setup, task-specific preparation, implementation, acceptance work and repair without assigning invented durations. Leave a category unmeasured until instrumentation exists. When tasks overlap, record the overlap rather than add worker durations and call the sum elapsed time. This prepares an honest cost account while leaving the tiny repair itself small.

Do not assume the opposite result either. A compact repair may depend on a costly understanding of the existing system. The developer who already knows the route, tests and authorization boundary brings context to the task. A new operator may need more investigation before the same small diff is defensible. Line count alone does not describe that difference.

Process size should follow unresolved decisions, consequences and the available evidence. A short request can change a business policy; a longer presentation update can preserve one. If the record shows no material policy or technical uncertainty, choose a small path. If inspection exposes one, enlarge the process for that reason and preserve the discovery.

The method chapters will therefore examine both ordinary increments and bounded repairs. We will not force a large planning route onto a method that offers a smaller one, or remove a required activity to make another method

appear cheaper. The evaluated path must be a recognizable configuration of the process being discussed.

Choose the smallest record by looking at the cost of being wrong

A maintainer can change an explanatory comment after checking that it describes the current behavior. The issue and diff may carry everything a reviewer needs. A change to a shared lookup has a wider effect. The issue should name the intended tenant rule, the paths using the helper and the control that demonstrates authorized reads continue to work. The extra record earns its keep because a reviewer can use it to find a missed path.

When a team debates how much process to apply, name the failure it wants to detect. If the concern is an unstated product decision, assign an owner and record the answer. If the concern is an unnoticed side effect, design an observation where the effect would appear. If the concern is an unattended restart, identify the record the next operator needs. The result can be a small set of concrete duties that already fit the team's review practice.

What evidence can establish

A demonstration, a generated test suite and an independently assessed acceptance result answer different questions. A demonstration shows a particular interaction under particular conditions. A suite establishes its assertions for the executed revision and environment. An acceptance judgment relates observations to agreed behavior. None becomes broader merely because its output is confidently phrased.

Consider the history view again. Showing Mira her own request history is useful evidence about the positive path. It leaves foreign-tenant access unresolved. A negative check against an invalid identifier leaves the same gap. A suitable isolation check needs the relevant record and actors, plus an agreed interpretation of what the interface may disclose. Evidence becomes meaningful through that connection to the requirement.

Generated checks can contribute, but the origin of the expectation matters. If the assistant reads an implementation that mistakenly permits administrator self-approval, it might write a test that preserves that behavior. The test could pass while contradicting CS-05. Derive the acceptance expectation from the agreed requirement, then inspect whether the check actually exercises it.

We will call that expectation and its decision procedure the **acceptance oracle**. In a small exercise it may combine assertions, fixtures and a reviewer rule for ambiguous cases. Independence concerns its basis and review, not simply a different file or model invocation. The eventual case needs frozen, independently reviewed checks; none exists yet.

Evidence also needs a defined scope when making effectiveness claims. METR's July 2025 report describes a randomized study involving 16 experienced open-source developers and 246 issues, and reports that AI-allowed work took 19% longer in that setting. The finding concerns those developers, tasks and early-2025 tools. It does not compare our four frameworks or establish a universal slowdown. METR's original study report

METR's February 2026 update then warns that participation, task-selection and concurrent-agent timing issues make its later experiment an unreliable signal of the current productivity effect. We cannot turn those later estimates into a clean trend line. METR's study-design update

These sources illustrate why scope and uncertainty matter. They do not settle whether a process improves our case. Nor would a successful synthetic SaaS exercise establish organization-wide benefit. The book needs to state which intervention was observed, which conditions were held constant and what alternative explanations remain.

Preserve missing and adverse evidence as carefully as success. A failed installation, an unexecuted check or a task stopped at a resource ceiling affects what the reader can conclude. Label it at the appropriate level. An environment failure may leave business behavior unassessed; a failed business check may establish a defect under its fixture. Combining both into an unexplained score would hide the difference.

Finally, keep documentary evidence in its proper role. A source can prescribe a human approval or a traceability check. That supports a claim about design. An execution record is needed to show that the activity occurred, and further assessment is needed to establish whether it was effective. This distinction will appear throughout the four deep dives because the same words can otherwise imply much more than the evidence supports.

Match the completion claim to the entry point

A command handler can pass a unit check while its HTTP route supplies the wrong tenant. When reporting completion, name the entry point the observation exercised and the inputs supplied there. For a local function, record the function and fixture. For an integrated path, identify how identity, tenant and request state reached the decision. This small distinction helps the next reviewer decide which claim the output supports.

From habits to repeatable processes

The habits developed here form a small engineering loop: make intent explicit, investigate the relevant context, settle material decisions, bound the change, collect suitable observations and record acceptance or its limits. Repeatability means another person can understand how the work was arranged and examine its evidence. It does not promise identical generated text or code.

A named process may assign these activities to specialized roles, organize them into phases or maintain specifications through change. It may add useful checks and make some decisions easier to recover. It also introduces dependencies and work that deserve examination. The next chapter gives us a common vocabulary for that evaluation.

Before applying a framework, identify the point at which the current arrangement loses information. Does the agreed rule vanish between conversation and planning? Does implementation proceed while a product question remains open? Do acceptance checks reproduce the implementation's assumptions? Does the next session recover the decision, or infer it from stale code? These are questions to investigate in a particular project, not a diagnosis of every informal workflow.

Then ask how the proposed process addresses that point. An artifact needs an owner and a maintenance rule. A gate needs a decision-maker and evidence. A review needs an inspectable scope. A short path needs a way to notice when its assumptions no longer hold. These questions let us assess mechanisms before assigning them a benefit.

The compact record is available even if a team adopts no named methodology. The four frameworks provide larger arrangements to examine

against the same obligations. We will preserve their native vocabulary while asking what decisions move through it and what the evidence establishes.

Carry one practical question into chapter 3: for this change, what would another engineer need to know before accepting responsibility for it? The answer sets the minimum useful process. A framework should help the team meet that obligation under its actual constraints, and its suitability remains a conclusion to earn.

3. Understanding AI-Driven Methodologies

The comparison unit

“We used the same AI method” is an incomplete description of an engineering exercise. One team may have enabled optional test architecture, another may have used a short repair path, and a third may have followed a different generation of the documentation. Their work can share a name while involving different instructions, artifacts and decision rights.

The comparison unit in this book is a **named source snapshot and implementation profile, applied under stated operating conditions**. A profile identifies the relevant route, required activities, optional modules and settings. Operating conditions identify the task, starting repository, host, model access, operator and resource limits. Together they describe what was actually available to influence the work.

Start with a source identity. A repository commit gives a fixed documentary reference. A package version names a distributed artifact, whose contents still need to be checked for an executable exercise. A branch name can move. A product version may bundle process instructions with host behavior. Record these identities at the level needed to understand and reproduce the intervention.

Then describe configuration. An approval requested in a prompt, a hook that blocks an operation and a human review outside the tool are different mechanisms. A profile may enable one and omit another. The evaluation must identify the actual arrangement before interpreting a completed stage or a green status.

The following record defines the unit without prescribing a particular methodology:

Field	What the reader needs
Method/source identity	Name, repository or publication, exact source revision and relevant passages

Field	What the reader needs
Installed implementation	Package/plugin identity and contents used, if execution occurs
Profile and modules	Selected route, required steps, enabled add-ons and material overrides
Host/model conditions	Execution surface, model identification available to the operator, worker/tool permissions and relevant settings
Business task	Accepted requirements, exclusions and the specific episode being attempted
Starting state	Baseline revision, fixtures, existing application behavior and reset procedure
People and resources	Operator familiarity, decision owners, reviewer arrangements and resource ceiling
Evidence boundary	Documentary walkthrough or actual execution; which conclusions the records can support

These fields also help separate a concept from one implementation. AWS's July 2025 introduction describes inception, construction and operations. The pinned workflow implementation examined for this book includes initialization and ideation as well. The names therefore need source qualifiers; the difference does not establish that one source is defective. Original AI-DLC account, inspected implementation phases

A similar caution applies to a host change. If the model, available tools and reviewer dispatch all change with the workflow, a different outcome cannot be attributed to the method alone. We may still learn something about that configured arrangement. The report must identify the combined intervention and preserve the limitation.

The source snapshots used here are research references, not a completed installation freeze. The GSD chapter retains the author's requested title while contemporary Core sources provide a dated implementation reference. The final edition/lineage choice and exact run packages remain open. This visible boundary is preferable to silently applying one generation's behavior to every use of a name.

Methods, approaches and tools

For this book, a method or process framework arranges decisions, roles, artifacts and feedback across development work. An approach describes a way to organize some of that work. An execution tool supplies capabilities through which the work happens. These are working distinctions for comparison, not universal labels that every project must adopt.

A single package can contain substantial process guidance and executable tooling. An editor can supply specification workflows. A framework can be distributed as skills or templates. We therefore need to inspect what each component does rather than classify it by its installation format.

The accepted coverage has four framework chapters and a cross-cutting approach:

Coverage	Role in the book	Question the classification helps answer
BMAD Method	Framework deep dive, chapter 4	How does a selected path size planning, carry intent and organize implementation/review?
GSD framework lineage	Framework deep dive, chapter 5	How does the selected implementation organize phases, context and verification?
AWS AI-DLC	Lifecycle/framework deep dive, chapter 6	How do concept, implementation and configured profile assign work and authority?
AI Unified Process (AIUP)	Framework deep dive, chapter 7	How do specifications, use cases and selected construction paths govern implementation and change?
Spec-Driven Development (SDD)	Cross-cutting approach, chapter 8	What role do specifications play before, during and after implementation?
Coding hosts, IDEs and implementation toolkits	Execution and process context	Which capabilities, dependencies and constraints belong to the selected arrangement?

The four frameworks are included because they are the book's chosen subjects, not because every other toolkit lacks a methodology. Excluding a toolkit from peer coverage is an editorial scope decision. When a toolkit contains planning or change rules, acknowledge those rules rather than reduce the toolkit to a command-line interface.

The source material gives us concrete reasons to inspect each framework. BMAD's planning guide distinguishes defined intent from work that needs further planning and provides different paths according to that need. That is a process choice, not merely an editor feature. BMAD planning-path guidance

The inspected GSD Core guide describes discussion decisions carried in phase context, plans, execution, verification and shipment. Its artifacts connect activities across sessions. This supports examining the selected process design; it does not establish its claims about reduced rework or context quality. GSD Core phase loop

AWS's implementation makes configured routes consequential: its Classic and Express descriptions differ in stage selection, artifacts, reviews and guard behavior. The lifecycle concept and a particular route through that implementation should therefore be recorded separately. Inspected workflow profiles

AIUP's workflow maps requirements, use cases, architecture decisions, test journeys and acceptance records to their producers and consumers. That artifact flow supplies a process subject for analysis, while construction adapters introduce further scope and dependencies. AIUP artifact flow

These descriptions establish the presence of process mechanisms in the selected sources. They do not establish equivalent lifecycle coverage, equal effort or effectiveness. One framework's phase may encompass work that another splits across stories or use cases. Comparing the count of those units would answer a naming question rather than an engineering one.

The implementation layer still matters. A process that expects worker dispatch depends on a host's relevant capabilities. A stack-specific construction adapter may constrain the application architecture. Those dependencies belong in the comparison record. They should not quietly become universal properties of the method, or disappear because the method describes itself as portable.

A short specification primer

In this book, a specification is an identified description of required behavior or constraints, with a known scope and change policy. It may be concise and written in natural language. Its engineering role depends on what decisions it carries, who maintains it and how it relates to implementation and acceptance.

A long prompt does not acquire that role through length. A short issue can serve as a useful behavioral contract if it identifies the requirement, preserves material constraints and has a clear disposition. Conversely, an extensive document can remain ambiguous about who may approve a change or which behavior should prevail on conflict.

Birgitta Böckeler's 2025 analysis distinguishes spec-first, spec-anchored and spec-as-source patterns: respectively, specifications used before implementation, retained for later evolution, and treated as the human-edited source from which implementation is produced. We use the vocabulary with attribution as a lifecycle lens, not an effectiveness ranking. Böckeler's SDD analysis

For the deep dives, four practical questions are enough to begin:

Policy question	Why the answer matters for a change
Who owns the specification?	Identifies who may change the accepted behavior and by which route
What is retained?	Distinguishes a temporary implementation instruction from a maintained product record
What happens on disagreement?	Identifies whether the intent, specification, code or check needs correction
How is alignment assessed?	Separates links and assertions from observations that address the required behavior

Consider the proposed approval-policy change. “High-risk requests need two approvals” leaves the treatment of existing requests unresolved. The case's agreed direction binds requests to the policy active at creation. A useful specification change needs to carry that decision as well as the new count.

Otherwise two implementations could follow the short sentence and disagree materially about existing v1 requests.

Ownership is equally concrete. BMAD's inspected specification guide directs updates to its named `SPEC.md` through `bmad-spec`, preserving capability identifiers. That policy concerns a particular artifact; it does not tell us that every specification in every framework has the same writer. BMAD specification ownership

AIUP's inspected change guidance starts with changed intent or requirements, then reconciles related specifications and implementation/tests through the selected construction path. This is a documented maintenance policy. Its existence does not prove that every later implementation stays aligned. AIUP change guidance

The phrase “source of truth” needs the same precision. Truth for which decision? If a business rule and generated endpoint disagree, who resolves the conflict? If an implementation exposes a previously unknown constraint, how does that finding change the specification? A declaration of authority supplies no synchronization mechanism by itself.

We also reserve **executable specification** for an artifact with an identified mechanical execution or validation path. A natural-language instruction that an agent interprets is not enough to establish that property. Nor does generation imply deterministic output: a reproducible exercise needs inputs, configuration and observations, even when repeated outputs differ.

Chapter 8 examines specification policies in depth. For now, retain the distinction between using a specification at the start, maintaining it through change and governing how implementation may be edited. A framework can combine these policies in different ways across artifacts. We will inspect the actual rules before applying a broad label.

The common SaaS problem

The common case is a synthetic B2B service for recording vendor-access requests and their decisions. A requester asks for temporary access for a vendor. Eligible approvers in the same customer tenant decide the request. The service preserves decision history; it does not provision access in an external system. This boundary keeps the business problem meaningful without requiring live vendor accounts or infrastructure.

A **tenant** is the customer-organization boundary in the exercise. Tenant A's membership, requests and history must not become available to a tenant B actor through the business interface. The test fixtures will contain both tenants so the boundary can be examined. No real customer or vendor data are needed.

An actor may hold more than one role. A requester with approver or administrator privileges remains disqualified from approving their own request. Approval also requires current eligible membership in the request's tenant. A role label alone therefore does not settle a decision; actor identity, target request and current membership all matter to the agreed behavior.

Under the base policy, one eligible distinct approver can complete a pending request. A permitted rejection is terminal. Repeated decision requests need a defined idempotency contract, and concurrent decisions must preserve the policy and one valid terminal outcome. Successful transitions need durable, tenant-scoped audit history. The detailed requirements and unresolved interface choices are in the case specification.

The shared increment can be expressed without selecting a framework's native artifacts:

Neutral business brief	Agreed direction
Useful behavior	Create a valid request, obtain an eligible decision, inspect the permitted request/history
Important boundaries	Identity and current membership, tenant isolation, no self-approval and consistent terminal behavior
State/evidence obligations	Defined retries and concurrency; durable business state and audit history
Technology family	TypeScript / Node.js; exact framework, database, compiler and runtime versions remain open
Exclusions	External provisioning, billing, outbound email, production federation and regulatory certification
Acceptance status	Requirements are design obligations; no application or independent executable checks have been completed

The actors and rules are common; native planning artifacts remain different. BMAD may carry the intent in its specification/story arrangement. A GSD path may use phase decisions and plans. AIUP may organize behavior through use-case artifacts. Requiring every arm to use one of these structures would impose part of that framework on the others.

We plan three episodes. **E1** implements the base increment from the same neutral scaffold. **E2** changes the policy for newly created high-risk requests to require two distinct eligible approvers, while existing v1 requests keep their bound policy and standard-risk v2 requests still need one. **E3** repairs a predetermined tenant-boundary defect in a shared prepared baseline. They examine creation, change and repair rather than repeatedly scoring a greenfield demonstration.

Each episode needs its own matched starting condition. In particular, E2 should not begin from each framework's different E1 output if we intend to compare the change procedure under the same baseline. Those prior outputs could differ in structure, defects and decisions. A longitudinal study of maintaining each output would be a different design and should be reported separately.

The external acceptance boundary should also remain common. A method may generate its own checks and reviews, but the independent case obligations still ask whether the same business behavior holds. Preserve the native artifacts alongside the common requirement/check mapping. That lets us inspect how each process carries the work without demanding identical prose or code.

This design is proposed, not executed. Choosing a common application stack may expose adapter constraints; choosing a host may alter reviewer or worker capabilities. Those issues need explicit disposition before runs. Chapter 11 owns the complete contract, baselines, commands and observations if execution occurs. The short case here supplies the business vocabulary needed to understand the four deep dives.

Questions applied to every framework

The comparison instrument contains eleven dimensions, F01-F11. They organize questions about mechanisms and outcomes. They are not eleven scores, and a strong answer in one dimension cannot silently compensate for an unacceptable boundary in another.

Use the same questions while preserving native terminology. A framework may call an acceptance activity verification, review or a gate. The relevant question is who makes which decision, using what evidence, under which configuration. Names and counts are starting points for investigation.

ID	Dimension	Common question	Evidence that would strengthen an outcome claim
F01	Decision rights	Who can propose, approve, change scope or stop work, and how does the selected gate operate?	Recorded approvals, bypass behavior, interventions and unresolved decisions
F02	Planning overhead and scale	What work is required for a repair, feature or uncertain initiative?	Setup, planning, review and relevant rework under stated scope
F03	Artifact lifecycle	What is created, retained, retired and reconciled; who owns it?	Artifact changes and disagreements through a maintenance episode
F04	Requirements traceability	How do requirements reach plans, implementation, checks and acceptance?	Inspected valid, missing and semantically incorrect links
F05	Context management	What information reaches each session or worker, and what survives interruption?	Recorded inputs, omissions, recovery and resulting corrections
F06	Solo/team operation	How are business, design, implementation and review duties assigned and handed off?	Observed human coordination under disclosed team conditions
F07	Brownfield suitability	How is existing behavior discovered and the intended change bounded?	Shared change/repair baselines and regression observations

ID	Dimension	Common question	Evidence that would strengthen an outcome claim
F08	Testing and acceptance	Which checks are prescribed, who owns the oracle and how is disagreement handled?	Independently reviewed checks and actual command/output records
F09	Deployment governance and operations	Which release, observability, recovery and service-ownership duties are covered?	Environment-specific operational evidence and reviewed limits
F10	Portability and dependencies	What depends on a host, model, adapter, stack or service?	Defined compatibility/migration observations and disclosed workarounds
F11	Maintainability	Who keeps specifications, code and generated assets aligned after change?	Follow-up changes, drift findings and repair effort over time

The dimensions overlap where the underlying work overlaps. An outdated specification concerns artifact lifecycle, traceability and maintenance. Record the event once, then explain its relevance to each question. Repeating the event in several columns does not create several independent observations.

Required and optional mechanisms need separate treatment. A mandatory review contributes to the selected route's work. An optional module contributes only when enabled. An external reviewer or tool may be necessary to satisfy the book's acceptance boundary without being part of the framework core. Describe that arrangement rather than attributing all resulting coverage to the method.

Take F04 as a worked example. Suppose a hypothetical plan links a capability to CS-05. The implementation contains a corresponding marker, and a test cites the same identifier. The references establish presence. They do not establish that the test prevents self-approval. If it targets a completed request, it could succeed through the wrong rule. Completeness and semantic correctness require further inspection.

F01 presents a different distinction. A document that asks an operator to approve a plan supports a claim that approval is prescribed. A recorded pause and response can show that approval occurred in a run. A configured block

with bypass tests can establish something about enforcement in that environment. These are progressively different questions; no adjective in the workflow name answers them all.

For F02, report what effort includes. Installation, familiarization, artifact preparation, active human work, waiting and repair may be allocated differently. Agent duration and wall-clock duration can overlap. A single unexplained completion time would make comparisons hard to interpret. The future run protocol must define these observations before measurement, not after a favorable result appears.

For F06 and F09, respect the scale of the exercise. One author playing several roles can demonstrate a proposed handoff but cannot establish a human team's coordination benefit. A local service can expose useful behavior but does not supply evidence of real incident response, recovery practice or production ownership. Those dimensions may remain unassessed even when the local exercise succeeds.

An unanswered dimension is not an automatic failure. A process could be suitable for a bounded task while leaving an operational duty to the organization's existing practice. The comparison must identify that dependency and decide whether it is acceptable for the scenario. Conversely, a documented feature should not receive an effectiveness rating merely because it is relevant.

Chapter 9 will synthesize findings under these questions. Here, their role is to make the deep dives inspectable. The research comparison framework retains the full instrument, evidence categories and reporting rules.

Apply the instrument in a consistent order

Begin an inspection with the selected method identity, then the task and its accepted boundary. Ask how the workflow obtains decisions, carries them into work and interprets the resulting checks. Finish with the dependencies and duties supplied by the surrounding organization. That order reduces the chance of praising a verification mechanism before establishing what it was supposed to verify.

For a small existing-code repair, some lifecycle questions may have little work behind them. Record the reason. An operating environment supplied by an organization can remain outside the repair while still constraining release. A

larger feature may need explicit design and operational preparation. The same instrument can describe both arrangements while preserving the difference in scope.

Keep the completed instrument with the source revision and scenario. Another reviewer can then dispute a particular interpretation and identify the evidence that would resolve it. Updating one assessment becomes possible without rewriting the framework's identity or importing a conclusion from an unrelated task.

Reading claims and observations

Every substantive statement in a comparison needs an evidence type. This is especially important when documentation mixes an instruction with an expected benefit. “Use a fresh session” describes a mechanism. “Fresh sessions prevent relevant errors” is an effectiveness claim. The first does not establish the second.

The inspected GSD Core phase-loop guide provides a concrete example: it describes fresh executor contexts, names a 200k-token window and attributes benefits to that arrangement. Our documentary conclusion is that this source prescribes a context strategy. Actual window availability depends on the selected host/model arrangement, and effectiveness requires observations of what context arrived and what errors occurred. We do not turn the documentation's number into a portable capacity guarantee. GSD Core context description within the phase loop

This is the reading discipline we apply to favorable and unfavorable claims alike. Preserve the mechanism, identify its dependencies, and ask what additional evidence would establish the stated consequence. A source-reported limitation also needs its exact version and scope before it can rule out another configuration.

We distinguish the following categories:

Evidence type	What it can establish within its scope	Question to retain
Documented design	A selected source prescribes a workflow, artifact or dependency	Which version/profile, and was actual behavior checked?

Evidence type	What it can establish within its scope	Question to retain
Source-reported outcome	A maintainer or vendor reports a result	Under what conditions, with what records and interests?
Field observation	A practitioner reports what happened in a particular setting	Which task/configuration, what failures, and how transferable is it?
Controlled observation	An executed comparison records outcomes under a disclosed design	What was held constant, what varied, and which confounders remain?
Editorial analysis	A reasoned interpretation or conditional recommendation	Which premises and evidence would change the conclusion?

An illustrative walkthrough is a separate presentation mode. It can explain a procedure or expose a design ambiguity. It does not become a field observation because the story uses names and realistic business rules. Our unexecuted vendor-access examples remain in that mode until actual records exist.

The categories are not a simple quality ladder. Primary documentation is often the best evidence for a prescribed writer policy. A controlled experiment may be more relevant to an effect under its tested conditions, while telling us little about another release. A field report can reveal a concrete failure that a broader study did not examine. Match the evidence to the claim.

Interests also need disclosure. A maintainer may understand an implementation deeply and still benefit from its adoption. A practitioner may have invested in a particular workflow. An external analysis may use a judgment-based rubric rather than measure outcomes. These conditions affect interpretation; they do not justify ignoring the source or treating its assertions as independently established.

The same precision applies to absence and uncertainty:

Statement	Appropriate interpretation
Not assessed	The current work does not establish an answer
Not found in inspected scope	The bounded inspection did not locate the mechanism; wider coverage may differ

Statement	Appropriate interpretation
Outside the selected profile	The described capability is not part of the configuration being evaluated
Absent in a defined scope	A sufficiently complete check supports a missing-capability claim for that scope

Do not convert those states into a common zero. A missing test result differs from an executed failed test. An unreviewed adapter differs from a demonstrated incompatibility. A documented optional capability differs from something the base route produces. Preserving those distinctions makes a later decision more defensible than filling a table with intuitive ratings.

Version qualifiers need to survive paraphrase. If a source describes a particular adapter, a summary that drops the adapter can overstate the method. If an original article describes a concept, a current implementation's phase count should not be substituted without explanation. Trace the claim back to the source and record the exact scope it supports.

For outcomes, preserve failures, deviations and stopping conditions. A trial that stops during installation has not tested the business behavior. A successful run after substantial manual redesign is evidence about the assisted arrangement, including that intervention. Removing inconvenient attempts or omitting the redesign would alter the apparent result.

Reproducibility also has a bounded meaning here. The record should permit another person to recover the starting conditions, follow the procedure and compare observations. It need not promise identical model output. When model identity, access or behavior cannot be fully fixed, disclose that limit rather than claim exact repeatability.

These rules let a reader challenge the argument at the appropriate point: source interpretation, design, execution, inference or recommendation. The book's research protocol records the corresponding source, interest, rights and claim-handling duties.

Give each claim the evidence it asks for

A maintainer may document a route for small changes. Reading the relevant revision can support a statement about that route. To say a team used it on the tenant repair, the investigator needs records of the configured workflow and the repair. To say the route reduced effort, the investigator needs an accounting policy and a comparison design. A reader should be able to tell when an author moves between these levels.

One practical reading habit is to underline the verb in a claim. “Describes,” “requires,” “blocked” and “improved” demand different observations. A written requirement can be inspected in the document. A blocked action needs an executed attempt and a record of what prevented it. An improvement needs an explicit reference point and a way to handle changes in task difficulty or operator experience. Keep the claim local when the available evidence is local.

A source locator is an inspection aid. Open the cited revision and read the passage around the claim. Check whether the prose describes a requirement, an optional path or an observed result. If the claim depends on a selected adapter or setting, keep that condition in the assessment. This makes source review repeatable at the level of an individual statement.

How to read the four deep dives

Read each framework chapter as an examination of a selected arrangement. Begin with its scope note, then follow the common business intent through its native decisions and artifacts. The chapter should make clear what the source prescribes, what the teaching example proposes and what has actually been observed.

Use five questions as a reading guide:

1. What intervention is being described: source, profile, modules and host assumptions?
2. Which unresolved decisions must be settled before implementation, and who settles them?
3. What carries those decisions into work and through a later change?
4. Which checks support acceptance, and what do they leave unresolved?

5. Under which conditions would this arrangement merit investigation, adaptation or rejection?

The native artifact trail should differ. A BMAD specification is not automatically an AIUP use case under another filename. A GSD phase is not an AWS lifecycle phase of equal size. Preserve what makes the mechanism distinct, then use the common questions to examine its role.

The chapters should also give small changes a fair treatment. A minimal route must retain its relevant obligations; a larger route should be used when the selected process and task call for it. The comparison should not force every framework through its longest demonstration or silently omit a required step. Optional enhancements remain visible configuration choices.

Source caveats are part of the explanation. A framework can change while the reviewed source stays fixed. A host can add capabilities while an older adapter remains limited. The useful question is which statement holds for the arrangement under discussion. Edition notes and exact references let the reader distinguish a documented historical design from a current environment they may later evaluate.

After the four deep dives, chapter 8 expands specification lifecycle policies. Chapter 9 brings the evidence together; chapter 10 turns supported constraints and uncertainties into conditional choices. Chapter 11 owns the complete case records. This order gives the reader a common vocabulary before asking for a comparative judgment.

The result should be a comparison a reader can inspect and disagree with constructively. A missing observation stays visible. A claimed advantage has a source and scope. A recommendation names its premises. With that contract in place, we can begin BMAD's deep dive without treating its vocabulary as the default for every method that follows.

Before moving to chapter 4, choose one maintenance task from a project you know. Describe its accepted behavior, who can settle ambiguity and one observation that would change your decision about completion. Keep the description method-neutral. As you read each account, identify where those three items would live and which dependencies the proposed arrangement would add. This exercise produces a local comparison question that can survive differences in vocabulary.

4. BMAD Method - Deep Dive

Define the BMAD intervention

The first decision in a BMAD evaluation is which BMAD we mean. A framework name is an incomplete description of an intervention. We need the source revision, the enabled modules, the coding host and the path selected for the work. Those choices determine what instructions and checks an operator actually encounters.

This chapter examines the stable-source candidate reviewed for the book, at commit `bab9082`…. Its package manifest identifies version 6.12.4. The pin fixes which source we discuss; registry contents and installation behavior require separate checks. Inspected manifest, lines 1-8

Within that boundary, we will follow the planning, specification, Build and completed-work testing guidance. Built-in QA is included in the discussion. TEA is a separate optional module, whose scope we distinguish from the base configuration.

Our business problem is the vendor-access request from the common case. A tenant member submits a request, an eligible different member decides it, and the service preserves the decision. The example has just enough policy to expose a dangerous shortcut: an administrator who submitted a request remains disqualified from approving that same item. The case has a TypeScript / Node.js direction, but that choice does not select an application framework or database.

The intervention can now be stated precisely enough to investigate:

Item	Scope of this chapter
Source	Pinned BMAD source identifying manifest 6.12.4
Workflow material	Planning path, specification, Build and existing-code guidance
Test-generation scope	Built-in QA described; TEA remains an optional alternative, unexecuted
Business slice	Tenant-local request/decision behavior and a later approval-policy change

Item	Scope of this chapter
Evidence	Inspected documentation, author-designed examples and explicit evaluation questions
Outcomes	Execution, effort, defect detection and maintenance benefit: not assessed

Record optional modules and host behavior separately. An instruction to perform a review establishes the intended workflow; an execution record must establish whether and how the review happened.

Choose how much planning the change needs

BMAD's selected planning guide begins with the state of the intent. When the intent is defined, it can be shaped into a specification and built; when important intent is missing, planning activities help establish it. Those activities are selectable aids rather than a compulsory sequence of stages. The guide also permits an obvious low-risk edit without invoking BMAD. Planning-path guidance, lines 8-65

The practical question is what remains undecided. “Build vendor approvals” is not a defined intent for our exercise. It leaves the actor, tenant boundary, number of approvals, terminal behavior and existing-request policy open. “Reject self-approval even when the requester holds an administrator role, preserving other valid decisions” is a much narrower instruction. It still requires investigation of the existing application, but the product rule is clear.

For the first increment, we would give the process the agreed business behavior and identify unresolved interface details. We would not treat unanswered questions as implementation freedom. The idempotency-key contract, for example, affects what a retry means. Its answer belongs in the agreed behavior before the implementation is assessed against it.

For a small maintenance change, the selected Build documentation describes investigation before asking the user for information the repository can settle. It then distinguishes a light route from a written-plan route using missing intent, irreversible actions and the footprint of the change. Flagged designs require questions to be answered and a plan approved. Build investigation and routing, lines 62-81

That route is a documented decision procedure, not a certified risk assessment. A reviewer must still examine whether the system identified the relevant uncertainty. If the proposed design says that self-approval is a user-interface concern, it has misunderstood a business-interface obligation in the case. Taking a light path after that misunderstanding would not settle the requirement.

We can make the routing judgment inspectable with a short, original planning note:

Question for the proposed increment	Current case answer	Consequence for the operator
Is the actor rule agreed?	Yes: current same-tenant approver, excluding the requester	Carry CS-04 and CS-05 into the work
Is the retry behavior fully specified?	Not yet	Resolve the key/conflict contract before executable acceptance
Is external access provisioned?	No	Keep external systems outside the change
Is existing data affected?	Depends on the later policy episode	Make version binding explicit before that episode

This original teaching note identifies decisions for a future route review. It is not output from a BMAD run.

Planning must also have an endpoint. Once the relevant intent is defined, repeatedly asking for a new product document would add work without necessarily adding a decision. Conversely, a short request does not make a change low risk. A one-line instruction that changes the meaning of existing approvals may require more investigation than a larger, isolated formatting change.

Stress the route with a concrete change boundary

Consider three proposed requests for the same application. Correcting a label leaves policy and stored state alone. Moving an eligibility condition into a shared decision path may change several callers even if the diff is short. Introducing creation-time policy binding may affect existing records and require a migration whose behavior has not been agreed. These are different settled designs, not merely different amounts of text in a prompt.

For each, write a routing justification after discovery. Identify what is already known, what the operator would notice if guessed incorrectly, which actions affect durable state and where the change reaches beyond the apparent entry point. A statement such as small fix is not that justification. The reviewer should be able to challenge it using repository evidence.

Our v2 increment contains a useful trap: changing the current threshold to two might appear to implement the headline request. It would not preserve v1 semantics if every pending request reads that threshold dynamically. Discovery must therefore inspect the decision dependency, not merely locate the number one. The product rule is settled; the representation and migration remain technical questions.

If discovery reveals a new uncertainty, update the route before treating implementation progress as acceptance. A light-path decision need not be permanent. Equally, discovering a well-established convention can remove a technical question without reopening an agreed business rule. Record what changed the judgment so later evaluation can distinguish useful investigation from repeated ceremony.

Choose a planning route from the unresolved work

An illustrative repair begins with a precise issue, an accepted response contract and a regression check that can reproduce the failure. The engineer knows which shared lookup is responsible and which callers need to keep working. A focused plan can identify the affected paths, the intended change and the observations required for acceptance. The review asks whether anything material remains undecided.

Now change one premise. The reported behavior is a failure only if the organization wants foreign identifiers and nonexistent identifiers to receive the same externally visible treatment. Nobody has settled that question. A

detailed implementation plan cannot supply the missing authority. The next useful output is a short clarification that shows the alternatives and the paths affected. After the owner settles it, the planner can select a route sized to the actual repair.

Change another premise. The lookup is generated from a maintained schema, and the reported endpoint is one of several consumers. The plan now needs the generation source, a way to reproduce the derived code and a review of affected consumers. Even if the textual patch is small, the change crosses a broader maintenance boundary. The planning route should reveal that boundary before construction.

Record why a route was sufficient. For the first repair, the explanation may be that intent is accepted, the affected surface is bounded and existing regression infrastructure can observe it. For the second, clarification is a prerequisite. For the third, generation and integration require explicit coordination. These are case-specific reasons the reviewer can inspect after the work, without making route choice a proxy for sophistication.

Specialized perspectives and human decisions

The planning material assigns different jobs to research, product, UX, architecture and specification skills. Its guidance links the choice to the missing information: evidence for a decision, agreement about the product or shared technical choices, for example. A team does not have to invoke every perspective merely because the framework supplies it. Planning activities and outputs, lines 50-86

For the vendor-access increment, we would distinguish proposing a policy from accepting it. An AI-generated analysis could identify the ambiguity about existing v1 requests. It could propose retroactive migration or binding at creation. The product decision owner must choose the intended behavior. In this case, the accepted design direction binds a request to its creation-time policy. That answer comes from the case specification, not from the authority of a generated role.

Similarly, a technical perspective can propose where to enforce the rule and what persistence behavior is needed. The implementing engineer remains accountable for explaining the choice. If a proposal assumes a database or framework not yet selected for the book, it is a candidate design, not an

authorized baseline. Good prose must keep that difference visible even when the proposed architecture sounds reasonable.

Build's documentation describes implementation followed by reviewer perspectives and correction of issues within the change. It also says that stronger dispatch depends on platform capabilities such as spawning workers or calling another model. Implementation and review, lines 82-92

For our evaluation, several model perspectives would remain AI-assisted review. We would record which host could dispatch them, which model or process performed them, and what they inspected. We would not describe that arrangement as independent human acceptance. Even a separate model invocation could share the same mistaken interpretation of the case.

The operator's intervention is therefore part of the process evidence. Suppose a reviewer flags a possible self-approval issue. Did the operator reject the finding because it was out of scope, accept it as a requirement defect or ask for a code repair? Did the resolution alter the intent or merely its implementation? A record that contains only the final green verdict loses the decision we need to understand.

We would preserve disagreements at the level where they belong. If the requirement is wrong, revising the code alone leaves the wrong instruction available for the next session. If the requirement is right but the code is wrong, changing the requirement to match the output erases the defect. The selected Build guidance explicitly allows returning to an upstream plan or goal when that is the source of the problem. That is a mechanism worth evaluating through actual records, rather than assuming every reviewer exchange uses it correctly.

Preserve the reason for a finding's disposition

A proposed review finding should identify the affected obligation, the relevant change and the evidence needed to decide it. For example, a concern about requester identity belongs to the current decision path when that path implements CS-05. A preference for an unrelated formatting convention may belong to later work. A tenant-boundary problem cannot be dismissed as unrelated if it invalidates the accepted scope of the increment.

The triage record can be compact: finding, basis, decision, responsible owner and follow-up evidence. Useful dispositions include correct the

implementation, revise an inadequate plan, obtain a product decision, defer genuinely separate work, or reject with a reason. The labels describe different actions. They should not all collapse into resolved when the last review message ends.

Suppose an illustrative plan promises that administrators may always decide requests. A reviewer points to the requester exception. Repairing only the generated condition would leave the plan misleading for the next unit. The plan needs reconciliation with accepted intent, followed by the appropriate downstream change. Conversely, when the plan already carries the exception, the defect may be in implementation or testing. The same visible symptom can require a different upstream correction.

This review design is authored for the case. It does not claim a native report contains these fields or that a particular reviewer would discover the mistake. It defines the evidence we would preserve to evaluate the documented correction mechanism.

Carry artifacts and context forward

The selected specification guide gives `SPEC.md` a compact role. It describes five fields covering purpose, capabilities with success conditions, constraints, non-goals and a success signal. Supporting documents sit alongside the specification and are referenced from it. Updates go through `bmad-spec`, which is designated the writer of the file and is instructed to preserve capability identifiers. Specification structure and maintenance, lines 97-124

That maintenance rule concerns the named `SPEC.md` artifact. It should not be generalized to every planning note or to the minimal specification mentioned in Build's light path. For a change to this artifact, the selected profile directs the operator through the designated skill. Chapter 8 compares specification ownership policies across approaches.

Here is an original content sketch for our case. It is a teaching table, not a generated `SPEC.md`, a copied prompt or an instruction to hand-edit that file:

Content to carry	Proposed case content
Purpose	Record an accountable tenant-local decision about temporary vendor access

Content to carry	Proposed case content
Capability and success condition	An eligible person can decide a pending request; the resulting state and audit history agree
Constraints	Tenant isolation, current membership, no self-approval, defined retry/concurrency behavior
Non-goals	External provisioning, billing, email and production identity federation
Success signal	Independent acceptance observations support the agreed behavior; unresolved cases stay visible

The final row is our evaluation requirement. It is not a claim that BMAD supplies the independent acceptance reviewer or a completed oracle. A framework's artifact and the book's external acceptance boundary serve different purposes.

The linkage can be made explicit without forcing every method to use our file names. Retain BMAD's native capability identifiers and map them to the common CS requirements. The following teaching example shows the content of such a mapping; it does not assign native identifiers or claim that artifacts already exist:

Proposed capability	Common requirement	Question for the implementation record	Acceptance evidence still needed
Decide a pending request	CS-04	Where is current same-tenant approver membership checked?	Eligible actor succeeds; revoked and foreign-tenant actors are denied
Decide a pending request	CS-05	Where is requester identity compared with decision-maker identity?	Overlapping-role requester denied on a pending request; separate eligible actor succeeds
Preserve the accepted decision	CS-07	Which operation prevents a later conflicting terminal decision?	Controlled competing decisions retain one permitted outcome

Proposed capability	Common requirement	Question for the implementation record	Acceptance evidence still needed
Preserve decision history	CS-10	How are the business decision and its audit record persisted together?	Matching state/history survive restart and remain tenant-scoped

Every evidence cell describes future work. A real record would link the native capability, relevant story, implementation change and actual acceptance observation. If the story links to CS-05 but its check only hides a button, the chain has a semantic gap even though every file reference resolves. The correction belongs in the check or implementation, not in a stronger-looking traceability label.

If the work needs several sessions, the guide offers Story Breakdown, producing `stories.yaml` alongside the specification and letting the operator choose checkpoints. It describes implementation records linked to their parent specification or story. Epic execution and linkage, lines 118-137

Our proposed decomposition would preserve behavior across those slices. Creating requests, deciding requests and reading audit history may be separately reviewable work, but tenant isolation applies throughout. A story list that mentions isolation only in the final slice would need examination. The question is whether earlier slices can expose an unacceptable boundary while they are treated as complete.

Context transfer also needs a deliberate boundary. Build calls for a fresh chat, while investigation reads the repository and relevant upstream records. Fresh-session instruction, investigation scope. A fresh session is not evidence that the right context arrived. We would inspect whether the capability, constraints, agreed policy and relevant code were actually available when the change was made.

The useful artifact trail is consequently a set of checkable relationships: agreed intent to specification, specification to story, story to implementation record, and requirement to acceptance observation. Counting files would tell us little about the correctness of those relationships. A later experiment must examine the content and the decisions carried through it.

Keep the unit and the larger undertaking distinct

The newly inspected Build page explicitly limits the unit's responsibility: larger work needs coordination above individual Build runs, with integration and later review outside that unit's own scope. It does not make Build the backlog owner. Unit and larger-work boundary

For our proposed increment, a story that creates requests and a story that decides them must agree on tenant ownership and policy binding. Finishing both independently does not establish that the combined behavior is correct. Identify who chooses the next unit, who resolves a shared contract and who checks the integrated result. Those responsibilities must exist even if one person performs all three.

An illustrative decomposition could establish the request/identity boundary first, add eligible decisions next and then complete permitted history access, while checking the shared boundary at every exposed operation. This is not a requirement to use exactly three stories. It shows why cross-cutting constraints cannot be postponed to an unspecified final security story.

Checkpoint placement should follow decisions with downstream consequences. The first implementation may establish an authorization or persistence convention reused by later units. Review that convention before multiplying it. Later routine work can use an already accepted pattern, but a newly discovered contradiction reopens the relevant decision rather than becoming a local exception in each story.

Maintain a distinction between unit completion and increment acceptance in the evidence index. A unit can record a successful implementation step while the integrated requirement remains unassessed. The parent record needs enough information to find remaining work, failed checks and dependencies. A collection of individually confident summaries cannot supply that relationship automatically.

Inspect a story split at a schema boundary

An illustrative BMAD planning record could split version binding into a creation story and a decision story. The creation story assigns a policy and persists the identifier. The decision story reads that identifier and resolves the eligible transition. Each seems small on its own. The order matters because a decision worker cannot distinguish an old request from a new one if the creation path leaves the identifier implicit. A reviewer can use the named specification to explain the dependency and ask the two story owners for one shared example.

Consider an existing request whose policy identifier is absent. One proposed migration fills the identifier with v1. Another makes the resolver infer the version when the field is missing. These lead to different consequences during rollout. Backfilling creates a migration step and an observable classification. Inferring in the resolver keeps the database shape simpler at first and makes every consumer of that resolver depend on the interpretation of an absent value. Before asking a story worker to implement either, record how historical rows will be classified and how ambiguous rows will be handled.

A useful story records the deliverable at its boundary. The creation worker can demonstrate that a request created under each supported policy carries the expected version. The decision worker can show how the saved version determines the threshold even after tenant settings change. The joint acceptance example begins with real creation, changes the active setting and decides the same stored request. Keeping the third observation visible prevents two individually complete stories from hiding a missing integration.

A different split puts the smallest end-to-end path into one story, then adds replay and exceptional histories separately. This exposes integration earlier, at the cost of a wider initial diff. The planner should choose based on whether the architecture is known and whether the team can review a vertical change coherently. The specification supplies the preserved rule in either split. Record the dependency and the acceptance owner in the work packet so a later reviewer can see why the split was chosen.

Proposed handoff	Producer's deliverable	Consumer's question
Accepted behavior to story	Rule, preservation example and unresolved contracts	Which decisions can this story use?
Creation to decision	Defined persisted identifier and historical-data policy	Can the decision path recover the creation-time policy?
Integrated work to acceptance	Candidate revision and observations through creation and decision	Does an actual preserved request behave under its original version?

These are proposed inspection points for the book's case. A reviewer applying them to an existing project should use the repository's actual data lifecycle and keep untested rollout behavior visible.

Build, review and establish acceptance

The selected Build page describes modified source, a local commit, an implementation record and passing tests when the project has a suite as intended outputs. It also separates unrelated work into a deferred-work record. These are documented outputs, not observations from our repository. Build outputs and deferred work, lines 113-131

The distinction matters most at acceptance. An implementation record can say what changed and which checks were attempted. We still need the actual commands, the outputs and the relationship of each assertion to the business rule. If the suite has no test for overlapping roles, a passing result cannot answer CS-05.

The completed-work testing guide states the scope of its built-in QA: API and end-to-end coverage generated from existing code, with happy paths and a limited set of error cases. It distinguishes test generation from implementation review and describes more complex fixtures and additional coverage as follow-up work or reasons to use the separate TEA module. Testing scope and optional module, explicit limits, lines 76-105

Generating tests from existing code can provide useful checks of that implementation. The acceptance question is broader: does the implementation satisfy behavior agreed independently of it? Our proposed

external checks derive from the common case requirements. They should be prepared and reviewed before we inspect a method's generated solution. Otherwise, the implementation could quietly become the definition of what was required.

For this increment, we would look for evidence at several boundaries:

Obligation	Proposed observation	What the observation would leave open
CS-05: no self-approval	On a pending request, an overlapping-role requester is denied; a separate equivalent fixture permits an eligible different approver	Other identity and authorization paths
CS-03: tenant isolation	Foreign-tenant direct-ID, list and search checks follow the disclosure contract	Unexamined interfaces and wider production assurance
CS-08: idempotent retry	A repeated identical decision creates no duplicate successful action	Key retention and other payload conflicts unless separately tested
CS-09: concurrency	Controlled competing decisions preserve one valid terminal outcome	Schedules and failure conditions outside the exercise
CS-10: durable audit	State and matching history survive a controlled restart	Backup, recovery and operational guarantees outside the test

These checks need explicit starting conditions. If the self-approval attempt targets an already completed request, a terminal-state guard could reject it even when the identity rule is absent. Keep that request pending, give the requester the otherwise required role, and use a separate equivalent fixture for the allowed action. This isolates the rule the check is intended to examine.

Persistence and concurrency mechanisms remain unselected and untested. The case contract and independent reviewer must settle the executable checks before matched runs.

Adding TEA would change the evaluated configuration. It might be an appropriate separate profile, but its setup, selected skills and operator effort would need to be recorded. We should not silently give one arm more preparation or describe optional coverage as if it were produced by the base

path. Equally, using a small built-in path does not justify claiming that BMAD cannot support broader testing.

The same discipline applies to review findings. A local fix, a generated test and an AI review verdict are different evidence items. The report should show which business uncertainty each addresses and which remains open. Chapter 12 develops the wider assurance obligations; this chapter keeps them connected to the workflow that produced the change.

Inspect what generated coverage assumes

For a proposed approval test, ask where the expected outcome originated. A test derived from current code may assert that an administrator succeeds because the code currently permits it. The assertion is executable but contradicts the requester exception when that administrator created the request. More executions of that assertion would strengthen confidence in the wrong behavior.

Use the external CS inventory to inspect omissions as well as disagreements. A suite can cover valid approvals yet contain no foreign-tenant read. A test titled no self-approval can target an already terminal request and exercise only its terminal guard. Review the starting state and the observable effect, not just the test name or generated coverage summary.

The book's proposed check map should preserve which assertions came from native generation, which came from independent requirement review and which were added after a finding. That provenance matters when interpreting defect discovery. A later human-supplied counterexample is part of the evaluated arrangement; it must not be retroactively presented as autonomous coverage.

Changing from built-in QA to a larger optional testing path can be a reasonable response to an identified need. Record the profile change, the additional preparation and the obligations it is intended to address. Do not treat either the expanded configuration or the original narrower one as the only possible BMAD intervention.

Review the Build handoff when the patch looks plausible

Consider a change that adds the two-approval rule by looking up a tenant's current settings inside the decision handler. The diff may be short and the happy path may pass. In the proposed case, this design still needs inspection because a v1 request can outlive the setting that was current when it was created. Give the reviewer the named specification and the linked work record, then ask them to trace the source of policy identity through creation and decision.

A useful review finding states the expected behavior, the proposed failure and the evidence needed. For example, a v1 request created before the policy switch should still be decidable with one eligible approval. If the implementation reads the newest settings, ask for an observation using a genuine pre-switch fixture. The developer can repair the resolver and update the linked work record. A refreshed result should identify the revision that contains both the fix and its check.

If the native review reports no findings, retain the report with its inputs and scope. The acceptance reviewer still has a distinct job. The native result helps explain how the change was reviewed, and the case observation explains whether the selected requirement held. The two records can be inspected together without treating one as a substitute for the other.

Existing code and evolving requirements

The existing-code guide prioritizes the current application, recommends compact project context where needed and advises keeping old greenfield material away from ordinary small changes. It also tells the process to use existing documentation and code when planning and to follow established patterns unless a change explicitly calls for a different one. These are source prescriptions, not evidence that the existing code is necessarily correct. Existing-code guidance, lines 8-79

This guidance creates a useful review question for the common case. Current code can show where an approval decision is made and what conventions the project uses. The accepted policy must still tell us whether that behavior is right. If an existing endpoint permits self-approval, faithfully preserving the pattern would preserve the defect.

We would separate repository discovery from normative intent. Discovery describes what the application does. The change request states what should become true. When the two disagree, the operator needs a record of the intended correction. Archiving stale product context can reduce competing instructions, but the evaluation must check that current invariants remain discoverable. We cannot infer either benefit or loss from an archive policy alone.

The proposed second episode makes that tension concrete. Policy v2 requires two distinct approvers for newly created high-risk requests. Existing v1 requests retain their creation-time policy; standard-risk v2 requests still need one approval. An instruction that mentions only the number two would leave several decisions unresolved.

We would first inspect the baseline's representation of policy, decisions and terminal state. Then the specification change would carry the agreed version-binding rule through the designated maintenance path, retaining capability identifiers where appropriate. The proposed story or implementation record would identify which existing behavior must stay valid and which new behavior is introduced. None of these steps has been performed in an installed BMAD session.

Acceptance needs both sides of the transition. A new high-risk v2 request should remain pending after the first eligible approval and complete after a second distinct one. A pending v1 request should keep the old threshold. Repeating the first approver must not advance the distinct count. Those examples come from CS-12 and CS-13, not from an imagined framework result.

For a fair method comparison, this episode would start from the same externally prepared baseline for every arm. Using each arm's feature output would make the change depend on different code structures and earlier mistakes. Native planning artifacts may differ; the starting business behavior and the acceptance boundary should not.

The third episode would ask for a narrow repair in a shared baseline containing one predefined tenant-boundary defect. BMAD's small-change path is relevant here. The evaluation should allow an appropriate bounded route rather than impose a full product-planning ceremony. It should also retain enough evidence to show that the repaired query and related behavior meet the agreed contract.

A discovery note that preserves contradictions

Before the proposed policy change, record the baseline locations that currently create a request, establish membership, count contributions, determine a terminal outcome and expose history. For each location, separate observed behavior from the intended rule and the evidence still required. This is an investigation aid, not a prescribed BMAD file format.

If a maintained document says policy is bound at creation while the current decision path reads a global setting, retain both observations. Their disagreement is the important finding. Choosing the newer-looking file without inspecting the dependency would merely hide the conflict. Accepted intent determines what the change should achieve; discovery establishes what must actually be changed.

The follow-through then becomes reviewable: specification revision through its designated owner, affected story/implementation records, actual code and checks, and the integrated disposition. Stable references help connect the trail, but the reviewer still has to inspect their meaning. An unchanged capability identifier can carry a revised rule whose downstream consumers have not yet caught up.

No such baseline investigation has been executed here. The note's value is its concrete division between repository evidence, accepted behavior and unresolved work. It lets a future reader challenge the implementation plan before treating a clean artifact trail as a complete maintenance result.

Inspect a repair that changes a generated surface

An existing-code repair may affect files produced by a generator. Ask the contributor to identify the maintained input, the derived output and the reproduction command used by the project. Review the proposed correction at the input that owns the behavior. Then compare regenerated output with the intended repair and inspect other consumers that generation affected.

If the repository cannot reproduce the current output, record that observation before choosing a repair path. A bounded manual correction can be proposed with its maintenance consequence and owner. The named specification and work record should expose where a future regeneration could undo the correction. That lets the reviewer assess the actual repair under the repository's conditions.

Limits and conditional fit

For this source profile, the central design is a choice of planning depth tied to intent, a compact specification with explicit maintenance ownership, linked implementation work and review/test activities whose scope must be identified. These mechanisms give us concrete things to inspect. They do not establish lower effort, better quality or sustained alignment.

Our editorial judgment is conditional. A team with reasonably clear intent and a need to scale between a bounded repair and multi-session work could investigate this profile. It should test whether the routing decision preserves its important constraints, whether the native artifact links remain useful after change, and whether its selected host can support the intended reviewer behavior. This is a proposal for evaluation, not a recommendation to adopt BMAD on reputation.

Three unknowns would materially affect that judgment. First, can operators reliably notice when apparently clear intent hides an important gap? Second, do the maintained artifacts help the next change enough to justify their cost? Third, does the chosen QA/review configuration find relevant defects under the available resource ceiling? Documentation supplies design expectations for these questions. Our research has not supplied their measured answers.

The operations boundary remains open as well. The inspected passages do not establish comprehensive production-operation or rollback governance. We should report that limit of inspection without turning it into a claim that the wider method lacks such practices. A local implementation exercise would still leave deployment, recovery and service ownership to be assessed separately.

Before accepting this chapter, a technical reviewer must check the selected source scope, artifact-ownership descriptions and the distinction between base QA and optional TEA. Before accepting executable examples, the book needs an actual environment, frozen inputs, independent checks and retrievable run records. The pilot has none of those results.

Chapter 9 compares the supported mechanisms across the four frameworks. Chapter 10 uses those boundaries to make conditional choices. The question carried forward from BMAD is practical: for this change, did the selected path preserve the decisions and evidence the team actually needed?

5. GSD - Deep Dive

Name the lineage and snapshot

GSD's role in this book is the process layer: how decisions, plans, work and verification are organized. A coding-agent product sharing related naming is outside the peer-method comparison. The distinction matters because an interface feature, model integration or product subscription is not automatically part of the process being examined.

The inspected former repository directs readers to `open-gsd/gsd-core`. That notice establishes a declared continuation, not a complete founding or ownership history. We use a pinned Core source for the contemporary workflow discussion and retain the requested chapter title. Inspected migration notice

The intervention we can examine is therefore specific: the phase-loop design and related verification guidance at `72c4e14`…. We have not installed that source or established that a published package delivers identical instructions. The chapter asks what the source prescribes, then designs a way to inspect its consequences for the common task.

One compatibility discrepancy illustrates why the pin matters. The same source's installation guide says Node 18 or later, while the package manifest declares Node 24 or later and npm 10 or later. A prose prerequisite is not sufficient to freeze an environment when its manifest disagrees. This is a documentary conflict, not an observed failed installation. Guide, manifest

For the SaaS example, the chosen language family remains TypeScript / Node.js. That does not resolve the discrepancy or select a web framework. A future executable profile must identify both the workflow's installation environment and the application's runtime. Those may impose different dependencies.

The distinctive design question is continuity: can bounded workers act on explicit decisions while persistent records make the larger work resumable? File-based continuity is a mechanism to investigate. Its existence does not establish that workers received the right information or that a later session interpreted it correctly.

Discuss decisions before decomposition

The phase-loop guide places discussion before planning. Discussion records implementation decisions in `CONTEXT.md`; the subsequent planning activity reads those decisions alongside research. The purpose described by the source is to make preferences and constraints available before an executor chooses a plausible alternative. Discussion and planning

For the approval increment, distinguish three kinds of input. Business behavior includes current same-tenant eligibility and no self-approval. Project constraints include the existing application boundary and conventions. Unresolved implementation decisions include the exact persistence/interface design where the shared contract has not settled it. A discussion should not make the third category override the first.

Suppose an operator says, “Administrators normally approve requests.” That preference must be reconciled with CS-05 before plans are written. In our case, an administrator who requested the item is still disqualified. Recording the preference without its exception could give a planner a coherent but incorrect rule.

The useful discussion result is therefore a set of decisions with boundaries. The following is an original content sketch, not a generated Core file:

Decision to carry	Proposed case content	What remains unresolved
Approval eligibility	Current approver membership in the request's tenant; requester excluded	Exact identity fixture and interface contract
Existing behavior	Preserve valid pending/terminal rules and tenant-local reads	Baseline implementation has not been prepared
Retry semantics	Apply the frozen identical-key/payload contract when one exists	Scope, retention and conflict response of the key
Policy transition	Bind a request to its creation-time policy	Technical representation and migration procedure

These entries keep an answered business question from being reopened accidentally and an unanswered technical question from looking settled. A future record should also identify who supplied each consequential answer. A

conversation that contains the answer is useful; a persistent summary must preserve its meaning.

Discussion has a stopping point. It should resolve the decisions needed for bounded planning, not become an indefinite interview about every possible product feature. If an uncertainty does not affect this increment, record its exclusion. If it affects acceptance, resolve it or explicitly stop the affected work. The question is whether the planner has enough agreed information to avoid inventing business policy.

For an established application, answers may come from repository evidence. A current route, accepted decision record or maintained convention can settle an implementation choice. The source of that answer matters: matching current code is appropriate only where the intended change does not correct that code's behavior.

Make a decision recoverable outside the conversation

A decision record needs its scope as well as its answer. For E2, two approvals is incomplete. The recoverable decision is that newly created high-risk v2 requests need two distinct eligible contributors, while standard-risk v2 and existing v1 requests retain their defined thresholds. Attach the authority and applicable requirement version so a later planner can distinguish a product rule from an implementation suggestion.

Separate mandatory behavior, permitted technical choices and unresolved questions. The no-self-approval rule is mandatory. A particular repository abstraction may be a technical choice within the application boundary. The response to a retried key after authority changes remains unfrozen. Recording all three in the same confident tone would give a worker no basis for deciding what it may change.

An original context audit can therefore ask what an executor could reasonably infer from each entry. Could it infer that current administrator status overrides requester exclusion? Could it infer that v2 changes old requests? Could it mistake an example storage field for the required public interface? Fix the entry's meaning before decomposing work around it.

This inspection differs from adding more context indiscriminately. An obsolete design can be more harmful than a missing optional note when it appears authoritative. Keep the active decision linked to its evidence and

identify superseded alternatives. The aim is a recoverable choice, not a transcript of every possibility considered.

Turn discussion into a dependency a planner can use

A discussion about request policy can end with an apparently complete sentence that still leaves planning difficult. “Keep old requests working” needs a definition of old, a source for the version and an owner for records whose origin is unclear. Ask the discussion record to resolve the part that constrains decomposition. A worker can choose an internal function name later. It cannot safely choose which existing customers receive a different threshold.

A useful decision record for the case names the selected behavior, its rationale and an example that crosses the policy switch. It also lists any unresolved questions that do not block that behavior. The planner can then create work for version assignment, resolution and the integrated observation. If historical records cannot be classified, the record should expose that dependency and identify the work that remains possible while the owner investigates it.

The distinction between a decision and a preference also matters. A preferred layout for a summary may be adaptable. A preserved v1 promise constrains implementation. Mark that difference in the work packet so a worker can make routine choices without reopening accepted behavior. If a technical discovery contradicts a decision's premise, return the observation to its owner and record the revision before continuing dependent work.

A later reader should be able to recover the discussion's result without reproducing its entire conversational path. Retain rejected alternatives where they explain a consequential tradeoff. Keep the actual accepted condition easy to find. For example, the reason for saving a policy identifier may be that pending requests outlive tenant settings. That rationale helps a maintainer resist replacing the identifier with a current-settings lookup during a future cleanup.

This is a proposed inspection of the discussion-to-plan boundary. A local exercise could ask another operator to plan the same bounded change using only the retained decision. Their questions would reveal which premises still depend on the original operator's memory.

Plan bounded work with persistent state

The inspected loop describes a research/planning/checking sequence, bounded `PLAN.md` files and dependency waves. Each plan identifies a work unit, relevant files, changes and acceptance criteria. Execution summaries, verification and state records connect the activity to later closure. These are prescribed structures, not artifacts produced in this project. Core phase loop

A plan boundary should make the decision path easier to inspect. “Implement approvals” hides several business obligations. “Implement an eligible decision transition while retaining tenant isolation and its audit event” identifies a more specific unit, but it still needs an agreed contract and a prepared baseline before it can become an executable plan.

Do not decompose away a shared invariant. A request reader, decision path and history reader all carry tenant scope. If isolation appears only in a final cleanup plan, earlier work may be treated as complete before the relevant obligation is satisfied. Native plans can differ in size while retaining a common external acceptance boundary.

For a documentary walkthrough, preserve the role of each native record:

Record described in the loop	Question we would ask of its case content
Phase goal	Does the goal identify the bounded business outcome?
`CONTEXT.md`	Are the agreed actor, tenant and policy decisions still explicit?
`RESEARCH.md`	Does each technical recommendation distinguish evidence from an assumption?
`PLAN.md`	Are dependencies, change scope and acceptance criteria inspectable?
Execution summary	Does it identify actual changes, deviations and unexecuted work?
Verification record	Does it relate requirements and decisions to inspected evidence?
State/roadmap records	Do they reflect the same completion state and unresolved gaps?

A durable file is useful only if it remains relevant and recoverable. A new session that loads a phase goal but misses the policy exception still has incomplete context. A summary that reports “all checks passed” without the checked revision or outputs remains insufficient for our acceptance question.

The dependency graph also needs semantic review. Two plans may touch different files yet depend on the same policy interpretation or shared data contract. Non-overlapping edits do not prove independent work. For the two-approval change, the decision writer and reader must agree about policy binding even if their files differ.

We would preserve both declared dependencies and discoveries during execution. If a worker finds that the baseline does not expose the assumed repository operation, the plan needs correction or a visible deviation. Editing around the assumption and reporting completion would hide the cost of reconciling the design with the actual application.

Persistent state makes this reconciliation inspectable in principle. A future exercise must show whether its records actually supported recovery, how much human interpretation they required and what information was lost. The documentation alone does not supply those outcomes.

Examine the dependency behind the file list

For an original E2 planning exercise, consider these proposed units. Their labels are teaching references, not generated plan identifiers:

Proposed unit	Necessary input	Dependency to inspect
A - establish policy interpretation	Accepted v1/v2 transition and baseline evidence	Binding semantics must be agreed before consumers are changed
B - reconcile creation and decision paths	The same version/threshold contract	A request's recorded policy must agree with the decision reader
C - reconcile permitted reads/history	The same state and contribution meaning	Partial approval must not be presented as terminal completion
D - assemble acceptance observations	Reviewed fixtures and resulting implementation	Earlier regression and new behavior both need actual checks

Some work within B and C may be parallel after the shared contract is established. That possibility is a design question, not evidence that every file-separated task is independent. If one worker invents a representation and another assumes a different one, their eventual integration requires reconciliation even when neither edit conflicts textually.

The plan checker should be examined against those semantic dependencies. A plan can name a file and a check while lacking the accepted rule that gives the check its expectation. Conversely, one plan may legitimately carry several tightly related operations when splitting them would obscure a shared invariant. Granularity earns its place through clarity and executable boundaries.

Preserve any dependency discovered later. If the baseline does not contain the assumed policy source, record the correction, affected plans and manual work. The final arrangement may still be useful, but the evaluation should distinguish following a well-founded plan from repairing an inadequate one during execution.

Plan around the point where independent work stops being independent

Consider two workers with different tasks. One adds a migration and creation behavior. The other changes decision evaluation. Both need a policy identifier, but each may be tempted to create its own vocabulary while the other is working. Name the shared concept in the accepted plan and identify who owns the interface. An agreed identifier plus a provisional fixture may let the decision worker proceed. Mark that fixture explicitly and schedule a later exercise that uses the completed creation path.

Parallel work has a boundary beyond source file overlap. Two edits can touch different files while disagreeing about whether an absent identifier means v1, an invalid record or a request awaiting repair. The plan should distinguish a choice settled by the owner from one a worker is free to make. An accepted historical-data rule constrains both tasks. If nobody can establish the provenance of a historical record, pause the dependent transition design and keep unrelated work moving.

A summary is useful when it says what inputs it consumed. A decision worker can record the agreed interface revision, the provisional fixture and the checks performed against it. The integration worker can compare those

inputs with the actual migration and creation change. If the interface changed, identify which earlier observation still applies to the policy function and which service-level check must run again.

A phase can close with evidence for the work actually delivered. If the integrated path is still open, record that state directly. The next operator should be able to see the accepted decision, the implemented pieces, the mismatch if one exists and the action needed to complete the phase. The exercise tests the continuity of the proposed work record in addition to its decomposition.

Execute with explicit context boundaries

The guide describes fresh executor contexts loaded with selected project, phase, research and plan information. It also gives a specific window size and strong claims about avoiding context degradation. We can establish the prescribed strategy from the source. We cannot generalize its capacity or effectiveness to every host/model configuration. Execution-context description

Treat a worker handoff as an input contract. What is the worker expected to know, which decisions may it make, and where should it send an unresolved issue? The goal is not to maximize loaded material. It is to make the information required by the bounded task available without allowing stale or irrelevant material to settle current decisions.

For our decision-transition worker, the minimum relevant business context includes the pending-state precondition, current membership rule, no-self-approval rule, policy threshold and durable audit obligation. Technical context includes the selected baseline conventions and the frozen interface contract. The context record should point to their authoritative locations rather than silently restate a different version.

An original handoff inspection could ask:

1. Does the plan link to the actual requirement version?
2. Can the worker identify which request policy governs this transition?
3. Is the retry contract available, or is the worker expected to guess it?
4. Which existing behavior must survive the change?

5. Where is a discovered contradiction recorded before the scope changes?

These questions require the actual inputs or an auditable reconstruction during an executed trial. A worker summary saying that it considered security is not equivalent evidence. Nor can a new worker's clean context establish that it received a complete decision record.

Concurrency in orchestration needs its own scope. The source's waves coordinate plans. The application also has concurrent business decisions. Successfully dispatching workers would not establish correctness when two approvers act at once. The first concerns process execution; CS-09 concerns application behavior under competing decisions.

Record operator intervention as part of the arrangement. If a person supplies a missing policy exception halfway through execution, retain the correction and affected work. The resulting implementation may be acceptable, but its record should show that the initial handoff was incomplete. This is information for the context-management dimension, not an embarrassment to remove.

For a host migration, inspect which parts of the same handoff and reviewer arrangement remain available. Multiple supported installation paths do not establish identical hooks, permissions or worker behavior. A portability claim needs a defined migration exercise, with all compensating manual work included.

Inspect a handoff before interpreting worker behavior

Suppose a future decision-path worker receives the phase goal, its own bounded plan and a link to the policy decision. A useful handoff inspection follows the link and verifies that the available version includes creation-time binding and requester exclusion. Merely listing a filename in a prompt does not establish that the content was readable or that the executor used it.

Record the actual input selection at the point of execution where the host permits it. If only a reconstruction is possible, state that limitation. Keep retrieved authoritative context separate from an operator's later explanation of what the worker should have known. The difference matters when diagnosing an omission.

An executor's new question can reveal an incomplete plan, a genuinely new discovery or an ignored available answer. Those explanations have different

implications. Preserve the question, the input it had and the response rather than counting every clarification as the same kind of human intervention.

The application contract remains external to this process analysis. A worker can complete its bounded implementation yet leave a cross-unit history or tenant obligation unassessed. The phase-level evidence needs to reconnect the bounded outputs before claiming that the business increment is accepted.

Design a resumability probe without changing the task

A proposed continuity exercise can stop after decisions and plans have been recorded, then resume with the declared retained state in a fresh session. Another can stop during the acceptance conversation. Specify the interruption point and retained artifacts before an attempt, rather than choose whichever interruption makes recovery look straightforward.

The recovery question is whether the operator can identify completed work, remaining work, accepted constraints and current evidence. It is not simply whether the workflow opens another chat. Record any manual reconstruction needed and any contradiction between state records and actual artifacts.

For a measured comparison, decide whether this continuity exercise belongs inside the episode or is a separate intervention. Native state produced during an attempt may legitimately be retained for its continuation; state leaked from an earlier independent attempt must not quietly become an advantage. The reset policy needs to distinguish the two.

Inspect a worker assignment for missing constraints

In the illustrative Core arrangement, split a policy change into work that assigns version at creation and work that resolves it at decision time. Give both assignments the same accepted vocabulary for v1, v2 and risk. Name where the resolver will obtain a version and which change owns the persistence format. Otherwise two workers can independently make plausible choices that cannot be integrated without rewriting one side.

A handoff should make a dependency concrete. If a worker needs the stored identifier but the creation work has yet to land, record whether that worker is building against an agreed interface, a temporary fixture or a completed upstream change. A temporary fixture can support early work. When it is replaced, inspect the actual creation path and refresh the checks that

depended on the fixture. The plan can point to that integration observation without prescribing a particular task engine.

After a session interruption, have the next operator reconstruct the state from the retained decisions, plans and outputs. Ask them to identify what is ready to integrate and which verification used a temporary fixture. Any question they can answer only by asking the previous operator is a candidate handoff defect. This proposed exercise addresses the usefulness of the retained records for this change.

Make a worker's completion report usable by integration

A proposed completion report can name the interface consumed, the change produced and the observation performed. For a resolver worker, the report might say that it consumed the agreed stored policy identifier and exercised the threshold against synthetic inputs. The integration owner still needs to connect creation and persistence to that interface.

List any provisional input beside the result. If the worker substituted a fixture for an unfinished creation path, identify that substitution and the observation required after replacement. A reviewer can then accept the component work while preserving the integration task. If the interface changes, the report gives the next operator a way to find affected evidence.

A useful final line states the next dependency. For example, the integrated preservation probe awaits the creation change and a migrated historical fixture. That line makes handoff actionable and keeps a favorable component result attached to its actual scope. The integration owner can update the dependency when the prerequisite arrives and retain the earlier report with its revision.

Verify, recover and close the work

Core's verification guide describes a user-acceptance conversation derived from execution summaries, with persisted progress and gap-fix planning. It separately describes shipping preflight and pull-request creation. Code review is optional rather than an automatic part of shipping. These are distinct activities in the inspected design. Verification, gap closure and shipping

The summary-derived checklist can help an operator inspect what the process reports as delivered. It cannot, by itself, establish that omitted requirements

were tested. If the implementation summary never mentions a foreign-tenant direct-ID path, a checklist derived from it may omit that boundary as well.

Our case therefore retains an independent acceptance inventory. The process's verification record and the external checks should be related, not treated as interchangeable. For each CS requirement, identify the native evidence, any independent observation and the unresolved gap. This also makes it possible to discover when a source-native check is more useful than the initial book instrument expected.

Recovery begins with the failure's scope. A failed assertion may indicate wrong code, an incorrect fixture or an unresolved requirement. A diagnosis should distinguish those possibilities before generating a fix plan. Changing the expected outcome to agree with the implementation requires product authority when it changes accepted behavior.

The guide also describes stale verification. Current reports can fingerprint covered inputs and implementation files; older reports use a summary-modification-time rule. When relevant inputs change, the gate is supposed to require renewed verification. This establishes a designed freshness mechanism, not a test of its completeness or bypass resistance. Staleness policy

Freshness and correctness remain different. A verdict can be current for all files it covers and still omit a relevant dependency or business rule. A future review must inspect coverage as well as the digest. Conversely, a stale verdict should not be reused merely because the change looks small.

Closing a phase is another decision boundary. The source's ship behavior concerns repository delivery and PR preparation. It does not establish deployment, operational ownership or production acceptance. In this book, we do not execute that shipping command or infer authority to push an external change from a documentary description.

For an executed case, closure would preserve the baseline and final revision, actual checks and outputs, human responses, failed attempts, deviations and unresolved requirements. If the resource ceiling is reached, record a stopped attempt rather than relabel the remaining work complete. The current chapter has no such run record.

Keep the acceptance response attached to its question

The re-inspected guide describes an interactive UAT flow in which an empty response can be treated as a passing answer. That documented interaction makes response interpretation part of the evaluation design. It does not establish that any absent answer in another workflow counts as consent. UAT interaction and scope

For a future trial, preserve the presented checkpoint, actual response and evidence the person used. A person saying yes after examining a display has answered that display question; they have not necessarily inspected a direct-ID authorization path or concurrent writes. The independent CS inventory needs its own observations.

If a checkpoint fails, retain the original observation through diagnosis and gap closure. An inadequate fixture may require a new check, whereas a real implementation defect requires repair. A changed product expectation requires authority and can change task identity. The final successful run should not erase which of these occurred.

Review freshness coverage as well as freshness status

A freshness mechanism relates a verdict to its covered inputs. Inspect the actual covered set before interpreting a current status. For E2, the relevant requirement, policy interpretation and changed implementation need a clear relationship to the verification record. If fixture or environment changes matter, inspect how the selected arrangement accounts for them rather than assume that a file digest covers the whole exercise.

A changed fingerprint identifies a change in covered bytes, not its business meaning. A semantically unchanged edit may still require the designed recheck, while a relevant omission outside the covered set may need a separate acceptance observation. Neither fact justifies disabling the freshness rule. It identifies the additional interpretation duty.

The proposed evidence packet should retain both the earlier verdict and the renewed one, with the change that prompted reconsideration. That history can establish what was re-examined. A final status alone cannot show whether an operator reran the appropriate checks or merely edited a completion label.

Reconstruct what a stale verdict covered

Imagine verification ran after the decision worker's changes but before the creation worker replaced the fixture. Later the integrated application compiles and someone sees a favorable verification status. The reviewer needs to identify the revision and the checked boundary before deciding whether the verdict can be reused. The earlier observation may still support the threshold calculation under its fixture. It leaves assignment through normal creation open.

Write a recovery note with three entries. The first identifies the old verdict and what it exercised. The second lists the inputs that changed, including the fixture and the source of the version identifier. The third names the fresh exercise and its prerequisite. For this example, create a v1 request through the service, switch the active policy and decide that same request. The result gives the next operator a way to distinguish an input change from a policy regression.

Run status and work status also age differently. An old summary may accurately describe a finished implementation task while a new finding reopens acceptance. Keep the summary as history, and record the open finding against the current integration revision. If the reviewer must redo the whole investigation to recover that distinction, improve the handoff before opening another phase.

An operator can apply this inspection to a stopped session without treating every old record as stale. Preserve the accepted product decision unless the owner changed it. Revisit checks that depend on the altered implementation or fixture. Identify any evidence whose revision cannot be recovered. This keeps recovery proportional while making the basis for closure visible.

Small repairs and policy changes

The phase-loop guide warns against phases that are either too broad or too fragmented and points a trivial repair toward a quick path. That documented allowance matters when comparing overhead. We should not require the full loop for a task the source routes differently. Phase sizing and small-work guidance

A small presentation repair from chapter 2 could therefore be evaluated under a bounded route, preserving the same intent and regression duty. Its

record would disclose the path actually used. If the task reveals a policy or data-model uncertainty, revisit the scope rather than cling to the initial small-task label.

The E2 policy change is different. A new high-risk v2 request needs two distinct eligible approvals; an existing v1 request retains its creation-time rule. A plan that changes only a global approval-count constant would leave policy binding unresolved. The required discussion and context must carry both the new behavior and the preserved old behavior.

Our documentary change trail would revisit the phase goal and decisions, inspect the prepared baseline, identify bounded plan dependencies and update the verification obligation. It would not reuse an obsolete success assertion from E1. The actual native file contents and commands remain to be produced by an executed profile, if readiness later permits one.

Acceptance examples need independent fixtures. A new high-risk request should remain pending after one eligible approval and complete after a second distinct approval. Repeating the first approver must not advance the count. A pending v1 request should retain its original threshold. These are case obligations, not observed Core outcomes.

For E3, use the same prepared defect and report for every method. The defect is a predetermined tenant-boundary error, not a different problem chosen after observing each method. The small-path comparison should preserve the diagnostic record, narrow repair and relevant regression evidence.

Existing code deserves particular care. Persistent plans describe intended work; the application describes current behavior. A conflict between them needs a decision. The inspected source set did not establish a complete reverse-engineering baseline procedure for every Core path, so we leave that broader capability unassessed rather than infer absence.

Carry the change through the phase record

For our proposed v2 episode, compare the current phase goal, accepted context, affected plans and their later summaries. Each should describe the same change boundary. A summary saying support two approvers is not sufficient when the context also preserves v1 requests and standard-risk v2 behavior. The external inventory makes those omissions visible without dictating the native artifact format.

Then inspect closure dependencies. If one plan updates creation and another updates decision handling, a phase verdict should not rely on only one plan's local checks. If a policy question remains unresolved, retain the blocked scope rather than make a new worker guess. If the task legitimately becomes smaller, revise the episode definition visibly instead of treating omitted obligations as completed work.

These inspection steps do not prescribe the contents of generated files or predict an executor's behavior. They define how a reviewer could evaluate the phase-loop mechanism against one shared maintenance obligation, including partial progress and human correction.

Separate the problem that needs a decision from the work that needs a plan

For a brownfield repair, the first uncertainty can be about what the software already does. Give the investigation a bounded deliverable: find the failing entry point, show which query it reaches and list the other callers of that query. Stop with those observations if the intended disclosure contract is still open. This keeps an investigation from quietly deciding what every unauthorized client should receive.

Once the owner settles the contract, scope the repair by affected call sites. An endpoint patch and a shared helper change carry different verification duties. An endpoint patch needs an explanation of why the other paths already enforce the boundary. A helper change needs observations that existing permitted reads continue to work across its callers. An investigation summary should preserve enough context to make that choice after the original session ends.

If the work begins as a quick repair and reveals three distinct data paths with incompatible assumptions, record why the original route stopped being sufficient. The operator can choose a plan sized to the expanded boundary and link it to the original finding. If the investigation finds one well-understood predicate and adequate existing controls, the original route can still be enough. Decide from the work discovered in the repository and the observations needed to accept it.

Rehearse context recovery on a deliberately incomplete task

A team can inspect its proposed context arrangement with a small exercise. Stop after one worker completes a component check while integration remains open. Give a new operator the retained records and ask them to name the next authorized action, the input it needs and the old observation that cannot establish integration. Record where they find each answer.

The exercise has an expected outcome. The new operator should be able to distinguish accepted intent from a provisional fixture and identify the observation still required. If they infer that all work is complete from a favorable component result, inspect the summary and its link to the active plan. Correct the record at the first point where scope was lost.

Repeat the recovery using the corrected packet and retain the result. A successful local recovery would support a narrow claim about those records and conditions. It would also give the team a concrete handoff example to use when the next change stops halfway through.

Limits and conditional fit

The inspected Core design offers a concrete arrangement of decision context, bounded plans, workers, summaries and verification. Its potentially useful property is the explicit information flow between activities. Whether that flow reduces errors or effort remains an empirical question for the selected task and configuration.

Our conditional assessment is to investigate this arrangement when a project needs resumable bounded work and can support the required host/operator practices. Examine whether material decisions survive handoffs, whether plan dependencies reflect shared contracts, and whether verification detects omissions as well as implemented deliverables. These are evaluation premises, not a default adoption recommendation.

Setup and maintenance costs remain unmeasured. The same is true of actual context quality, collaboration among people and long-term operations. The installation discrepancy and source-versus-package identity must be resolved before a reproducible executable profile is claimed. None of these gaps is converted into a numerical penalty.

The model-worker arrangement is also not independent human review. Additional agents can provide useful perspectives, but the book still needs external scrutiny of its sourcing and any eventual case evidence. Solo orchestration cannot establish organization-wide coordination benefit.

For the comparison chapter, retain the strongest supported documentary conclusions: explicit phase decisions and plans, deliberately bounded context, summary-based UAT, separate optional review, and a designed freshness policy. Retain the qualifications beside them. We have no controlled GSD-versus-peer outcome from this project.

The practical question carried forward is whether the persistent record makes the next decision easier to recover without hiding what was never settled. That question can be asked of a quick repair, a policy change or a multi-session feature, with a proportionate path for each.

6. AWS AI-DLC - Deep Dive

Concept, implementation and profile

An approval can mean several things in this chapter: a person accepts a development plan, a workflow permits a stage transition, or an application actor approves a vendor-access request. Confusing those decisions would make both the process and the case harder to evaluate. We will name the decision and its owner each time.

AWS's original AI-Driven Development Life Cycle article presents AI preparing plans and artifacts with human oversight across inception, construction and operations. It is a primary account of the proposed method. Its delivery and quality benefits remain publisher assertions, not outcomes measured by this book. Original AI-DLC article

The inspected workflow implementation has additional named phases, including initialization and ideation. It also supports configurable profiles and routes. We therefore need three separate identities: the lifecycle concept, the pinned implementation and the selected profile/settings. A statement about one does not automatically establish the others. Implementation phase map

For this documentary chapter, Classic and Express illustrate how a profile changes the intervention. They are not interchangeable configurations of identical work. The source describes different stage inclusion, artifact depth, reviewer behavior and guard defaults. We have not chosen one as a frozen experimental profile. Profile descriptions

Our business problem remains the same tenant-local vendor-access decision used in the other deep dives. The AWS name does not, by itself, answer which services or deployment environment the exercise requires. Inspect the selected implementation's dependencies and record local substitutions. A service tutorial would answer a different question from the process comparison.

The distinctive question is how intent, authority and evidence move through the lifecycle. If the tool proposes a plan, what does the person approve? If a profile reduces ceremony, what duties remain? If the workflow marks a stage

complete, which observations justify that state? These questions guide the chapter before any benefit is inferred.

Describe the selected arrangement in one inspectable card

An original intervention card for this case would identify the lifecycle concept, source/package, named profile, active overrides, host capabilities, human owners and local exclusions. The card should tell another operator what they must reproduce and what they must not infer. The word AI-DLC alone cannot supply those fields.

Record settings at the moment the episode begins, then preserve changes during work. A profile default can explain an initial expectation, but an actual override changes the evaluated arrangement. If the host cannot implement a documented control, identify the compensating manual duty and its owner. Do not describe that adaptation as equivalent enforcement without observing it.

The earlier Nov 2 chooser inspection also identifies Feature as a complete-lifecycle candidate, distinct from Classic's narrower described route. The label describes intended coverage, not observed operational completion. Previously rechecked chooser

For the book's local exercise, leave an unfrozen field explicitly open. A documentary explanation can compare route responsibilities before installation. An executable comparison cannot replace missing configuration with whichever default happens to appear when the operator starts.

Distinguish a concept review from a configured evaluation

A concept review asks which lifecycle duties the original account proposes and how it assigns participation to people and AI. Its evidence can be an attributed publication and the terms used there. A configured evaluation asks what the inspected workflow does under a selected profile, host and settings. Its evidence needs the actual selected inputs and observations from the relevant path.

A useful setup card names both. Record the concept source used for orientation, then the implementation revision and configuration used for work. If the organization supplies incident response or deployment controls, identify those additions and their handoff. An evaluator can then inspect

whether an outcome depends on the native route, a host capability or an existing team practice.

Suppose a team changes the review cap because its host cannot provide the planned reviewer arrangement. Record the changed cap and the replacement review. The team may still have a sensible process, while the configured arrangement differs from the original plan. Preserve the distinction when reporting setup effort and acceptance.

The same care applies to stage counts. A count can describe the configured route and help an operator anticipate work. Whether a conditional stage ran and what evidence it produced require an actual execution record. Keep the selected configuration alongside the observed sequence so a reader can see which parts were planned and which were encountered.

Intent, clarification and collaborative decisions

Start with the request to build tenant-local approval behavior. Its intent includes current membership, no self-approval, terminal-state consistency and durable history. The detailed interface and retry contract are still open. A lifecycle conversation should expose those uncertainties and preserve the answers rather than allow an implementation to settle them by convenience.

Clarification becomes useful when it identifies a consequential decision. “What should happen to existing v1 requests?” is a precise question for E2. The accepted case direction supplies the answer: requests keep the policy active at creation. “Do you approve the design?” is less useful if the design has not made that choice visible.

For an original teaching example, the intent review might contain these entries:

Decision	Case direction	Review duty
Tenant boundary	All request/history operations remain tenant-local	Inspect direct identifiers, lists and searches, not only the visible page
Approval actor	Current eligible approver; requester excluded even with other roles	Preserve the exception in requirements, design and acceptance

Decision	Case direction	Review duty
Policy transition	New high-risk v2 requests need two distinct approvers; existing v1 requests retain v1	Identify the creation-time binding explicitly
Operational scope	Record decisions locally; no external access provisioning	Keep provisioning and production claims outside the example

This table is authored for the book. It is not an intent artifact emitted by an AI-DLC session. A later run would retain the native record and actual human responses, with the table serving only as an external interpretation aid.

Collaborative decisions need authority as well as participation. Several model perspectives can propose design or test concerns, but a product-policy answer still needs the designated owner. Several people can participate while leaving an unresolved choice to whoever implements first. Count the decisions made and preserved, not the number of voices present.

When the same person plays several roles in a local walkthrough, disclose that arrangement. It can make the prescribed handoff understandable; it does not establish the experience of a human team negotiating different responsibilities. The original collaboration concept therefore remains broader than what our unexecuted solo teaching example can observe.

Clarification should also preserve scope. If a proposed operational activity requires credentials, paid infrastructure or a live vendor integration, it exceeds this case's agreed boundary. Record the dependency or use a clearly labeled local substitute where appropriate. A substitute changes the evidential scope even if it lets a demonstration continue.

The outcome of intent work should be a reviewable commitment: what becomes true, what stays true, what remains unresolved and who may decide the remaining questions. A polished plan without that content cannot earn acceptance merely by fitting a named lifecycle stage.

Give an approval a bounded object

For the two-approval change, a proposed decision review should expose the new high-risk rule, the preserved v1 behavior, unchanged eligibility constraints and technical questions still awaiting resolution. Identify the artifact revision being considered. A later plan edit may require renewed review of affected decisions; the earlier acknowledgement does not automatically cover new content.

The person reviewing product intent need not settle every storage detail. They do need to see whether the design preserves old requests or silently changes them. The person reviewing implementation feasibility needs a different view: baseline evidence, dependencies and the chosen application boundary. Both decisions can reference the same packet while answering different questions.

An original review record can state decision, owner, inspected evidence, accepted scope, unresolved conditions and consequence for the next activity. It should make conditional acceptance distinguishable from unconditional permission. If unresolved request-key semantics affect the proposed implementation, proceeding with unrelated design work is different from accepting the retry behavior.

This design makes disagreement useful. A reviewer who rejects retroactive policy application has changed an explicit plan decision, not merely supplied general feedback. The downstream record should show the resulting revision. The process can then be inspected for preservation of that answer across construction units.

Turn a clarification into something the next stage can use

In the teaching case, the owner says existing requests keep their original policy. A written answer needs three references to travel: the behavior it governs, the stage output that consumes it and the observation the team expects later. Here those references are creation-time binding, the design for storing and resolving the assigned version, and a test that begins before the switch and decides the same request afterward.

The clarification changes construction work even if the approved intent already said “support dual approval.” A team should be able to see that change without reading the full conversation. Record the affected unit and an

open dependency if the persistence format is still under discussion. A construction worker can use a provisional interface where appropriate, but the boundary review needs to expose its provisional status.

The owner may also leave some decisions explicitly open. If external clients have never been promised a response for replay with a changed payload, the team can proceed on version binding while that contract is resolved separately. The stage record should make the scope of any progress clear, so an accepted design for the threshold does not become an accidental acceptance of replay behavior.

Phases, routes and persistent artifacts

The implementation's phase map and route descriptions organize work over time. The selected source describes phase-boundary trace checks and construction that can proceed by unit or by stage, with further conditions on parallel work. This is a designed orchestration policy; it is not a timing or throughput result. Ordering and construction routes

Classic is described with substantial human checkpoints, standard artifacts/tests and an operations placeholder. Express removes several design/decomposition activities and reviewer dispatch, while retaining particular human duties and conditional stages. Those differences must remain visible in any comparison. A shorter route is not automatically lower effort or sufficient assurance for the same scenario. Classic and Express

Phase counts alone are consequently a weak comparison unit. Setup may be counted separately, profiles may skip activities, and construction may repeat work per unit. Ask which decisions and evidence the route requires for the same bounded task. A count that ignores those distinctions can make equivalent work look different or different work look equivalent.

The artifact reference separates committed workflow state, the original request, audit records and boundary-verification records from transient engine state. That distinction matters for a handoff or reproduction exercise. A record available in one local checkout may not survive merely because some framework artifacts are committed. Artifact retention boundaries

For the case, inspect the purpose of each record rather than reproduce the entire directory tree:

Record category	Question for the proposed increment
Original intent	Does it preserve what the operator actually requested?
Workflow state	Does the reported stage agree with work and unresolved decisions?
Framework audit	Which decisions, interventions and configuration changes can be reconstructed?
Boundary verification	Which links and consistency conditions were checked?
Local engine state	Which information is transient, and what must be captured separately?

Framework audit is not application audit. The first concerns development workflow events. CS-10 concerns the business service's tenant-scoped decision history. Committing a workflow audit record cannot establish that the application persists an approval event correctly or prevents another tenant from reading it.

Similarly, a boundary link check is not the business oracle. A requirement and design can link correctly while sharing a mistaken policy. The process record may help locate the mistake; independent acceptance still needs to inspect the behavior and expectation.

When the route changes midstream, retain the configuration and reason. If an operator skips a stage, the resulting evidence should say that it was skipped, not imply it ran. A local walkthrough can explain the consequences of that choice; only an actual run can show what the engine recorded and how subsequent work behaved.

Inspect cross-unit contracts under either route

Imagine three proposed application units: request creation, decision handling and permitted history access. Their exact architecture remains open. Creation establishes the request's tenant and policy. Decisions consume that interpretation and preserve contributions/state. History must expose the permitted tenant-local account of those events. These relationships exist whether construction is organized by unit or by stage.

Under a unit-oriented arrangement, an early completed unit can establish assumptions for later ones. Inspect those assumptions before calling the next unit independent. Under a stage-oriented arrangement, a common design can make the intended contract visible first, but implementation still needs to realize it consistently. Neither ordering removes the integration duty.

For a proposed boundary review, ask whether a newly created high-risk v2 item is interpreted the same way by creation, decision and history paths. Does the first accepted contribution leave it pending everywhere? Can the history distinguish that contribution from terminal completion? Does the second distinct eligible contribution produce a consistent accepted state? The native records should make these questions traceable before actual checks answer them.

If a unit changes a shared contract, identify every consumer and the decision owner. A local completion status must not conceal unfinished reconciliation elsewhere. Retain an explicit integrated disposition separate from individual unit summaries. This is a review design for the case, not a statement that the inspected engine automatically performs every proposed comparison.

Design the return path when a stage discovers a missing rule

Imagine a construction unit finds that v1 records have no stored identifier. The current plan assumes every request can resolve a version directly. Construction can report the discrepancy with a synthetic example and the affected interfaces. It needs a disposition before a worker starts classifying records according to whatever rule makes the first test pass.

An operator can route the question back to the owner of the affected decision. Present the evidence, alternatives and affected scope. For example, if a known legacy cohort can be classified by an existing release marker, one design can migrate that cohort explicitly. If there are records of unknown provenance, the owner needs to decide how the service treats them while classification is unresolved. Write that answer in the durable decision record and refresh the construction input that depended on the older assumption.

A boundary review can check whether the revised record reached downstream work. Inspect the migration plan, the decision resolver and the acceptance fixture. Each should use the same interpretation of an older request. A profile may supply a place to record or review the return, while the team still owns the content and acceptance of the decision. An example audit

row could contain the original unit, the observed discrepancy, the revision of the new decision and the conditions under which construction resumed.

This is also an opportunity to test whether the selected route captures exceptions clearly. If the normal stage record only gives the newest plan, retain a link to the observation that prompted the revision. If the process preserves both, identify which record controls execution. An operator returning later should be able to see how an apparently approved plan gave way to a different construction instruction and who authorized the change.

Human gates and execution authority

The phrase human oversight is not a complete control specification. A review prompt, a required response, a stage-transition check and a tool permission each constrain different actions. The selected profile and environment determine which mechanisms are active.

The inspected profiles describe guard policies that can stand aside for some work while human-turn authority or certain plan-approval duties remain. They also expose overrides. We should not summarize this as either “all decisions are enforced” or “all approval is off.” The actual policy is more specific. Profile guard and approval settings

A selected plan-approval hook contains a disable-flag branch that records bookkeeping where possible and returns without the fence refusing the call. That is a static observation of a few source lines. It does not establish every bypass, a security vulnerability, or the behavior of an installed environment. Inspected disable branch

The review record should therefore identify at least four things: the decision being gated, the evidence shown, the owner authorized to respond, and the mechanism that constrains the next action. If a tool can continue after an advisory finding, that is a different arrangement from a blocking check. Both may be appropriate under different constraints, but they support different assurance claims.

For example, accepting the development plan for the two-approval policy does not perform a business approval in the SaaS. The operator's plan response should confirm the intended policy and change boundary. The application's later decision path must independently enforce eligible actors, distinct approvals and creation-time binding.

In a future configuration exercise, test the gate with harmless representative actions under declared permissions. Record which action is blocked, which is allowed, how an authorized override is represented and what survives in the audit trail. We have not performed that exercise. A diagram showing a stop sign cannot substitute for it.

Human attention also has a scope. An operator can approve a long plan after checking only one visible concern. Record what was reviewed when it affects the conclusion. Do not infer thorough business, architecture and security review from a single acknowledgement whose content is unknown.

The same discipline applies to autonomy settings. Broader execution permission changes who may take the next step; it does not change the truth of a business requirement. If a workflow profile delegates more implementation choices, retain the boundary between those choices and product-policy changes. Our common requirement contract remains external to profile convenience.

Separate controls by the action they constrain

The following original control map helps prevent a single gate label from acquiring too much meaning:

Control to inspect	Relevant action	Evidence it cannot supply by itself
Human clarification	Settling the intended product behavior	Correct implementation of the answer
Development-plan approval	Accepting a defined construction proposal	Runtime permission for an application user
Workflow transition mechanism	Permitting a configured development step	Completeness of every business check
Tool/environment permission	Allowing a host action	Organizational acceptance of resulting behavior
Application authorization	Permitting a tenant-local business operation	Independence of development review

For each row, identify the actual owner, input and constraint in the selected environment. A conversational instruction and an executable block can contribute to the same duty while operating differently. The evaluation

should preserve that difference rather than summarize both as enforced oversight.

A proposed harmless gate exercise can attempt a representative local action before and after the required decision, under declared test permissions. Record expected behavior, observed behavior and audit evidence. If an authorized override is used, preserve its scope and reason. An override may be part of the intended arrangement; an unexplained one prevents a reviewer from reconstructing the control.

The static disable branch discussed above does not answer this exercise. Source inspection identifies a path worth testing. The runtime observation would establish what the installed configuration actually did for the chosen action. Neither establishes the application's tenant boundary, which needs separate business checks.

Inspect a gate at the action it is supposed to control

An illustrative operator wants the plan reviewed before the agent edits the approval service. The record needs to identify the selected profile, relevant settings, approval target and kinds of edits covered. An accepted plan can become stale if the agent later discovers a migration and changes the proposed approach. The operator should decide whether the revised plan needs another approval and how that decision will be associated with the active revision.

To inspect enforcement in a future local environment, define an unapproved action the gate is intended to block. Attempt it within a disposable fixture and collect the resulting decision and evidence of any side effects. Repeat under an approved condition with a matching action so the tester can distinguish enforcement from a tool failure. If there are several write paths, name the one used. A result on the guarded path does not settle behavior through an alternate host or configuration.

The process record also needs to preserve intentional overrides. If an operator changes settings to investigate a defect, the resulting work belongs to the override configuration. Record the reason, the action it enabled and how the experiment returned to its baseline. Readers can then separate a property of the configured workflow from a decision the operator made while diagnosing it.

Treat an override as a versioned part of the arrangement

An operator may deliberately change a default to investigate a stopped workflow. Give that override a bounded scope and retain the condition that prompted it. If the setting changes which approvals or reviews occur, the rest of the evidence should identify the configuration active at each consequential action.

After the investigation, record whether the original setting was restored or a new configuration was accepted. A later reviewer can then interpret favorable outputs under the arrangement that produced them. An override log is especially valuable when a transcript contains both blocked and permitted actions. Without the settings at those moments, the sequence can suggest contradictory enforcement where the actual difference was configuration.

Construction and acceptance evidence

Construction organizes the proposed implementation, but acceptance remains a relation between agreed behavior and observations. Unit-by-unit progress may help expose a foundational design choice early. Stage-by-stage work may make common design decisions visible before generation. These are possible uses of the documented routes, not measured advantages in this project.

For the base increment, the acceptance boundary is unchanged from the other methods. A valid request begins pending. An eligible different tenant member can decide it. A requester cannot approve their own item even with overlapping roles. Foreign-tenant reads and decisions remain prohibited under the agreed disclosure contract. Successful transitions require matching durable history.

The selected route should identify where these obligations enter its native artifacts. An original review map could be:

Obligation	Where to seek the process decision	Independent observation needed later
CS-04/05 actor rules	Intent, requirements and the approved decision design	Pending-state positive/negative actor checks

Obligation	Where to seek the process decision	Independent observation needed later
CS-03 tenant boundary	Data-access and interface decisions	Existing foreign record queried by direct ID/list/search paths
CS-08 retries	Frozen key/payload contract and persistence design	Repetition and conflicting reuse checks
CS-09 concurrency	Decision-state design and transaction assumptions	Controlled competing actions with retained state/history
CS-10 audit	Business audit design	Restart and tenant-scoped history observations

These rows identify future work. The current chapter supplies no implementation, transaction strategy or passing check. A source-native verification report may provide useful evidence, but it must be examined against the same external obligations.

A successful stage should also expose what was not checked. If a generated suite covers the positive approval route but no overlapping-role requester, CS-05 remains unresolved. If an operations stage is a placeholder, it cannot close deployment readiness. The completion record should retain that distinction rather than let a lifecycle diagram imply universal coverage.

In E2, the first approval of a high-risk v2 request is a valid partial decision while the request remains pending. The second distinct eligible approval completes it. Replaying the first actor must not substitute for another distinct approval. A framework's process approval gates do not establish this application state machine.

When tests disagree with the requirement, decide which side is wrong. A generated test that expects retroactive v2 thresholds for v1 requests contradicts the accepted case direction. Updating the test may be correct, but the record needs to preserve why the expectation changed. The same applies when a requirement itself needs authorized revision.

Retain evidence across an incomplete boundary

Suppose construction of decision handling is ready for review while history integration is still incomplete. The completion record should identify the implemented scope and the remaining CS-10 obligation. It should not borrow acceptance from an adjacent stage or from a workflow audit entry. The boundary record needs to connect the available evidence to the duty it actually addresses.

When a reviewer finds a gap, retain the original state, finding, affected decision and subsequent work. If the gap is an omitted accepted rule, reconciliation may need to move upstream. If it is a missing assertion, the evidence plan needs revision. If the rule itself is still undefined, a product decision must precede scoring. These paths have different implications for the process comparison.

Transient engine information can matter to this reconstruction. Determine which inputs, configuration and decisions the retained artifacts actually capture and which require another record. Do not assume that committing a workflow directory preserves every relevant interaction. Conversely, capture only what is needed for the evidence duty, with any necessary disclosure/redaction limits stated.

Review a construction exit at the profile boundary

A proposed exit for the approval service contains a build record, a policy check and a service-level acceptance probe. The build identifies the runnable revision. The policy check exercises the distinct actor threshold. The service probe starts with a pre-switch request and confirms it retains v1 behavior. Each item carries a different claim. The reviewer should be able to open the underlying evidence and see the environment and inputs for each.

Suppose a profile produces the first two items while the service probe is a team-owned duty. The boundary record can mark construction complete under its own criteria and keep application acceptance open pending the third observation. This distinction allows the team to use the stage result without overstating it. If the same configuration is later evaluated in a comparison, record the team-owned duty and the work it took to fulfill it as part of the evaluated arrangement.

A late correction can reopen the relevant observations without discarding the whole packet. If the build was rerun after changing the resolver but the service probe still refers to the previous revision, attach the refreshed build to the new revision and list the open probe. If the check environment uses fixtures that bypass identity, record that as an observation boundary. A subsequent review should be able to tell which claims were checked and which depend on another stage or control.

Inspect a review finding that survives a stage exit

An advisory review can identify a material question even when the native stage is allowed to finish. Give the finding an owner, an affected requirement and a disposition before interpreting the exit in a release packet. The team may accept some questions for later work and require others before a consequential action.

For the case, a finding about unresolved historical-data classification affects the meaning of a migrated request. The construction record can identify the missing decision and the fixture that would be needed to observe it. A release owner can then see why a local component pass leaves rollout open.

If the owner resolves the question, attach the answer and the revised observation to the same finding. If the team accepts a bounded exception, record its population and expiry condition. A later reader can distinguish an intentionally deferred duty from an issue that disappeared when a workflow status changed.

Operations, teams and local boundaries

An operations phase names a category of work, not evidence that the service is operable. The inspected full phase map includes operation, while some profiles reduce or leave parts of that scope as placeholders. Profile-specific coverage must remain visible. Phase map, profile scope

Our exercise is local and synthetic. It excludes production federation, live vendor provisioning, billing and regulatory certification. A future local restart or migration check could establish a bounded observation about that environment. It would not establish service-level recovery, incident response, backup restoration or real customer support.

Operational readiness would require additional owners, environments and evidence. Who can stop a release? How is a problematic policy deployment recognized? Can the business state and audit history be recovered together? Which changes are reversible, and what happens to requests created under the new policy? Chapter 12 develops these questions without pretending the present case answered them.

Team operation has a similar boundary. The source describes team-owned units and conditional parallel work with dependency/ownership rules. Those mechanisms deserve inspection under real people and checkouts. They do not prove faster coordination or fewer conflicts merely because the implementation supports them. Construction/team routing

For a team evaluation, retain waiting as well as active work. A person may wait for an owner to clarify a policy, for a dependency to complete or for an environment to become available. Those delays have different causes. Combining them into one unexplained lifecycle duration would hide where the arrangement helped or constrained work.

A solo documentary walkthrough remains valuable for explaining the intended route, artifact roles and authority questions. Its limits should determine the claims made, rather than be treated as reasons to invent a team story. No organizational outcome is reported here.

Carry policy meaning into operational preparation

For this case, operational preparation has a specific business question: what happens to requests and history if a policy deployment is interrupted or withdrawn? Returning a global setting to an older value would not necessarily undo already accepted decisions or restore creation-time bindings correctly. A rollback label cannot settle that behavior.

Before an operational exercise, define the expected treatment of requests created before, during and after the transition, including terminal decisions already recorded. Preserve the case's immutable accepted history and version-binding rules. The selected storage and migration design must supply an actual recovery procedure; none is selected here.

A useful future handoff would identify the person who can stop the change, the observations that expose inconsistent state/history and the evidence required before normal work resumes. Chapter 12 develops those assurance

questions. Their presence here explains what the lifecycle handoff must carry, rather than claiming that an operations phase has already discharged them.

For real team work, inspect handoffs between owners as well as records between stages. A waiting interval might reflect a missing policy answer, a shared dependency or unavailable review capacity. Record its cause before interpreting lifecycle duration. A solo operator moving through the same labels would not establish how independent people resolve those dependencies.

Hand off a rollout condition with an owner and a signal

A proposed construction exit can include an observation that v1 and v2 behave correctly in a disposable environment. The operational handoff needs a different level of detail. It should say what environment was exercised, which state must survive the real rollout and who decides whether to pause writes if the migration changes the expected behavior of an existing request.

Imagine the rollout signal shows successful HTTP responses while the business check reports a v1 request still pending after an eligible approval. The runbook should tell the operator which signal governs the stop decision and where to find the checked request. It should preserve enough information to distinguish a policy error from a failed observation. A later recovery decision should say what has to be checked before reopening writes.

An organization can integrate its existing release controls at this boundary. Record which duty the organization supplies and which workflow output it consumes. If the selected profile ends with a handoff, the team's release record can carry that duty forward. If a stage produces a placeholder, the handoff should identify the actual owner and outstanding work before anyone interprets stage completion as authorization.

Review the handoff from construction to a separately owned release

In a proposed team arrangement, construction produces a candidate and a scoped acceptance packet. The release team receives that packet and adds its own deployment and recovery observations. The handoff should identify what is accepted, what remains open and which environment each result describes. An interface between teams can carry these items even when their internal tools differ.

Try the handoff with a deliberately missing migration rehearsal. Ask the release owner to identify the absent observation and the condition that prevents the stateful rollout from proceeding. Ask the construction owner to identify which work can continue while that observation is prepared. The two answers should agree about the boundary and preserve the same candidate identity.

Then add a revised candidate after a resolver fix. The packet should make the stale result visible without obscuring the still-applicable product decision. This exercise examines whether the selected profile and organizational controls can exchange evidence with enough precision for their respective decisions.

Limits and conditional fit

The inspected implementation provides explicit lifecycle routing, persistent records, configurable review/guard behavior and construction ownership arrangements. These mechanisms make particular decisions inspectable. They also make configuration part of the method comparison rather than a footnote.

Our conditional judgment is to investigate an AI-DLC profile when the team needs an explicit route across intent, construction and delivery duties and can assign real owners to its decisions. Begin with the route required by the task, not the longest diagram. Record any local substitute or omitted operational duty.

Before an executable comparison, resolve the installed artifact identity, selected profile, guard settings, host/model capabilities and common application constraints. Before an effectiveness claim, collect actual behavior and effort evidence. Before a production claim, add operational validation beyond the local exercise.

The original vendor's benefit language, a named lifecycle stage and a configured gate do not collapse those evidence requirements. We have not established that this arrangement is faster, safer or more suitable for enterprise work than its peers. We have established which source-specific mechanisms and qualifications a future evaluation must preserve.

The question carried into the comparison is concrete: which decisions can the selected route make visible and constrain, which duties remain outside it, and what observations would show that the arrangement worked for this task?

7. AI Unified Process - Deep Dive

Define the method and adapter boundary

AI Unified Process (AIUP) makes requirements, use cases and their downstream relationships central to its documented workflow. The engineering question is what those relationships let a reviewer establish and how they survive change. A set of identifiers is a starting structure, not proof of aligned behavior.

The inspected artifact map connects requirements, domain records, use cases, architecture decisions, test journeys and acceptance/release documents. It identifies producers and consumers, including human responsibilities. This supplies a concrete process to examine without adopting every implementation product as part of the method. Core artifact flow

The TypeScript / Node.js case preference makes adapter scope material. The inspected combined construction plugin targets NestJS, Drizzle over PostgreSQL and a Next.js App Router frontend. That is a particular application arrangement. Choosing a language family does not automatically choose these technologies or demonstrate that the plugin works in the book's environment. Combined Node adapter

Keep three layers separate: core behavioral artifacts, the selected construction path, and any host or product used to operate it. A core specification convention can be common across paths while the implementation and coverage mechanisms differ. A repository snapshot is also not an umbrella semantic version for the entire methodology.

This chapter therefore examines core artifact/maintenance rules with the combined Node path as a documentary candidate. No plugin installation, database, frontend or compiler has been frozen. The future comparison must either adopt a compatible common stack deliberately or disclose the work and limitations of an adaptation.

The case remains our original vendor-access service. Its actors, tenant boundary and policy change come from the common specification. We do not reproduce an external book's application or sequence of exercises. Lawful

full-text originality review remains outstanding; independently composed case material does not close that review by itself.

Identify which part of an adapter is being evaluated

A stack adapter can supply construction conventions, testing guidance and ways to interpret markers. Inventory the pieces actually used in the candidate setup. A document that describes an implementation route supports a claim about the prescribed route. A successful local build would add evidence about its compatibility in the chosen environment. A coverage report would address another boundary again.

For the Node case, keep the inspected limitation on a shipped equivalent auditor visible when describing traceability. A team may perform the review manually or adopt another supported arrangement. Record that work in the evaluation packet, including who interpreted the specification-to-code relationship and what evidence they inspected. The configured arrangement then has an identifiable review duty even where automation differs.

Requirements and use-case ownership

A useful requirement identifies what the service owes its users. A use case organizes behavior around an actor's goal and the conditions under which the goal succeeds or fails. In the inspected workflow, stable requirement/use-case/rule references form a trace chain into construction and tests. The source separately describes isolated use-case tests and journeys connecting use cases. Traceability convention

For the common approval increment, the relevant goal is to contribute a permitted approval to a pending vendor-access request. The actor must be currently eligible in the request's tenant and must not be the requester. The goal cannot be specified as “click Approve” alone: that wording leaves the business preconditions and effects outside the statement. Rejection authority needs an explicit contract; the self-approval exclusion alone does not settle whether an otherwise eligible requester may reject their own item.

The following is an original use-case content sketch. Its local identifier is illustrative; the common CS identifiers remain the external acceptance vocabulary:

Field	Teaching content for `UC-001 - Contribute an approval`
Primary actor	Current eligible approver in the request's tenant
Goal	Record a permitted approval contribution on a pending request
Preconditions	Authenticated tenant membership; existing tenant-local pending request; selected policy known
Main success direction	Validate approval eligibility, apply the bound policy, preserve the contribution, resulting state and audit history
Required alternatives	Requester deciding own request, foreign/revoked actor, terminal request, retry/conflict and competing decisions
Related requirements	CS-04-CS-10, with tenant reads governed by CS-03
Open questions	Detailed interface/disclosure contract, idempotency-key scope and persistence mechanism

This sketch is not a complete executable specification. It helps a reviewer identify which business decisions are present and which remain unresolved. A generated artifact could improve or contradict it; the comparison should inspect that difference rather than force the sketch into every native file.

Ownership matters when a use case is ambiguous. A model can propose an alternative flow; the product decision owner must accept a change to intended behavior. The implementing path can choose technical details within its allowed boundary, but it must not silently settle an unanswered business limit.

The workflow's explicit authority categories are scoped to layered orchestration. Its defaults distinguish agent-alone decisions, questions requiring an answer and prohibited unilateral changes. The combined Node path has separate instructions. We should not apply the layered list universally merely because both paths belong to AIUP. Construction inputs and scoped decision rights

For our teaching case, authority remains explicit: the author supplies the accepted business direction; technical choices and actual acceptance need review. The same author acting in several roles is not independent validation. The workflow can organize responsibilities, but it cannot create an outside reviewer by naming an agent role.

Specify the actor's contribution, not only the final state

The E2 policy changes the meaning of a successful approval action. Under high-risk v2, the first eligible contribution succeeds while the request remains pending. A use case whose only success outcome is request approved would leave this legitimate intermediate result unclear. The behavioral record needs to distinguish recording a valid contribution from completing the overall request.

An original refinement of the sketch can express a common entry condition, the actor's permitted contribution and the resulting state under the bound policy. Alternatives then address requester exclusion, missing current eligibility, an already terminal request and duplicate/conflicting commands under the future contract. This is a content design, not a demand for one particular native decomposition.

Avoid resolving that ambiguity by splitting the same rule into several uncoordinated descriptions. Separate use cases may be appropriate when actors' goals or responsibilities differ. They still need one consistent account of policy and shared invariants. A reader should be able to identify which accepted requirement determines each outcome.

The actor is also more specific than approver. A person can possess that role and remain disqualified for their own request. A person can have contributed validly earlier while a later command must satisfy current eligibility. The effect of revoking a prior contribution remains a separate open contract question; adding an alternative flow must not silently decide it.

Write an alternate flow with its own observable result

In the proposed approval use case, the primary flow begins with a pending request and an eligible actor. It ends with the state allowed by the bound policy. A useful alternate flow covers an overlapping-role requester. That actor has enough role privileges to reach the ordinary eligibility check and still cannot approve their own request. If the flow only says “authorization fails,” an implementer may exercise it with a missing role and leave the requester exclusion untested.

Write the preconditions and preservation obligation separately. The actor is a current member of the tenant. The request is pending and belongs to that actor. The command reaches decision authorization. The expected result is

the contract's denial behavior with the prior state and success history preserved. The exact response and any permitted denial event belong to the selected interface contract.

An alternate flow for a second distinct actor begins in a different state. The request is high-risk v2 and already carries one eligible approval. The second actor must differ from the first and satisfy the same eligibility constraints. This gives the reviewer a path to inspect the threshold without conflating actor identity with request retries. A repeat by the original actor and a replay of the same command can have separate observable results once the replay contract has been settled.

Review the flows against the decisions they own. If a variant requires a different response for foreign-tenant lookups, make the disclosure decision explicit before assigning an expected result. Link common invariants rather than copying them into every variant. A correction can then update the authoritative rule and identify which flows need revised evidence.

Persistent specifications as a working record

The core workflow selects construction context through linked requirement rows, a use case, domain information and applicable accepted decisions/conventions. It does not describe every construction task as reading the whole catalog. This makes the completeness of the relevant links consequential. Construction inputs

Suppose the illustrative use case links CS-04 but omits CS-05. An implementer could see the membership obligation while missing the self-approval exception. A well-formatted requirement line would then carry incomplete intent. Review must ask whether all applicable constraints are linked, not only whether listed references resolve.

Persistent artifacts can make that omission inspectable. They also create a maintenance duty: changes must reach the places that depend on them. Repetition of the same rule in several records can produce disagreement unless ownership and reconciliation are clear. Keep a stable requirement's meaning authoritative and use links to connect its applications where the selected convention permits.

AIUP's inspected specification review separates deterministic lint from judgment-based semantic review. The former checks defined structural

conditions; the latter raises findings requiring interpretation. The source explicitly separates their blocking authority and keeps the review report distinct from fixing the findings. Specification-review boundary

For the case, three questions should remain separate:

Question	Example	What it establishes
Does the reference resolve?	The use case names an existing requirement	Structural linkage
Is the linked intent complete and consistent?	The requester exception and tenant boundary apply to this operation	A semantic judgment requiring review
Does the application realize it?	An otherwise eligible requester cannot approve a pending item	Behavioral evidence from an actual check

Passing the first does not settle the other two. A semantic reviewer can also be mistaken, so retain its findings and resolutions rather than transform judgment into certainty. Where a report is advisory, the acceptance owner needs an explicit disposition of material issues.

The persistent record is most useful when a later engineer can recover why the behavior exists and which change is now intended. It is less useful if status labels outpace evidence or if a current-looking artifact quietly contains an old policy. Long-term benefit therefore remains an outcome to investigate, not a consequence of file retention alone.

Inspect missing links as well as broken ones

A structural check can discover that a named requirement does not exist. A different failure is that an applicable requirement was never named. The latter may leave every visible reference valid. For the decision use case, current membership and requester exclusion both apply; linking only one can produce a clean-looking but incomplete construction input.

An original trace review therefore begins in two directions. From a requirement, identify every behavior that should consume it. From a use case, identify the constraints that apply even when the current links omit them. Compare those sets with the selected native record. The review is about semantic coverage, not a count of references.

Then follow a concrete alternative into its check. The requester-exclusion alternative needs an otherwise eligible requester and a pending request. A label attached to an assertion that only exercises terminal state would not establish the intended behavior. The implementation location, test fixture and observation must agree about the rule being examined.

This review can produce a useful finding without executing an application: a missing link or contradictory expectation is a documentary defect. It is not yet an observed application failure. Retain the finding, its basis and the affected downstream inputs so later construction does not inherit an already known omission.

Give semantic findings an explicit disposition

Because structural lint and semantic judgment have different authority, a review report needs a route for consequential advisory findings. An acceptance owner should be able to see whether a finding was accepted, rejected with reasons, deferred outside scope or left unresolved. The existence of a report does not establish that someone acted on it.

For an illustrative contradiction, a use case may describe current same-tenant eligibility while an alternative implies that administrator status bypasses requester exclusion. The correction needs the accepted business rule, not a vote among generated perspectives. If the rule is already clear, reconcile the record; if it is genuinely open, obtain the responsible owner's decision before treating it as executable intent.

Keep review and repair distinct in the evidence trail. A report authored before correction and a revised specification serve different purposes. Preserve enough identity to tell which implementation consumed which version. Otherwise a final clean document can conceal that the original construction input was materially different.

Maintain a small glossary at a shared business boundary

The approval case uses words that can hide different identities. A requester is an actor in relation to a specific request. An approver role is a privilege held through membership. An approval is a successful contribution to a particular decision history. A repeated command can refer to the same contribution. Writing these meanings once gives the use-case owner and implementer a common starting point.

The distinction becomes concrete in a fixture. Mira can hold an approver role and still be the requester excluded from this request. Leon can send the same command twice and still contribute one distinct approval. Priya can provide the second eligible contribution. The glossary should support those examples and identify where each meaning is applied. It need not describe every class or field.

Review terminology when an implementation introduces a new name. If a field called approvalCount actually stores successful commands, an engineer should inspect whether the name reflects the business concept and whether replay can inflate it. The glossary cannot repair the implementation, but it can make the mismatch visible. A reviewed rename can preserve behavior while making a later defect easier to recognize.

A shared term can also be too broad. A status called complete could refer to the use-case document, construction work, test coverage or the business request. Qualify the record so a contributor can tell which lifecycle it belongs to. A reader should be able to follow an implementation status without mistaking it for a release decision or a terminal request state.

Keep glossary changes under the specified ownership policy. If an agent discovers a contradiction, it can report the competing uses and affected records. The owner can then decide whether the rule needs clarification or the code needs reconciliation. The resulting change should identify downstream examples that need fresh review.

Implementation and traceable tests

The construction path should connect the reviewed use case to a bounded implementation and appropriate checks. For the inspected Node adapter, the test skill describes a unit tier with stubbed repositories and a real-application HTTP tier using Supertest and PostgreSQL in Testcontainers. This is a prescribed testing arrangement, not a suite executed here. Node test tiers

Those tiers address different uncertainties. A pure policy check can inspect the distinct-approver rule under selected inputs. An application/database check can inspect whether actual persistence and request handling realize that rule. Neither should be described as proving the whole service secure. Their fixtures and assertions determine the observation's scope.

The workflow explicitly qualifies coverage automation: the combined Node adapter does not yet ship the equivalent auditor described for certain other adapters. Manual checking is discussed as a route. Preserve this limit when evaluating traceability rather than borrowing another stack's assurance claim. Coverage-auditor boundary

For the common case, a requirement-to-evidence map might identify:

Requirement	Implementation question	Check required later
CS-04	Is current same-tenant eligibility checked at decision time?	Eligible actor succeeds; revoked/foreign actors fail under appropriate fixtures
CS-05	Is requester identity independently considered?	Pending-state overlapping-role negative case and separate positive control
CS-08	Is retry/conflict behavior consistent across state and audit?	Same-key repetition and conflicting reuse under the frozen contract
CS-09	Is the policy preserved under competing decisions?	Controlled concurrency with final state and history inspected
CS-10	Are successful transitions and history durable and tenant-scoped?	Restart and foreign-history access checks

These are proposed checks. The application, database configuration and independent oracle remain unprepared. Source-native tests can contribute to the evidence packet, but their expected behavior must be checked against the agreed requirement rather than inferred solely from generated code.

The implementation skill's scope also matters. Its instructions separate implementation from test generation and restrict changes to existing tests/documents for that role. This does not mean that the entire method never changes tests. It means the selected workflow assigns that work elsewhere. Node implementation role boundaries

At acceptance, distinguish a structural marker, an agent's coverage finding and a human decision supported by actual observations. A rule marker can help locate code; it cannot prove the code's condition is correct. A coverage matrix can expose gaps; it needs inspection of what each covered entry

means. Human confirmation can close a workflow state while leaving wider operational claims unresolved.

The transition guidance describes acceptance records with named results and human confirmation before accepted use cases reach Done. This is a documented decision procedure, not a completed round for our example. Transition and acceptance

State which instruction set governs construction

A reproducible profile needs the core revision, selected adapter, applicable construction instructions and any local substitution. Do not combine a layered-orchestration decision rule with a Node implementation command and assume that the combination is the documented default. Record why each instruction applies and where its authority ends.

For the book's hypothetical Node path, an original profile note would identify the accepted requirements/use case, the stack assumptions, the implementation/testing role split, the chosen review route and outstanding environment checks. A language-family preference cannot fill the database, framework, package or compiler fields. An adapter's availability is a compatibility lead until those dependencies are actually exercised.

Role boundaries become practical when intended behavior changes an existing test expectation. If the implementation role is not allowed to edit that test, the workflow needs an explicit handoff to the responsible testing role. The old expectation should not be preserved merely to avoid a prohibited edit, and the implementation role should not silently exceed its scope. Reconcile the accepted rule through the assigned route.

That handoff has a cost and an observable outcome. Retain the question, relevant requirement revision, test change and actual command/result when they exist. A future evaluation should count compensating manual work rather than attribute it to an automatic adapter capability.

Match each test tier to its uncertainty

A repository-stubbed policy test can vary eligibility and contribution inputs cheaply, but the stub does not establish real query scoping or transaction behavior. A real-application HTTP/database tier can inspect those integration boundaries when it is actually configured with meaningful fixtures. Its existence in a skill description is not an executed observation.

For CS-03, establish a known foreign record and a permitted positive control at the real boundary being checked. For CS-10, observe the required state/history relationship across the declared persistence/restart procedure. For CS-09, define legal competing histories before choosing the concurrency exercise. Those duties do not become satisfied merely because tests are categorized as integration tests.

The Node coverage-auditor gap makes the selected review route especially important. A manual review can be legitimate when its owner, scope, inputs and decisions are explicit. It still requires inspection of assertions and omissions. Counting requirement markers or rows marked covered cannot establish the semantic relation between behavior and evidence.

Inspect the path between a use case and an observation

Take the use case in which an eligible actor approves a high-risk v2 request for the second time in the sequence. The use case's success condition needs to identify a second distinct actor, a terminal state and durable history. A structural check can establish that the use case identifier appears in construction and test records. A reviewer still needs to read the fixture and determine whether it supplies two different actors and exercises the expected transition.

In the Node teaching arrangement, map that observation through the selected application layers. A policy-function assertion may confirm that two different actors are counted. A service-level probe can check that an authenticated command reaches that policy with the correct request version. A persistence observation can check that the final state and history agree. Label the evidence by boundary so one passing assertion cannot be mistaken for coverage across all three.

If the implementation reveals an absent contract decision, record it at the behavior level before reconciling the lower-level artifacts. For example, two

calls could present the same key with different payloads. The use-case owner needs to settle the expected result before a construction path can produce meaningful acceptance evidence for it. A developer's local workaround is useful as a proposal and should be reviewed in that role.

Policy change and existing systems

The inspected change guidance begins with updated intent/requirements, reconciliation of affected specifications and downstream implementation/tests. Its reverse-engineering guidance treats recovered artifacts as a proposed baseline whose gaps and contradictions need resolution. Change and existing-code guidance

The word proposed matters. Existing code can reveal what the service does, but the desired behavior may differ. If reverse engineering documents self-approval as allowed because that is the current implementation, the accepted case rule still requires a correction. A recovered specification cannot acquire normative authority simply by describing code accurately.

For E2, the revised intent must include both the new high-risk threshold and the preserved creation-time policy for v1 requests. Review the effects on eligibility, distinct counting, pending/terminal state and audit. Some requirements stay unchanged but remain relevant to every modified use case.

An original change-impact note could separate:

- Changed behavior: new high-risk v2 requests need two distinct eligible approvals.
- Preserved behavior: v1 requests retain their bound policy; standard-risk v2 still needs one.
- Preserved invariants: tenant locality, no self-approval, retry consistency and durable history.
- Technical uncertainty: representation, migration and concurrency mechanism, to be settled under the common stack/contract.

The selected Node implementation instructions favor reconciling existing functionality in place, preserving unrelated behavior and adding migrations rather than rewriting already-applied ones. This is a source-specific change discipline. We have not exercised a migration or established that the

application's schema realizes the intended policy binding. In-place reconciliation

For a fair episode, every method starts from the same prepared baseline, not its own potentially different E1 output. Preserve the native AIUP artifact trail while retaining common business obligations and external checks. If the adapter needs a different stack or a custom adaptation, report that intervention and its preparation effort.

The smaller E3 repair should also preserve proportionality. A tenant query correction may require a narrow implementation change and meaningful regression evidence, while specifications clarify the intended boundary. The inspected set does not establish the minimum artifact cost for every tiny AIUP task. We leave overhead unmeasured rather than infer it from the number of available documents.

Reconcile the change without rewriting its history

A proposed change-impact review starts with the accepted delta: new high-risk v2 requests need two distinct eligible contributions. It then names preserved cases, affected use cases, shared rules and technical decisions still open. An unchanged rule such as no self-approval can remain relevant even when its requirement text does not need revision.

Follow that delta through construction inputs. A use case that still treats every successful contribution as terminal completion needs reconciliation. A data representation that cannot retain creation-time policy needs a technical decision. A test expecting retroactive v2 thresholds needs correction to the accepted rule through the responsible testing path. These are distinct effects of the same change.

Preserve the prior accepted specification and the new revision in the evidence history. A current catalog should explain intended current work; the earlier version explains what a prior implementation or verdict was assessed against. Replacing the old file's meaning without retaining identity would make later maintenance claims difficult to examine.

For recovered existing systems, use a three-column investigation note: observed behavior, accepted intended behavior and reconciliation required. If observed code permits self-approval, record that fact without turning it into an accepted product rule. If a recovered use case is incomplete, mark the

baseline proposed rather than declaring requirements complete because a reverse-engineering step ended.

Preserve the meaning of a use case across a change in policy

An owner can express the E2 policy with one new variant for high-risk v2 and a preservation clause for requests that predate the change. This avoids duplicating the whole approval use case while showing which path now remains pending after the first eligible action. The use case still needs common invariants for tenant scope, actor eligibility and the relationship between the committed decision and its history.

A reconciliation review can look for two kinds of omission. The implementation may have added the new variant while forgetting the preserved path. The tests may cover the preserved path only by seeding records directly, leaving version selection at creation unchecked. Ask the contributor to show how each observation starts, where the stored version comes from, and which actor makes each decision. A use case with a readable flow gives the reviewer a stable basis for those questions.

Existing implementation evidence can reveal further variations. If older data lacks an explicit version field, the use case cannot settle the migration by assuming all rows were created the same way. The architecture and product owners should agree which evidence classifies old records and what happens when the classification is unclear. Carry that decision into construction and acceptance. Record any unresolved population separately so a check on clean synthetic rows cannot stand for it.

Review an obsolete flow before removing it

A product change can retire a use case while records created under that behavior remain in the system. Before marking the flow obsolete, identify which existing records still require its interpretation and whether any client or operational procedure depends on it. A new use case can govern future work while a historical rule remains necessary for maintenance.

For the approval case, retiring v1 for new creation does not decide how an old pending request is resolved. Preserve the relevant behavior and acceptance example until its population has an agreed disposition. The requirement record should let a reviewer see that distinction without relying on an obsolete implementation plan.

If the team later removes the compatibility path, give that change a separate decision and evidence. Identify the population inspected, the condition that makes removal acceptable and the owner who accepted it. A structural status update can then reflect the reviewed lifecycle decision rather than become the only evidence for it.

The same increment, original artifacts

Our teaching use case concerns a business decision, not a sample application borrowed from another publication. The neutral CS requirements stay external to native notation. A future AIUP record would map its own stable identifiers to those requirements so readers can inspect coverage without confusing the book's numbering with generated framework output.

The resulting trail would be intelligible in both directions. From CS-05, find the relevant use case, the requester-exclusion rule, implementing location and acceptance observation. From an implementation change, find the rule or technical decision that justifies it. If either direction ends at a generic statement such as “authorization handled,” the record needs more specific inspection.

Do not reproduce a full prompt collection to show this trail. The important content is the business decision and its consequence, not every instruction used to create a file. Likewise, a complete framework tutorial would distract from the comparison unless a particular setup choice changes the intervention.

For the base increment, the original artifact set we would expect to inspect includes a bounded requirement/use-case record, applicable domain/architecture decisions, open questions, implementation rationale and test/acceptance evidence. That is a functional inventory, not a claim that a workflow emitted those files here.

The case also keeps application architecture separate from process fit. NestJS/Next.js may be a plausible candidate for the inspected adapter, but plausibility is not a shared-stack decision. If the book chose it solely because one adapter requires it, that choice and any resulting advantages or constraints would need disclosure in the comparison design.

Human inspection should focus on whether persistent records carry the correct behavior through change. A use case that omits creation-time policy

binding can be polished, traceable and wrong for E2. An implementation that matches it would still fail the shared contract. This is why equal external obligations matter alongside native artifacts.

Walk one obligation in both directions

Begin an original paper exercise with CS-05. Locate its native requirement equivalent, the decision use case, the requester-exclusion alternative and the construction input that must carry it. Identify the proposed implementation boundary and the independently expected fixture. Leave actual code/test/result locators empty until the application exists.

Now reverse the exercise. Start from a proposed assertion denying an overlapping-role requester. Ask which accepted rule makes denial correct, whether the fixture isolates that rule and which use case requires the alternative. If the assertion instead targets a terminal request, the reverse walk exposes a mismatch even when the forward references look complete.

Repeat for CS-13, where creation-time binding crosses creation, decision and historical interpretation. One local success assertion cannot cover that whole chain. Identify the places that must retain the policy's meaning and the later observations needed to assess them.

The resulting artifact is a review map with specific gaps, not a simulated coverage result. Its usefulness lies in making a missing relation challengeable. Whether an executed AIUP arrangement produces and maintains a better map than another method remains a separate empirical question.

Reconcile a structural pass with a semantic finding

Suppose the requirement, use case, construction record and tests all carry the right identifiers. The structural report is green. A reader of the actual fixture notices that the purported second approver is represented by the same actor with another role. The test expects approval after two commands. Its trace links are intact, yet the linked example contradicts the intended behavior.

Record the defect at the level where it occurs. The specification is clear about distinct actors in this example. The test expectation needs repair, and the implementation must be inspected to see whether it also counts roles or commands. Use separate fixtures for the repeated actor and the permitted second actor. The review can then distinguish a test problem from a behavior

problem. Retain the structural pass with its own scope and record the semantic finding and disposition beside it.

If the use case was ambiguous, the owner would first decide what counts as distinct. A role-based implementation might have been a reasonable interpretation of an incomplete record. In that situation, preserve the observation, revise the owning specification and reconcile the construction and test records from that decision. The disposition should show whether the new work corrects a defect against existing intent or implements a newly settled rule.

Limits and conditional fit

The inspected AIUP design provides durable requirements/use-case relationships, specified construction inputs, review conventions and an explicit downstream change path. These mechanisms support concrete questions about ownership and traceability. They do not establish automatic synchronization, reduced maintenance cost or superior acceptance quality.

Our conditional assessment is to investigate this arrangement where a team wants owned behavioral specifications to remain a working product record and can assign the maintenance/review duties they require. Verify that the selected adapter fits the application constraints and that its actual coverage route meets the project's acceptance needs.

The Node auditor qualification, role-specific edit boundaries and stack assumptions are material. Keep them in any summary of fit. Core portability in design does not prove that every construction plugin is interchangeable, and a different plugin's completed check does not certify this one.

No controlled AIUP-versus-peer advantage is established here. Tiny-task overhead, real human-team coordination and long-term drift outcomes remain unassessed. Transition documentation is useful to inspect, but it is not evidence of production operation, recovery or regulatory compliance.

Independent source review and full-book originality/permissions review also remain open. The chapter's independently composed artifacts and shared synthetic case establish a distinct working route; they do not replace external scrutiny. We will carry the same limits into the SDD and comparison chapters.

The practical question is whether maintaining this behavioral record helps the team make and check its next change under real constraints. The method's design tells us where to look. Actual observations must establish what happened.

8. Spec-Driven Development as an Approach

From the primer to a lifecycle policy

A specification is most informative when something changes. Before implementation, it can make intent explicit. During review, it can identify the behavior the code owes its users. During maintenance, it can preserve an earlier decision or reveal a conflict. Those roles require a policy for ownership, retention and reconciliation.

Chapter 3 introduced the vocabulary. The four deep dives then showed why the label spec-driven is insufficient. A maintained use case, a phase plan, an epic contract and an original request record can all guide development. They differ in the decisions they own and the period for which they remain authoritative.

Consider our E2 requirement: new high-risk v2 requests need two distinct eligible approvals, while existing requests keep their creation-time policy. A document can state that rule clearly on the day it is written. The harder question is what happens when the implementation changes, a reviewer notices an omission or a later request proposes retroactive treatment.

A lifecycle policy needs answers to five questions: which artifact owns the behavior, who may change it, which dependent records must be reconciled, how code/test disagreement is resolved and what evidence justifies acceptance. Without those answers, calling a document the source of truth leaves the maintenance procedure unspecified.

We can compare the policy at several scopes. A small repair may need one current issue and a regression expectation. A long-lived business rule may merit a maintained specification with stable references. A generated subsystem may place stricter limits on direct implementation edits. The appropriate arrangement depends on the work and the team, not document length alone.

This is why Spec-Driven Development (SDD) is cross-cutting in the book. It describes choices about how specifications participate in development. Those choices can appear within several frameworks and through different tools.

The presence of a specification does not establish a distinct fifth intervention comparable to four complete process profiles.

Define what it means for a specification to stay current

A current specification identifies the accepted behavior for the relevant population and the condition under which it applies. A recently edited document can still describe a future policy while the deployed service handles earlier requests under another version. Put the applicability boundary in the rule and its examples.

A reconciliation record should say which implementation revision and policy cohort were reviewed. If a known gap remains, identify it alongside the intended rule and name the owner of its disposition. This lets a maintainer find accepted intent without confusing it with an observation that every path already follows it.

Three attributed patterns, not a ladder

Birgitta Böckeler's 2025 analysis supplies the terms spec-first, spec-anchored and spec-as-source. They distinguish initial specification use, retention through later work and an arrangement in which humans edit the specification rather than implementation. We attribute the vocabulary and use it to ask lifecycle questions, not award quality levels. Original analysis

The order question is the smallest: was intended behavior described before implementation? It leaves retention and maintenance unresolved. A team can use a useful temporary change specification without committing to maintaining that exact artifact forever. Conversely, retaining the file does not establish that later work consults or updates it.

The maintenance question asks whether the specification remains a working reference after the initial feature. Who owns an update? Does a changed requirement propagate into code and checks? How is an outdated record detected? A spec-anchored claim becomes meaningful through those policies and their actual use.

The editing-boundary question is stricter. If implementation is regenerated from specification changes, a manual code patch may be overwritten or may break the declared process. The team needs a rule for emergency changes, generated versus hand-maintained regions, and reconciliation afterward.

These are policy questions to investigate in the selected arrangement; the label does not answer every exception.

Do not treat the strictest editing boundary as the best outcome by definition. It may fit a well-bounded generation domain and impose costs elsewhere. A manually maintained implementation can still have explicit intent, strong acceptance evidence and a useful long-lived specification. Suitability requires the scenario and observations.

Nor should the labels be assigned to an entire framework from one file. BMAD's epic specification ownership and its existing-code context policy operate at different scopes. A GSD phase record may persist as workflow history without becoming the permanent product specification. Inspect the actual artifact and change rule before making a category claim.

Set a policy for each kind of record

Consider a team adopting a specification practice halfway through the approval service's life. It already has issue descriptions, a product rule about tenant ownership, a database migration convention and a directory of generated plans. Applying one edit policy to every file would make the workflow awkward. The product rule needs a decision owner. The migration convention needs a maintainer and a way to test changes. The generated plan needs the identity of its inputs and a clear point at which it can be archived.

The team can choose a durable specification for decision behavior and keep plans local to each change. When E2 arrives, the decision owner updates the rule for new high-risk requests and records the scope of the v1 exception. The developer revises the plan and brings the repository into alignment. The reviewer asks for an observation on both versions. After release, the accepted behavior and evidence stay available, while the task plan can be archived with the revision it describes.

A different team might generate a reviewable contract from a deliberately owned source. In that arrangement, editing the generated file during a repair can hide the change from the next regeneration. The team should identify where to correct the source, how to preview downstream changes and how to handle a release whose current implementation predates the new contract. These choices concern an individual artifact and its consumers. The same team might still edit its test fixtures directly and archive short-lived planning notes.

Record	Proposed authority	Evidence that an update took effect
Behavior specification	Product decision owner	Reviewed rule and acceptance examples for affected versions
Generated interface description	Maintained upstream input	Reproduction using the declared input and comparison with the reviewed output
Implementation plan	Engineer responsible for the change	Work references the accepted decision and identifies unresolved assumptions
Acceptance record	Reviewer responsible for the observation	Fixture, revision and actual output can be retrieved

Put this policy where contributors encounter the record. During a correction, the engineer should be able to tell where to make the change without asking everyone who ever edited the file. A record that no contributor can locate or interpret has little practical authority, regardless of how carefully it was written.

Choose which records should be regenerated

A team can use generated plans without making its accepted behavioral rules disposable. Identify the input that owns intent and the records derived for a particular task. When intent changes, regenerate the task-specific material from the revised input and inspect what changed. Retain the old plan with the revision it described.

A different artifact may deserve direct maintenance. A brief decision rationale that explains a preserved policy can remain useful across many tasks. Replacing it on every generation risks losing the reason a later maintainer needs. The team can retain that rationale and reference it from new plans.

Review the policy using a concrete correction. Ask where an engineer should edit an ambiguous rule, which downstream records must change and how a reviewer checks that they did. If those answers are clear, the policy has a practical maintenance path. If contributors routinely edit a generated output because they cannot find the owned input, improve the entry point before adding more generation.

Specifications, context and executable contracts

Specifications and context overlap, but their roles differ. A glossary provides meaning. A repository map helps navigation. A business rule states required behavior. A phase summary records work. An agent may need all of them; only some define the expectation against which the change is accepted.

For the approval service, “Mira created request R” is fixture context. “The requester may not approve that request” is a behavioral constraint. “The last session changed the decision reader” is workflow history. Mixing these in one prompt does not remove the need to identify which statements are authoritative and current.

An executable contract adds a defined mechanical path. A schema validator can check properties it encodes. A test can exercise selected behavior under a fixture. A reference model can provide a decision expectation for a defined input space. None automatically captures every business requirement, and a natural-language instruction interpreted by a model is not executable merely because code follows it.

Use a separation record when the artifact's role is ambiguous:

Content	Proposed authority	Appropriate check
Policy binding at request creation	Accepted business requirement	Reviewed expectation and later transition/regression observations
HTTP error representation	Frozen interface contract, once chosen	Contract checks for the selected interface
Native workflow status	Process record	Consistency with actual work/evidence and known gaps
Test fixture identities	Exercise definition	Positive/negative controls establish that the intended boundary is exercised
Agent's completion summary	Report of work	Inspect supporting changes and observations before accepting it

The record can be compact. Its purpose is to prevent an implementation or report from quietly becoming the authority for a missing requirement. It also helps reviewers identify where a correction belongs.

Generated code need not be identical across repeated runs for a process to be auditable. Retain the inputs, source/configuration, outputs and observations needed to compare behavior. If exact regeneration is part of a claim, test that claim under stated conditions rather than infer determinism from the phrase source of truth.

Show how one sentence becomes an observable contract

Start with “high-risk requests require two approvals.” That sentence leaves open who counts, when the new rule applies and what a client can see after the first action. Add a short sequence: create a high-risk request under v2, accept an eligible approval from Leon, and leave the request pending. Submit another action from Leon and verify that the count does not increase. Accept an eligible approval from Priya and check the approved state and durable history. Keep the response to a repeat unspecified until the contract distinguishes a replay from a different action.

Add a preservation example before describing implementation. Create an equivalent request under v1 and leave it pending while v2 becomes active. An eligible approval still resolves it under v1. This example makes the scope of the specification clearer than the threshold sentence alone. It also shows why a contract author should name the creation-time assignment separately from the rule that resolves a decision.

The examples can be used at several levels. A reviewer can inspect the behavior without knowing an endpoint. A test designer can turn it into a fixture after interface and observation rules are agreed. An implementation agent can use it alongside code locations and local conventions. Give each consumer the relevant context and preserve a link back to the owned decision. Copying the full specification into every task introduces additional places to reconcile when a single rule changes.

Distinguish a behavioral rule from a convenient test example

A specification can include a concrete example that is narrower than the rule it explains. “Leon and Priya approve this high-risk request” illustrates two distinct eligible actors. It does not make those two names part of the product contract. An implementation that recognizes only the fixture identities can pass the example while violating the general rule.

Write the general condition near the example. Identify the relationship among actor, membership, requester, tenant, policy and current state. Then vary one relationship in a second example. A different eligible actor should be able to make the same kind of contribution. The requester should remain excluded when their role set changes. A reviewer can inspect whether the implementation expresses the relationship or merely recognizes the demonstration.

The same problem appears in a generated interface. An example response might contain a single approval without explaining whether the operation has finalized the request. Give the response a defined meaning and identify the business state a client can infer. If the contract permits a successful contribution while the request remains pending, show both a partial and a terminal outcome.

A test derived from an example should identify the rule it addresses. This helps a reviewer decide which fixture variations are material. Varying an irrelevant display name may add little evidence. Varying the requester relationship or bound policy can expose a different decision branch. The specification owner and test reviewer can use the same short rule to explain why one variation belongs in acceptance.

When a fixture changes during maintenance, revisit the observation rather than simply updating expected output. If a previously distinct actor becomes the requester in the new fixture, the expected result changes for a reason. Record that reason with the requirement so future contributors can distinguish a deliberate fixture correction from a test weakened to match code.

Who resolves disagreement

Disagreement is useful evidence. It means two records or observations cannot both be accepted without qualification. The next step is to identify the authoritative decision and the type of conflict, rather than automatically editing the most convenient side.

If a specification forbids self-approval but the implementation allows it, the current case points toward an implementation defect. If the test expects administrators to approve their own requests, its expectation contradicts CS-05. If the product owner now wants that exception, the accepted

requirement must be changed explicitly and the study's task identity reconsidered.

An original reconciliation map can make the duties visible:

Conflict	First question	Owner/action
Requirement versus use case	Is behavior missing, contradictory or intentionally changed?	Product decision owner resolves intent; update dependent records
Specification versus code	Is the specification still the agreed behavior?	Implementing engineer proposes correction; policy changes require authority
Code versus test	Does the assertion express the agreed expectation and fixture?	Acceptance/test reviewer checks both sides
Two specifications	Which scope owns the rule, and why were copies allowed to diverge?	Artifact owners reconcile and clarify maintenance policy
Current record versus old verdict	Which inputs changed after review?	Revalidate affected evidence rather than reuse the old status

BMAD's selected source designates `bmad-spec` as writer of the named specification and preserves capability IDs through updates. AIUP describes intent/specification changes before downstream reconciliation. Both supply a maintenance route; neither source alone demonstrates successful synchronization in our case. BMAD ownership, AIUP change route

A semantic review can identify a possible conflict without having authority to settle it. Retain that distinction in automation. A tool that changes a requirement to make a check pass may erase precisely the disagreement a reviewer needed to inspect.

For urgent repairs, preserve a small decision record even when the implementation must be corrected first under an authorized emergency procedure. State the temporary deviation and the reconciliation duty. This is an editorial process design, not a claim that every selected framework supplies the same emergency path.

Resolve a conflict in the order that protects existing users

Suppose the specification says v1 behavior is preserved, the implementation reads the latest policy on every approval and the generated test expects the current implementation. Begin with the accepted decision and an observation against a pre-existing request. If the fixture confirms the contradiction, record the behavior people may already have encountered before editing anything. The owner may need a decision about previously processed records and a separate repair for future actions.

The repair must reach every record whose role changed. Fix the resolver, revise the test that encoded the wrong expectation, and annotate the old evidence with the revision it describes. A new test should observe the state change through the public entry path used by the affected workflow. A developer can additionally retain a direct resolver test for fault localization. The decision record can then point to both and explain what each covers.

When authority is genuinely ambiguous, write down the smallest decision that unlocks the next step. For example, decide whether newly created requests retain the policy even if a reviewer changes a tenant setting later the same day. Defer unrelated policy changes to their own decisions. This keeps the reconciliation bounded and gives an acceptance reviewer an explicit expected result.

Review a conflict without changing both sides at once

Choose a synthetic disagreement between a maintained specification and an implementation. Write down the accepted rule and the observed behavior before editing either. Ask the owner whether the observation reveals a defect or a desired change that has not yet been approved. Record that disposition with the affected population and revision.

If it is a defect, repair the implementation and the checks that repeated its assumption. If the owner chooses a changed rule, update the specification and reconcile its consumers from the new decision. In either case, retain the original observation so a reviewer can understand why the work began.

Changing the rule and expected output together can be legitimate after that decision. Without the recorded disposition, the same edit can conceal a regression. The review should be able to follow the direction of authority through the correction.

One policy change across native artifacts

E2 provides a common test of specification policy. The business change is fixed; each method's native artifacts remain different. The following is a documentary trace design, not a record of four runs:

Selected source	Native record to inspect	Question for the E2 change
BMAD stable source	Maintained specification, optional story breakdown and implementation record	Does creation-time binding survive the skill-mediated update and reach acceptance?
GSD Core source	Phase decisions, plans, summaries and refreshed verification	Do workers receive both the new threshold and preserved v1 behavior?
AWS workflow source	Intent/state, selected stage/unit artifacts and boundary evidence	Does the configured route preserve the policy decision and disclose any skipped duty?
AIUP core/Node path	Requirements, affected use cases and downstream reconciliation	Do stable references preserve unchanged constraints and identify the changed behavior?

The underlying source mechanisms are described in the deep dives: BMAD's linked specification path, Core's phase records, AWS's retained workflow/audit records and AIUP's specification-led changes. BMAD, Core, AWS, AIUP

Use the same semantic probes for each. Where is v1 preservation stated? Where is distinct-actor counting stated? Which record changes when a previous interpretation is wrong? Which observations would detect a missed update? The answer may require several linked artifacts, but their number is not the outcome.

Stable identifiers help locate continuity; they do not guarantee unchanged meaning. An identifier retained while its rule silently becomes retroactive can make the trace look continuous while changing the task. Inspect text and behavior as well as reference stability.

The acceptance examples remain common: one high-risk v2 approval leaves pending, a second distinct approval completes, replay does not advance the

count, and v1 retains its threshold. No method's generated suite replaces these obligations. A future exercise must retain actual expectations, outputs and exceptions.

Decide which artifact should change during an urgent repair

Imagine an operator discovers that a search result exposes a request from another tenant. A temporary mitigation disables search while the team prepares a repair. The intended tenant rule was already recorded. The mitigation belongs in the operational record with an owner and an expiry condition. The behavioral specification can continue to describe the intended protection, while a known gap records where the current software fails to implement it.

Now suppose the repair review exposes an unresolved behavior: the contract says nothing about whether a result count can reveal matching foreign requests. Resolve that question in the owned specification before writing the corresponding assertion. Keep the prior observation and distinguish it from the new decision. Otherwise a future reader could infer that the specification had always forbidden the exact behavior exposed by the report.

A short reconciliation checklist can name the changed rule, the dependent plan or use case, the relevant implementation and the refreshed observation. The checklist is finished when a reviewer can follow each relationship. The file names and record format may differ across the four arrangements. The SDD question stays focused on which artifact represents intent and how the team restores consistency after a change.

Costs, drift and useful limits

Retained specifications require maintenance. The cost includes reading, reviewing, resolving contradictions, keeping references useful and retiring stale material. The possible benefit is a more recoverable decision record. Both sides need observation under an actual project lifetime; file presence establishes neither.

BMAD's existing-code guidance prioritizes current code and keeps historical greenfield context away from ordinary small changes. That qualifies a blanket claim that retaining and loading every planning document is always useful. It

is source advice, not a measured benefit of archiving. Existing-code context policy

A useful retention decision distinguishes current intent from historical explanation. The original reason for a policy can remain available without being loaded into every small change. A superseded requirement should not compete silently with the current one. The authoritative record needs an owner and a visible supersession relationship.

Drift is also broader than code/spec disagreement. A test can lag behind both. A generated user guide can describe an old flow. A context file can preserve a discarded technical decision. Choose inspection and reconciliation by dependency, rather than assuming one updated document repairs every downstream artifact.

For a tiny display repair, a maintained domain specification may not need a change at all. Check whether the accepted behavior or contract changed before creating new product documentation. For a policy change, that same shortcut could omit the principal business decision. Proportionate SDD therefore depends on what the artifact owns.

No measured drift-prevention or maintenance advantage is reported here. The case design offers questions and controlled boundaries; it has no longitudinal evidence. A useful future evaluation would inspect several changes, reviewer disagreements and the effort needed to keep the selected record current.

Put a maintenance cost on the adoption decision

Try the practice on one change that has a stable policy owner and a concrete preservation constraint. Record where reviewers found the rule, what had to be updated, which contradictions were detected and how much manual reconciliation the change required. The team can inspect these records after the next change to see which artifacts were actually reused. A field report should give the team, change and circumstances before drawing a lesson from the exercise.

A small documentation-only correction may need just the existing text and an inspection of the rendered result. If it affects no behavior or durable contract, creating a new behavioral specification adds an artifact with no future decision to own. For a policy change spanning creation, migration and audit,

a short maintained rule may repay its upkeep by making the dependencies visible. The team can use the same decision criteria for both changes and choose a different amount of ceremony.

Retire a redundant artifact deliberately. Identify readers and downstream links, move any unique decisions into the surviving record, and leave an archive reference if someone will need historical evidence. Otherwise a stale copy can regain apparent authority the next time an agent searches the repository. An archive should say which revision or decision superseded it and where ongoing edits belong.

Set a policy for a derived contract during migration

Consider a team that keeps a human-owned behavioral rule and derives an interface description from a maintained schema. The change to v2 adds a pending outcome after the first eligible approval. Existing clients can already display a pending state, but one downstream integration assumes a successful approval response always signals completion. The behavioral spec, generated description and integration assumptions now need to be reviewed together.

Start by identifying which consumers need a decision. The product owner can define what the first successful action means. The interface owner can decide how that outcome is represented and how older clients learn about it. The integration owner can identify the behavior that depends on the older assumption. Once these decisions are recorded, regenerate derived material from its owned input, compare the result with the proposed contract and check the affected integration. If the generated artifact conceals the meaning of the transition, add a reviewed example to the source that owns it.

There is a period in which old and new software may coexist. During that period, a contract can identify when the new behavior becomes active and what a client should do with a request created under an older policy. A repair to the generated text alone would leave the next generation or rollout ambiguous. A repair to the owned input plus a versioned observation gives the team a reproducible basis for the next change.

The team can choose a limited exception in an incident. If a client needs an urgent compatibility patch, record the operational scope and the date or event that ends the exception. Keep the canonical behavior and compatibility promise distinguishable. Otherwise a temporary accommodation can become the rule an agent discovers and repeats in an unrelated change.

Apply the lens without creating a fifth arm

Apply SDD questions inside each selected framework profile. Identify the behavioral record, its owner, its lifespan, its edit policy, its conflict procedure and its acceptance relationship. Leave the classification unresolved where the inspected scope does not establish an answer.

The experimental subject remains a configured framework applied to the common work. Adding a fifth label called SDD without a concrete workflow would create an unequal comparison unit. If a later study defines a distinct specification workflow, that would require its own scope and evidence design; it is outside this book's four-arm plan.

Chapter 9 now has a more precise comparison question: which source-specific mechanisms preserve intent and support checking through change, under which dependencies and limits? Chapter 10 can treat specification policy as a scenario constraint. Chapter 11 keeps the full case trail separate from these analytical questions.

The durable lesson is a procedure, not a badge. Name what the specification governs, make its maintenance obligations explicit and inspect the evidence when a change challenges it. That remains useful whether the team keeps one compact issue or a larger network of behavioral artifacts.

Trace what happens when two teams own different records

Imagine that the service team owns the request policy and a separate client team owns the interaction. The accepted policy adds an intermediate pending outcome. The service team updates its behavior specification. The client team has a separate interaction specification that still assumes a successful approval takes a request out of the queue. Each specification has a legitimate owner. The conflict appears at the interface between them.

Write a small compatibility agreement for that interface. Name the response conditions the service promises, the meanings the client may infer, and when the changed behavior becomes active. Give the agreement one owner or define the joint decision it requires. A shared example can show one eligible approval on a high-risk request, the state the service reports, and the next action the client offers. The same example can be reviewed by both teams without exposing either team's internal design.

When someone proposes a change, inspect which agreement it crosses. A client wording change may leave the service obligation alone. Changing whether an intermediate approval returns the same success representation may need a joint decision. If one side releases independently, identify which combinations of old and new behavior can coexist and which require a coordinated rollout. The design can record a temporary compatibility window and the condition that ends it.

This exercise makes specification authority visible without introducing a new methodology category. The four native arrangements can use different records for these decisions. The inspection follows the same contract, its owners and its consumers. That gives a team a way to judge whether its existing practices are sufficient before creating another specification.

9. Comparing the Methodologies

Check what is being compared

The four deep dives describe different arrangements for making decisions, carrying context and checking work. Bringing them into one table does not make their scope equal. First identify the common business task and the selected source/profile for each arrangement.

Our documentary subjects are BMAD's inspected stable source, GSD Core's contemporary continuation, the inspected AWS workflow implementation/profiles and AIUP core with its Node-path qualifications. SDD supplies a lifecycle lens across them. Hosts, optional modules and application adapters remain dependencies of the intervention rather than additional peer methods.

The common task is the vendor-access increment and its later change/repair episodes. Requirements and intended starting conditions come from the book's case design. Native artifacts can differ. A comparison that forces AIUP use cases onto every arm or removes a required planning step to equalize file counts would change the methods being evaluated.

Record source and installed-package identities separately. A matching version string does not prove equivalent contents; the version-provenance review records source-versus-release differences. Until an actual environment is frozen and checked, the tables below compare inspected mechanisms only.

The same limit applies to operator and model conditions. If one arrangement receives a larger resource budget or additional manual architecture work, report it. Such an exercise can still describe a useful configured workflow, but it cannot isolate a method effect without further controls.

The instrument remains F01-F11 from chapter 3. The chapter groups related questions for readability while retaining every dimension. It asks which mechanisms are documented, which dependencies alter them and which outcomes remain unsupported. Chapter 10 owns scenario selection; this chapter supplies its evidence boundary.

Build a comparison row that can survive a profile change

Start a row with one business obligation and one question from the comparison instrument. For example, pair the v1 preservation rule with the question of how implementation workers receive a current decision. Give each row a source identity and a scenario, then specify what observation could change the row. A source revision can support a claim about a documented mechanism. A work record from a configured run would be needed to say how that mechanism behaved in the scenario.

For the proposed E2 exercise, freeze the same starting code, fixtures and v1 behavior for each arrangement. Record the native artifacts that each approach expects the operator to create or update. Different artifact names and sequences are acceptable. Compare whether the work received the preservation rule and whether the resulting implementation honored it. The comparison should still show where each method stored the rule and how the operator found it.

If a profile changes, create a new row with its own scope. An assessment of an optional review path cannot be transferred to a default profile simply because the repository contains both. If a later release replaces a verification step, the earlier row remains interpretable with its original pin. This keeps the chapter's documentary findings usable when readers evaluate an installed version of their own.

Compare decision rights and planning

F01 asks who may decide and how the decision constrains the next action. F02 asks how much preparation the selected route requires for the work. A framework's most elaborate route is not automatically the right comparison for a tiny repair.

Source scope	Documented decision/planning mechanism	Qualification
BMAD	Intent-sized planning and a light versus flagged-plan Build path	Approval is prompted where the selected design requires it; enforcement and cost were not tested

Source scope	Documented decision/planning mechanism	Qualification
GSD Core	Discussion decisions feed bounded plans and later verification	The phase loop can be disproportionate for tiny work; a quick path is described
AWS workflow	Profile-specific stages, checkpoints and guard policy	Configuration and overrides are part of the intervention; oversight is not one universal setting
AIUP core/Node path	Owned specifications, scoped decision rights and path-specific implementation rules	Layered defaults must not be generalized to every adapter; tiny-task overhead remains unmeasured

Primary support: BMAD routing, Core sizing, AWS profiles, AIUP scoped rights.

The table supports conditional questions, not effort rankings. BMAD and Core explicitly allow smaller routes, but we have not measured their use. AIUP's durable artifact structure does not prove that every task requires rewriting every artifact. AWS profile reduction changes coverage and must be evaluated against the scenario's actual duties.

For the two-approval change, the decisive product question is creation-time policy binding. The decision owner must settle it before an implementation silently chooses retroactivity. Each method provides a different place to record that answer. The evaluation should ask whether it remained available and constrained later work.

Human gates also differ in strength. An instruction to ask, a recorded approval and a mechanically enforced boundary require different evidence. The AWS hook's inspected disable branch is one concrete reminder that source configuration matters; it is not a comparative security audit. Selected static branch

Planning cost should include relevant setup, review and repair. A route that quickly generates code but needs extensive policy correction may compare differently from one that front-loads the decision. Without actual observations and a declared accounting policy, that remains a question rather than a result.

Compare the decision an operator has to make

Imagine giving each arrangement the same request to support dual approval. The first comparable moment is when the operator decides the work is ready to move into implementation. The reviewer can inspect what the proposed plan says about existing requests, who supplied that answer, and what records the implementer receives. A conditional approval that asks for a missing v1 fixture has a different meaning from an approval with no stated conditions. Preserve the condition and check whether it survived handoff.

The next comparable moment is a late policy question. Suppose an implementer notices that one actor can hold two approver roles. The accepted requirement counts distinct actors, so role count cannot satisfy it. A useful handoff lets that implementer find the rule and update the plan without requesting a new product decision. If the rule is absent, record the clarification, the operator's intervention and where it entered the remaining work.

Observation sought	Record to inspect	What remains unresolved
Decision was available at handoff	Accepted decision and actual worker inputs	Whether the implementation followed it
Approval constrained execution	Reproduction of a blocked unapproved action under the frozen configuration	Whether other execution paths enforce the same boundary
Planning effort was incurred	Defined record of operator actions and time categories	Whether the effort generalizes to another team or task

These are proposed observation rows. A documentary review can point to where an operator should look, while a trial has to retain what actually happened at each moment. Keeping the levels distinct also allows an investigator to report a useful mechanism whose enforcement remains untested.

Compare review authority under the same interruption

A useful future exercise stops each configured workflow after an owner has accepted a policy clarification and before all implementation tasks consume it. Give the next operator access to the same repository and its native records. Ask them to find the accepted decision, identify which tasks have consumed it and decide whether the next action is authorized. The interruption should preserve the same business state in all arms while allowing each arrangement to persist its own artifacts.

A comparison record can distinguish three observations. First, can the operator find the accepted rule? Second, can they identify the revision it governs? Third, does the arrangement expose which work remains blocked or stale? Finding the rule is weaker than showing it constrained the next action. An attempt that resumes correctly only after the prior operator supplies missing context should record that intervention.

The evaluator should decide how to handle operator familiarity. A person who wrote the first plan can sometimes infer missing context from memory. A second operator can expose handoff gaps but also introduces different experience. One proposed design gives each operator the same familiarization task and repeats the exercise across several arrangements in varied order. Report the preparation and order so the reader can inspect their possible influence. A small trial can still describe a failure mechanism without estimating a general effect.

Compare artifacts, context and change

F03-F05 examine artifact lifecycle, traceability and context. F07/F11 extend those questions into existing systems and maintenance. These dimensions converge when a policy changes: which record owns the new intent, which work receives it and what proves reconciliation?

Source scope	Persistent structure to inspect	Change/context question
BMAD	Named specification, linked stories/implementation records and current project context	Do skill-mediated updates preserve the right constraints while stale greenfield material is kept out of ordinary changes?

Source scope	Persistent structure to inspect	Change/context question
GSD Core	Phase decisions, research/plans, summaries, state and verification	Do bounded workers receive complete current decisions, and can the next session reconstruct unfinished work?
AWS workflow	Intent, workflow state, audit and boundary-verification records, with transient engine state distinguished	Which evidence survives a handoff, and which profile/route produced it?
AIUP	Requirements, use cases, domain/architecture context and downstream implementation/test links	Are applicable constraints linked and reconciled through the selected construction path?

Primary support: BMAD specification policy, Core record flow, AWS retained/transient records, AIUP inputs/change.

Durability has several meanings. A phase archive preserves process history. A behavioral specification is intended to guide future changes. A local engine cache supports execution. Treating all three as equivalent documentation would obscure the maintenance policy.

Traceability also needs three questions: are links present, do they cover the relevant obligations, and do the linked checks actually exercise those obligations? A plan, code marker and passing assertion can share CS-05 while testing an already terminal request. Their agreement would leave the self-approval rule unestablished.

For E2, inspect the same semantic probes across native artifacts: new high-risk v2 threshold, distinct approvers, preserved v1 semantics, unchanged tenant/self-approval constraints, and refreshed acceptance. An artifact count cannot answer those questions. Neither can a generated summary that merely declares completeness.

Brownfield guidance must distinguish observed from intended behavior. A reverse-engineered record can accurately describe a defect. An existing-code-first route can preserve an undesirable pattern. A specification-led route can impose stale behavior. The evaluation needs the accepted change contract and repository evidence together.

The sources give us different mechanisms for this work. They do not establish long-term maintenance benefit. That would require multiple changes, meaningful follow-up, contradictory artifacts and the effort needed to repair them. The current synthetic case has no such observations.

Evaluate recovery after a contradiction

Use a controlled discrepancy to see what the records make visible. In a future experiment, the operator could compare a plan saying “v1 requests keep their policy” with an implementation summary that implies the current setting governs all pending requests. Before changing either, determine whether the summary describes observed code, a proposed change or an unverified interpretation. An accurate summary of a defect should remain available as evidence while the intended rule guides the repair.

A reviewer can inspect four handoff properties. Find the accepted decision without relying on chat history. Identify the exact implementation revision the summary describes. Follow the rule to an acceptance example that exercises a pre-existing request. Identify who can resolve the contradiction. Record the steps needed in each native arrangement and any extra interpretation supplied by the operator.

The comparison remains sensitive to starting conditions. An arrangement beginning with a curated behavioral contract has a different context advantage from one beginning with an incomplete issue. Give all arms the same business decision and repository baseline. Allow the native arrangement to organize those inputs, then record any manual translation. The translation can be legitimate preparation and still matters when estimating adoption work.

Compare maintenance records by the question they let a reader answer

An artifact can be useful even when it is short. Ask a future maintainer to explain why the request keeps a stored version. A decision record with the accepted behavior and rationale can answer that question. A detailed task log can explain which files changed. A verification result can show what was observed. Give each record the job its evidence supports.

A future evaluation can supply the same maintenance question to each configured arrangement and retain the reader's path to an answer. Record

which native artifact they opened, whether it identified the accepted decision and whether they needed additional help. Then ask them to locate the implementation and observation affected by a proposed change. This second task exposes whether discoverability extends beyond a single well-written note.

Avoid rewarding artifacts merely for being numerous or penalizing them for being specialized. A workflow may keep a useful decision in one record and a useful check in another. The relevant question is whether their relationship is inspectable when it matters. If an operator has to manually assemble that relationship, retain the effort as part of the configured workflow.

This exercise also needs a stable task. A record for a large architectural change may naturally contain more context than one for a bounded bugfix. Hold the business question and starting state constant while allowing native representation. If a method requires a broader preparation step, record what it contributed and how much work it added before interpreting the result.

Compare verification, operations and portability

F06 and F08-F10 address people, acceptance, operations and dependencies. These are especially easy to overstate when a workflow includes named review roles or a delivery stage.

Source scope	Verification mechanism or boundary	Operational/dependency limit
BMAD	Built-in generated QA differs from optional TEA and has explicit coverage limits	Host capabilities affect reviewer dispatch; comprehensive operations were not established in this inspection
GSD Core	Summary-derived UAT, gap closure and stale-verdict handling; code review is separate/optional	Shipping a PR does not establish production operation; runtime prerequisites conflict in the inspected source
AWS workflow	Profile-specific checks, reviewer behavior and gates	Operations can be reduced/placeholder; team and guard configuration change the intervention

Source scope	Verification mechanism or boundary	Operational/dependency limit
AIUP core/Node	Deterministic spec lint, semantic advice, tests and human acceptance records	Node stack assumptions and the equivalent-auditor limitation must remain explicit

Primary support: BMAD QA limits, Core verification/shipping, AWS profiles, AIUP auditor boundary.

No cell is a test result. The book has not installed the four arrangements, run a common oracle or observed a human team. Where a mechanism is documented, its actual use and efficacy remain separate questions.

A local exercise can establish selected behavior under synthetic conditions when executed. It cannot establish a team's coordination benefit, enterprise suitability or production resilience. A process may integrate with existing organizational practices; those practices and their effort belong in the evaluated arrangement.

Portability has the same evidential boundary. A host adapter or stack plugin demonstrates an intended integration path. It does not prove equivalent hooks, permissions, reviewer dispatch or generated application behavior on another environment. A migration/smoke exercise needs its own inputs and records.

The TypeScript/Node case exposes this issue directly. The AIUP adapter's framework/database assumptions and Core's runtime discrepancy affect setup. Choosing a common stack could advantage or constrain particular integrations. The design must disclose that choice and count adaptation rather than silently make portability free.

Read a verification record at the right boundary

Suppose a native status reports completion for E2. Ask for the observations underneath it. A status backed by a reviewed plan, a compilation log and two policy-function checks may be accurate according to the workflow's own rules. Acceptance of the common business increment still needs a pre-existing v1 request and a high-risk v2 request exercised through the relevant service boundary. Count the status once as workflow evidence, and use the underlying observations only for the claims they support.

The same separation applies to approval enforcement. A transcript showing a prompt and a human answer documents one interaction. A controlled attempt to proceed without the answer can inspect enforcement on the selected path. Record the actual environment and settings before interpreting either. A capability supplied by the host, such as an approval dialog or process isolation, belongs in that environment record rather than being attributed automatically to the methodology.

For an existing repository, include recovery from a stopped or expired session in the evaluation. Ask a different operator to identify unfinished work from the retained records, with the same authorized repository access. Record which decision or check needed help, and whether native records were current. This proposed exercise addresses handoff. A delivery claim also needs observations about the actual release boundary and about how people respond when that boundary fails.

Record adaptation where it happens

Suppose a native workflow assumes a certain way to dispatch independent work, and the selected host exposes a different set of capabilities. An operator can adopt a manual handoff that preserves the intended division of responsibility. The resulting trial should identify that change. An evaluator who calls the setup stock would attribute part of the outcome to the wrong component.

Describe the adaptation at the affected step. Include the expected behavior, the constraint that required a change, the replacement action and the artifact it produced. If the operator manually copies an accepted decision into a work packet, record whether they merely transfer the decision or also interpret and expand it. The latter can change which business context the implementer receives. It may be appropriate in practice and still matters for comparison.

A similar issue arises when an organization adds its ordinary deployment process to each arrangement. Record which controls are shared and which controls come from the selected configuration. A common independent acceptance check can be a deliberate constant. Different amounts of manual setup or repair belong in the account of each attempt. Keep a clean distinction between evidence that an arrangement can be made to work and evidence about how much work that took.

A reviewer can use a short adaptation log during analysis. If one result depends on an unusual recovery procedure, describe the procedure and its scope before comparing it with other outcomes. If the procedure will be available to all future users, include it in the reproducibility packet. If only one operator knows how to perform it, the result may still be informative while remaining dependent on that operator.

Read gaps, conflicts and observations

Missing evidence does not have one meaning. We have documentary support for some mechanisms, an explicit source conflict for Core installation requirements and no execution outcomes. Preserve those states rather than turn them into a numerical scale.

The comparative preprint reviewed for this book provides useful prior documentary work, but its dimensional scoring is single-rater judgment with disclosed limitations. Such scores are not framework-effectiveness measurements. We do not import them or claim to be the first comparison. Prior taxonomy/comparison paper, method and limitations

Source confidence is also claim-specific. Confidence that a pinned file contains an instruction does not transfer to confidence that the instruction succeeds. A maintainer's exact terminology may be well established while an outcome claim remains interested-party evidence. A field account can expose a particular failure without establishing its frequency.

Our current outcome disposition is deliberately simple:

Outcome	Current book evidence
Implementation acceptance under the common oracle	Not run
Human effort or elapsed-time advantage	Not assessed
Defect detection or repair advantage	Not assessed
Human-team coordination	Not observed
Production operation/recovery	Not exercised
Long-term artifact/code alignment	Not assessed

Those entries describe the project, not an absence of all evidence everywhere. The literature search was directed and bounded. A new well-described study could change the assessment, provided its task, versions and conditions match the claim being considered.

The separate local policy-model checks described in chapter 11 do not fill this table. They exercise an author-written sequential function, not four installed processes against the common application oracle. Their passing result is useful executable teaching evidence with a much narrower scope.

If later trials occur, retain failures and deviations. Installation failure leaves business behavior unassessed; a failed acceptance assertion addresses behavior under its fixture. A run completed after large human redesign must include that intervention. Missing attempts should not disappear from a favorable summary.

When sources conflict, keep the conflict visible and define what would resolve it. The Core guide/manifest discrepancy calls for installed-artifact and compatibility checks, not averaging runtime requirements. A profile-specific AWS statement calls for configuration identification, not a global verdict about all AI-DLC work.

Keep denominators visible when work stops early

Consider four hypothetical attempts in a future evaluation. One stops at installation, one builds and fails an acceptance check, one passes after manual redesign, and one is interrupted by an unrelated outage. Give each attempt a row and record the stopping condition. The installation result informs compatibility under the selected setup. The acceptance failure informs the behavior of the built revision. The manually repaired pass is a result for the configured workflow with the repair included. The outage leaves the interrupted outcome unresolved.

An effort comparison across only the finished attempts would lose both the setup burden and the interrupted work. Report the population for every statistic or qualitative claim. If setup was shared, say how it was accounted for. If a single operator gained experience between attempts, keep the ordering and familiarization history visible. Reusing an accepted baseline for E2 can reduce carryover from E1, while operator learning still needs attention.

The book's present documentary exercise has no such attempt rows. Readers can use the distinctions to review other reports. Ask what was eligible to enter the study, how incomplete attempts were treated, and whether a result depends on help that another team would need to reproduce. An apparent performance advantage can become a narrower statement about a particular team, version and starting point once those conditions are recorded.

Give uncertainty a consequence in the report

An unresolved source conflict should change what the reader can act on. If a runtime requirement differs between a guide and an installed manifest, the report can describe the conflict and withhold an installation recipe until a smoke exercise resolves it. A claim about a configured approval guard can be supported for a named path while the report leaves other hosts open. Assigning both situations a generic low-confidence badge hides the next action.

For a documentary comparison, pair each important gap with an evidence request. A missing package identity needs the installed artifact and its provenance. A missing enforcement observation needs a controlled attempt with and without the prerequisite on the same path. A missing maintenance outcome needs another change and a way to inspect whether decisions remained discoverable after it. None of these requests needs a universal numerical confidence scale.

Then ask which decisions can proceed. A team can shortlist a method because its documented records are relevant to a handoff problem, while keeping installation and continued use conditional on its own pilot. A book can describe the documented mechanism and refrain from publishing comparative outcome claims. This makes uncertainty part of the decision record.

Compare the evidence before comparing the apparent result

Imagine two proposed reports on the same change. The first names the selected configuration, the revision, and the accepted criterion, and says the acceptance check could not be run. The second says the change passed, but provides only an aggregated green status. The first report supports a clear statement about where the attempt stopped. The second requires more information before a reviewer can decide what passed.

A reusable comparison record separates a method claim, a local observation and an interpretation. For example, the method claim describes a documented place for persistent decisions. The local observation identifies whether a second operator found the governing rule after interruption. The interpretation explains whether any missing context affected the next action. Each entry has its own evidence and can be revised when new material appears.

Ask how the reported finding could be wrong. An operator may already know the expected policy and find it faster than a new contributor. A seeded record may sidestep the path that introduced the defect. A report may capture an approved plan while omitting a later manual change. The comparison can retain these alternatives and propose follow-up observations without forcing every outcome into a shared numerical ranking.

Tradeoffs the evidence supports

The strongest current finding is that the selected arrangements expose different structures for the same engineering duties. BMAD makes planning-path choice and a named specification/Build trail explicit. Core emphasizes persistent phase decisions and bounded context/workers. AWS makes lifecycle routing and configuration a large part of the intervention. AIUP emphasizes maintained behavioral artifacts and scoped construction/review paths.

These are documentary differences, not benefit scores. They suggest questions about adoption and maintenance cost. A small route can be appropriate for a clear repair; a durable specification can help inspect a policy change; a configured lifecycle can expose wider duties. Which arrangement fits depends on the actual constraints and on evidence that its mechanisms work there.

All four need an external interpretation of accepted behavior. Generating a plan, test suite or review report does not establish its correctness. The common CS contract and eventual independent checks exist to keep the comparison's target stable while native work differs.

The SDD lens adds policy questions without a fifth arm. Which specifications persist, what do humans edit, and how is disagreement resolved? Answers

can differ by artifact within one framework. Strictness of edit policy is not an effectiveness ranking.

Form a conclusion someone can challenge

A defensible documentary conclusion names a mechanism, its scope and what would overturn the interpretation. For example, a team can treat a maintained behavioral artifact as a candidate source of context for future changes. To evaluate it, give a later contributor a versioned policy change and ask them to find the governing rule, update its dependents and expose stale evidence. If they need an undocumented decision from the original operator, the result identifies a handoff gap under the tested conditions.

Use the same standard for light routes. A smaller procedure looks appropriate for a well-understood repair when it preserves the relevant decision and produces adequate verification. A later evaluation can compare the amount of preparation with the completeness of the repair record. If the work depends on an unresolved tenant contract, the smallest route might reasonably pause at clarification. That pause belongs in the report and can be the correct outcome for the scenario.

The selection record in chapter 10 can carry the resulting conditions forward. State the local need, choose an arrangement whose documented mechanism addresses it, and name the observation that would justify keeping or revisiting the choice. For an organization, the useful decision is often about one workflow and one environment. That level allows the team to learn from the evaluation and allows another reader to decide which parts apply to their own setting.

Read a mixed outcome without compressing it into a winner

Consider a hypothetical exercise in which one configured arrangement makes the policy decision easy to find but needs manual work to refresh evidence. Another makes the review report easy to regenerate but requires a conversation to recover which version rule applies. These are distinct observations about handoff and verification. Reporting them side by side preserves the information a team needs to choose.

A maintenance team with frequent interruptions may prioritize recovery of accepted intent and accept some manual verification work. A team with a stable owner but frequent releases may prioritize evidence freshness and add

an explicit decision record to the second arrangement. Either team needs to check its adaptation on a subsequent change. A favorable observation about the original arrangement cannot by itself establish the effects of the adaptation.

The report can carry forward conditions for transfer. Identify the repository shape, the selected scope, the available integrations and the duties supplied by the team. If a reader wants to apply the result to a regulated release or an architecture with different data ownership, those differences become part of a new evaluation. The documentary comparison can suggest which native mechanism to inspect without guessing how it will behave under those conditions.

Chapter 10 turns this evidence into scenario-based choices and a reversible evaluation record. Chapter 11 presents the full qualitative case and its missing execution controls. A later measured study would add observations to these mechanisms; it must not be invented to make the table look complete.

Carry a qualitative finding into a local decision

A comparison can finish with several findings whose conditions differ. One arrangement may make an accepted decision easy to locate. Another may supply a more useful verification handoff. A third may require setup work that the team's environment cannot currently support. Write each finding with its task, configuration, observation and unresolved alternatives.

A local decision can then select one finding to investigate. For an interrupted maintenance workflow, the team might prioritize discovery of accepted intent. It can shortlist arrangements with relevant persistent records and plan a recovery exercise. For a release workflow, the team might prioritize the relationship between candidate identity and evidence. It can inspect that handoff using a stale observation and a revised candidate.

The comparison supplies reasons to choose an exercise and a candidate. The local exercise supplies observations for a decision under the team's conditions. Keeping that sequence visible gives another reader a way to challenge a premise or repeat a bounded check.

For a report, use one row per conclusion and include an evidence request beside any unresolved claim. If the configuration changes, revisit the affected

row. This preserves useful distinctions without making the entire comparison depend on a single aggregate verdict.

10. Choosing the Right Methodology

Describe the project before naming a method

Begin selection with a decision the team needs to make well. Describe the work before creating a shortlist.

Requirements may disappear between discussion and implementation. A maintenance queue may be burdened by documents that no one revisits. An established service may need traceable policy changes. Each hypothetical condition calls for a different evaluation question.

For the vendor-access service, our immediate question is narrow: can a selected process carry a tenant-local approval rule from agreed intent through implementation and acceptance, then preserve it during a policy change? The question does not require us to decide which framework is best for every service the organization might build.

Write down the conditions that could change the answer. Is this a new codebase or an existing one? Who can settle product ambiguity? How long must the service be maintained? What may the assistant change? Which host and runtime can the team actually use? Which obligations would make an otherwise plausible output unacceptable?

A useful project description is specific enough to disqualify an attractive candidate. “We use TypeScript” is too broad to establish compatibility. “We must retain this existing persistence and HTTP boundary, and cannot introduce another application stack during the exercise” is a real constraint. A framework with a stack-specific adapter would then need a separately assessed route.

State these conditions before reading a candidate's positioning. Otherwise it is easy to rewrite the problem in the vocabulary of the first polished workflow we encounter. The selection record should survive a change in popularity or branding because its reasons concern the team's work.

Identify non-negotiable decision duties

In the common case, no self-approval and tenant isolation are required behavior. A named ceremony is not. We need a way to establish the rule, implement it, check it and preserve the decision when the code changes. There may be several legitimate ways to arrange that work.

Separate these duties from preferences. The team may prefer Markdown artifacts, a particular command style or a small number of files. Those preferences can affect adoption cost. They cannot replace the obligation to show that a foreign-tenant actor cannot obtain another customer's request under the agreed interface policy.

The duties also need owners. A model can propose a policy but cannot supply missing organizational authority. A workflow can request approval, but the selected configuration and operator behavior determine what approval means in practice. An acceptance owner needs to know which decisions were actually reviewed, by whom, and against which requirement.

For a small team, ownership may overlap. Record the overlap and its limits. If the implementing engineer also prepares the only tests, call them implementation tests. Additional AI review may be helpful, but calling it independent human verification would overstate the arrangement.

Once these duties are explicit, ceremony becomes something to justify. A document earns its place when it carries a decision, supports a check or helps a later change. If it has no such role in the proposed evaluation, ask whether it belongs in that path. Equally, removing a document requires deciding where its useful information will live.

Distinguish permission to work from permission to accept

For the common service, the product owner settles whether an old request retains its original policy. An engineer chooses how the accepted rule is represented within the agreed application boundary. An acceptance reviewer checks evidence against the rule. A release owner decides whether remaining operational limitations permit a release. One person may hold several roles, but the decisions remain different.

Write that assignment before configuring automated approvals. Permission for an assistant to edit code does not authorize it to change a product rule.

Approval of a proposed plan does not establish that the implementation satisfies it. A review label should identify the decision actually made, not borrow authority from a nearby checkpoint.

Also define what happens when the responsible person is unavailable. A trial may pause, proceed only with unaffected work, or record a provisional interpretation excluded from acceptance. The operator should not infer consent from an unanswered question. For a local evaluation, the sensible action may be to stop the affected episode and retain its state.

This distinction helps diagnose an apparently slow process. Waiting for a required policy decision is different from generating an unnecessary planning artifact. Removing the checkpoint may shorten the visible workflow while transferring an unresolved decision into code. The evaluation needs to identify which duty caused the wait before recommending a smaller process.

Filter candidates with evidence

Chapter 9's comparison supplies mechanisms and gaps, not a league table. Selection should use those findings as constraints and questions. A supported documentary capability may justify investigating a profile. A demonstrated incompatibility may rule out that profile under current constraints. An unknown should remain an unknown until the team obtains the evidence it needs.

For example, the inspected GSD Core installation guide says Node 18 or later, while the same-pin package manifest declares Node 24 or later and npm 10 or later. That discrepancy does not establish that installation will fail. It does establish that the prose requirement alone is insufficient for an installation decision. A candidate environment must be tested and the actual package identity recorded. Installation guide, package engines

The inspected AWS workflow implementation makes profile choice consequential. Its Classic and Express descriptions differ in stages, artifacts, reviewer behavior and guard settings; some human-presence or approval duties remain even where particular guards stand aside. “Uses AI-DLC” therefore leaves too much unspecified for selection or reproduction. Pinned workflow profiles, lines 45-95

AIUP supplies another kind of boundary. The inspected combined Node adapter targets NestJS, Drizzle, PostgreSQL and a Next.js frontend. Its

workflow also says this adapter does not yet supply an equivalent of the coverage auditor described for certain other adapters. That is an adapter-specific qualification, not evidence that specifications cannot be reviewed in a Node project. Node adapter scope, coverage-auditor boundary

These examples show why a weighted total would be premature. A high score for documentation cannot cancel an incompatible mandatory stack. A low-confidence portability judgment should not become a numerical penalty for a capability we did not test. Record the constraint, the relevant evidence and what would resolve the uncertainty.

A useful candidate disposition has four possible states: investigate, conditionally retain, exclude for a stated constraint, or insufficient evidence. An unknown can matter differently for different duties. Unmeasured long-term benefit may be acceptable during exploration; an unknown mandatory runtime may block an executable pilot. Name the profile, the affected decision and the evidence needed to proceed.

Reject a candidate for a stated local reason

A team can remove an arrangement from its immediate shortlist when an essential dependency is unavailable in the authorized environment. Record the dependency, the attempted setup or documentary conflict and the duty it prevents the team from fulfilling. Keep the reason tied to that environment.

If the dependency later becomes available, the decision record gives the team a precise reason to reopen the candidate. The earlier exclusion remains an account of the conditions at the time. This makes shortlist decisions easier to maintain as hosts, integrations and team responsibilities change.

Work through contrasting scenarios

The following scenarios are editorial decision exercises. Their project conditions are invented and explicit; no successful adoption or performance result is implied. They show how a team can make a provisional choice from documented mechanisms without pretending to know its later outcomes.

A tiny, well-understood repair

Assume one developer understands a small presentation defect, the relevant code and accepted behavior are available, and the change does not alter policy, routes, authorization or persistent data. There is no demonstrated handoff problem to solve and the existing review record is current.

Our provisional decision is ordinary bounded change work with the compact record from chapter 2. We would not introduce a named full-process adoption for this task. The reason is the absence of a new coordination or specification-maintenance duty, not a general belief that small changes are harmless.

BMAD's inspected guidance permits a trivial direct edit and describes a bounded Build route for other small changes. This is evidence against forcing its full planning inventory onto every repair. It does not tell us how much effort our repair will take. Small-change guidance, lines 15-23

Retain the constrained diff and meaningful navigation/regression evidence. If inspection reveals a shared permission change or a stale requirement, reopen the sizing decision. That observation would invalidate the conditions under which the small path was chosen. The exit criterion is therefore concrete: stop treating it as a presentation-only repair when the work changes a protected behavior.

Defined intent with reviewable story checkpoints

Assume a small team has agreed the vendor-approval rules, wants a compact named specification, and needs a human checkpoint before implementing the first foundational slice. Team members are already familiar with the inspected BMAD path. Application constraints and an external acceptance inventory are available, while no project duty requires a permanent use-case catalog.

We would choose BMAD's specification/story/Build arrangement for a bounded evaluation, with optional modules stated explicitly. The documented story breakdown lets the operator choose checkpoints, and specification updates have a designated route. Those mechanisms match the declared work arrangement. This is not evidence that Core or AIUP could not address the same task. Specification and checkpoint guidance

The selection record would name the capability/constraint trail to inspect, a first-story review decision and the external checks that must remain independent. Built-in QA would not be described as comprehensive acceptance. Enabling a larger testing module would change the evaluated configuration and its preparation cost.

Revisit the choice if the first checkpoint exposes undefined product intent, if maintaining the record creates unresolved contradictions, or if the actual host cannot support the review arrangement. The team should then fix the premise or compare another route, rather than keep generating stories to preserve the original selection.

Multi-session work with phase continuity

Assume the main difficulty is recovering decisions and unfinished work across sessions. The team wants bounded plans, durable summaries and a verification record whose freshness can be checked. It has no fixed requirement to make a use-case catalog the enduring product record. For this exercise, assume the chosen Core package/runtime conflict has been resolved through actual environment checks; that premise is not a result from our book case.

We would choose the inspected Core phase loop as the first bounded trial. Its context/plan/summary arrangement and documented staleness policy address the declared continuity problem. The trial question is whether an interrupted episode can be resumed with the right current decisions, including the policy exception, and with stale verdicts rejected appropriately. Phase loop, verification freshness

The operator would retain the actual handoff inputs and the interruption/recovery point. Code review would be an explicit selected duty rather than assumed part of shipping. A summary-derived UAT list would coexist with independent CS checks, so omitted deliverables cannot define the whole oracle.

Revisit the choice if the recovered record requires substantial undocumented interpretation, if workers miss material constraints or if the required host behavior is unavailable. Those observations concern the scenario's reason for adopting the loop. A successful code-generation step alone would not answer the continuity question.

Lifecycle coordination with explicit stage authority

Assume an organization needs one visible route from intent through construction into operational preparation, with different people owning consequential decisions. It can resource those handoffs and wants to inspect stage/unit authority explicitly. A local coding-only route would leave important duties outside the chosen arrangement.

We would start a documentary/configuration evaluation with the AWS workflow's Feature candidate, which the inspected chooser describes as a complete-lifecycle route. We would record active guard/reviewer settings and actual ownership rather than infer them from the profile name. The chooser's static stage counts do not establish elapsed effort or operational readiness. Pinned profile chooser

The first evaluation deliverable is a route/authority map: which person reviews which decision, which evidence reaches them and what constrains the next action. Operational exclusions or placeholders would be retained as open work. A route labeled Enterprise would not, by its label, establish compliance or better acceptance for this scenario.

Revisit the choice if the named owners cannot perform their duties, if local substitutions remove a mandatory operational check, or if the configured enforcement differs from what the decision record assumed. A solo simulation cannot establish that this organization will coordinate effectively.

A maintained behavioral catalog in a compatible existing stack

Assume the organization already maintains requirements and use cases as a product record. Its application boundary is NestJS/Next.js with PostgreSQL, and it accepts the inspected Node adapter's construction assumptions. The team has explicit specification owners and resources for a manual coverage review where equivalent automation is absent. These are scenario premises, not technology selections for our common SaaS.

We would choose AIUP core plus the matching Node path as the first bounded evaluation. The documented artifact/change flow fits the desired maintenance duty, while the existing stack removes one obvious adapter mismatch. It does not prove compatibility of actual package versions or establish that the record will stay aligned. Adapter scope

The evaluation would follow one policy change from accepted requirements through affected use cases, implementation and checks. It would measure the real work of semantic review and reconciliation rather than count markers. The Node coverage-auditor limitation stays in the record; an explicitly resourced manual route is part of the arrangement.

Revisit the choice if maintaining the catalog creates unresolved duplicate rules, if adapter constraints force unaccepted architecture changes, or if coverage cannot be assessed under the available review capacity. The same principle would permit another framework to be investigated if it met the constraints more convincingly.

Uncertain intent is an earlier decision

These selections assume enough agreed intent to define the evaluation question. If the team has not decided who may approve or how old requests behave, first produce a bounded decision record that exposes those choices. A detailed implementation plan cannot supply missing organizational authority. Progress in that stage is an agreed rule and visible uncertainty, not a completed application.

The examples deliberately make different decisions from different premises: use a compact ordinary path, trial a named specification/story route, trial phase continuity, inspect lifecycle authority or trial a maintained use-case flow. They are not a hidden ranking. Change a mandatory premise and the same procedure may produce a different next commitment.

Adapt practices without losing ownership

A team may want the planning path from one framework and a specification practice discussed elsewhere. That is a possible process design, but it creates new ownership questions. Which artifact is authoritative? Who may change it? What happens when the story and specification disagree? Which check establishes that downstream work has caught up?

BMAD's selected specification guidance designates `bmad-spec` as the writer of `SPEC.md` and directs updates through that skill. Adding a second workflow that edits the file directly would conflict with the documented maintenance rule. An adaptation must resolve that conflict rather than describe both policies as simultaneously in force. Specification ownership, lines 112-116

Begin an adaptation by naming the problem it solves. Then assign one authoritative location for each decision and define the handoff. Preserve the native artifacts needed to understand the original process, or document exactly what has been replaced. If the adaptation changes mandatory behavior, describe the evaluated process as an adaptation; results from it should not automatically become claims about the unmodified framework.

Spec-Driven Development belongs in this discussion as an approach to organizing intent and change. Giving it a chapter does not require a fifth experimental arm. The relevant selection question is which specification policy the team needs and how that policy interacts with the chosen framework's lifecycle.

A worked adaptation boundary

Suppose the team retains BMAD's named specification route and adds an external change-request queue used by its existing maintenance process. The queue entry describes the proposed v2 rule and affected customer behavior. It is an input to the specification owner; it does not become a second authoritative specification simply because another assistant created it.

The proposed handoff records the queue entry, the accepted policy decision, the specification revision and the downstream implementation/check references. A rejected request remains visible without changing accepted intent. If implementation reveals an ambiguity, the question returns through the same ownership route rather than being settled by an unreviewed edit to the specification.

The adaptation record must say which native duties remain and which have changed. It should identify any new manual work and any automated behavior the arrangement no longer uses. Merely retaining a familiar command or filename is insufficient to claim that the unmodified framework was evaluated.

Before adoption, review a change in each direction: an accepted policy change moving toward implementation, and an implementation discovery returning to the policy owner. The aim is to inspect the handoff, not to generate two independent documents that happen to agree initially. Their first disagreement is exactly the event the ownership rule should handle.

This is an original process-design example, not a tested integration. The narrower adoption decision is whether the proposed ownership boundary is understandable and feasible for the team. Later execution would need to establish whether people and tools actually follow it.

Define an interface between a new workflow and the team's existing controls

A team may want to adopt the planning part of a workflow while keeping its current release review. Start by specifying what the existing reviewer needs to receive. For a stateful policy change, that might be the accepted behavior, a migration decision, the candidate revision and evidence about older requests. An integration can translate native records into that packet as long as each item retains its origin and scope.

Suppose the planning record marks all stories complete while the release review awaits a rehearsal. The integration should carry both statuses and attach them to their decisions. If it maps them to a single “complete,” the release owner loses information. If the process requires the reviewer to reconstruct every native artifact, the integration has transferred the manual work to the wrong person. A short record that points to the actual decision and observation often suffices.

Try this interface on one bounded change and inspect what happens when an observation is stale. The team should be able to replace that observation and retain the historical record. If each refresh requires copying entire reports, narrow the interface. If essential policy details are lost in the translation, add the missing reference before expanding adoption. Record the adapted arrangement as the object of evaluation.

Pilot, measure learning and define an exit

A reversible pilot should answer one bounded question. For the common case, one candidate question is: can the selected path carry CS-04 and CS-05 into a reviewable decision implementation while retaining CS-03's tenant boundary? The full case contains additional obligations; this question defines a smaller learning scope, not permission to omit them from final acceptance.

Before a run, record the exact profile, baseline, host/model settings, permitted interventions, independent checks and resource ceiling. If a field is

unresolved, the trial is not reproducible yet. Setup and adaptation effort belong in the record alongside active implementation effort. Human waiting, agent work and elapsed duration are different observations; combining them into an unexplained speed number hides the allocation of work.

Define exit conditions in advance. Stop the trial if the environment cannot be reproduced, required checks cannot be executed, an unresolved product decision would change the task, or the agreed resource ceiling is reached. Preserve the partial output and the reason. A stopped run can reveal an adoption obstacle; it should not quietly disappear from the comparison.

Success criteria should concern the question. The trial might establish that required artifacts can be linked and the independent checks executed under the selected configuration. It would not, by itself, establish a long-term maintenance benefit or predict another team's productivity. The team should name the additional observation that would justify expanding the evaluation.

This book has not executed that pilot. The companion case walkthrough describes the proposed evidence boundary. The readiness checklist and later run records must supply the execution evidence.

Budget the investigation before estimating a return

A useful pilot budget separates the effort of making a candidate evaluable from the effort of completing the business episode. Installing a host, resolving package requirements, learning the native path and preparing an adapter can all consume real capacity. Whether they are inside or outside the episode's timed interval must be stated before work begins; neither choice makes them disappear from adoption cost.

Use a blank effort ledger with activity, responsible role, start/stop rule, evidence locator and accounting category. Categories can include preparation, native planning, implementation, review, policy clarification, waiting and recovery. Overlapping agent and human activity needs a declared counting rule. Adding their durations as if all work were sequential would answer a different question from wall-clock completion time.

Do not convert this ledger into a return estimate without a meaningful comparator. A pilot can establish the effort observed for its own arrangement. It cannot show time saved against an unobserved alternative. Likewise, an

operator's familiarity is a condition to record, not a reason to assume that every future team will incur the same learning cost.

Include stopped attempts in the investigation account. A blocked installation may leave application correctness unassessed while still revealing an adoption obstacle. An unavailable reviewer may make a particular process arrangement infeasible under the team's staffing conditions without proving a defect in its written method. Keep the observations and their interpretation separate.

The next commitment can be deliberately small: retain one useful record, investigate a different profile, repeat with an independently reviewed contract, or return to the existing process. Before leaving a trial, identify any accepted decisions that must be transferred, preserve the native evidence and remove reliance on temporary automation. Reversibility is a procedure for recovering ordinary work, not simply a promise that the team can stop.

A longer adoption should need an additional learning question. For example, does the same ownership arrangement remain understandable during a later policy change performed by someone else? That question requires new observations. It cannot be answered by extending the interpretation of a single successful feature episode.

Specify what would cause you to end the pilot

A team choosing a workflow for multi-session maintenance can set a stop condition around handoff. After the first bounded change, ask a second operator to find the accepted policy, distinguish current from stale verification, and complete the next authorized step. If they need an undocumented assumption from the first operator, record where the handoff failed. The team can try a bounded correction and repeat the same exercise on a fresh change.

Set a separate exit condition for setup. If the chosen host cannot provide the necessary review or permission boundary under the agreed configuration, the pilot can end with a compatibility finding. Record work completed before the stop and preserve any useful artifact. A later configuration change can justify a new attempt with its own baseline.

Write and revisit the decision

The selection deliverable is a short decision record that another engineer can challenge. Use the following original template:

Field	Required entry
Decision question	One bounded problem the evaluation is intended to resolve
Context and constraints	Codebase, ownership, lifespan, application/host constraints and required acceptance duties
Candidates	Exact source/package/profile/module scope; tool context recorded separately
Evidence	Source passages or run records, each with its limited interpretation
Disposition	Investigate, retain conditionally, exclude for a specific constraint, or insufficient evidence
Costs and uncertainties	Known adaptation work and unmeasured effort; unresolved compatibility or review limits
Pilot contract	Baseline, checks, resource ceiling, permitted interventions and stop conditions
Owner and revisit trigger	Who decides; which observation or changed constraint reopens the choice

For our book, the present disposition is to retain four framework subjects for documentary comparison and possible later controlled evaluation, with SDD treated across them. Contemporary Core is the explicit working GSD prose scope. BMAD's pinned source supplied the earlier method pilot prose; that writing choice does not appoint it as the production process for the SaaS or freeze installed packages, host or stack.

Here is a populated example for the policy-change episode. It is an editorial decision exercise, not an approved run contract:

Field	Example entry
Decision question	Which profile should enter a bounded evaluation of the high-risk approval-policy change?

Field	Example entry
Fixed intent	New high-risk v2 requests need two distinct eligible approvers; existing v1 requests keep v1 semantics; tenant isolation and no self-approval remain required
Starting condition	The same externally prepared baseline for every arm; TypeScript / Node.js family only, with exact stack and host still open
BMAD candidate	Investigate the inspected specification/Build path; check whether the version-binding rule reaches the implementation record and acceptance evidence
GSD candidate	Contemporary Core for the prose; confirm the installed edition/package and resolve the guide/manifest runtime discrepancy before an install-based trial
AWS AI-DLC candidate	Retain for investigation; name the implementation, profile and guard configuration rather than treat the lifecycle label as the complete intervention
AIUP candidate	Retain for investigation; verify whether the selected adapter fits the common stack and assign an explicit coverage-review route
Current decision	Prepare comparable profiles and checks; do not name a winner or start measured runs while the mandatory inputs remain unresolved
Revisit trigger	A reviewed environment and acceptance contract, or evidence that a candidate cannot satisfy a fixed constraint without changing the experiment

The table turns uncertainty into work. GSD needs a compatibility answer, AWS needs a configuration identity, AIUP needs an adapter/review route, and BMAD needs a checkable artifact path. These are different questions, not equal numerical penalties. A team with already fixed constraints could reach a different shortlist; this example preserves the choices the book has actually made.

A later decision can change without invalidating this record. A resolved compatibility conflict, a different application constraint or an actual pilot observation may alter the choice. Record the new evidence and preserve the earlier reasoning. The purpose of selection is to make a defensible next commitment with a clear way to learn from it.

Leave a decision that can be revisited without reopening the whole selection

A completed selection record can name the local need, chosen arrangement, configuration and reason for accepting its dependencies. It should also name the next observation. For example, after choosing a persistent-context workflow for interrupted maintenance, observe whether the next operator can recover the accepted rule and the current verification boundary.

Give reevaluation a trigger. A failed handoff, changed host capability or new release responsibility can reopen the relevant part of the decision. The original choice remains understandable with its earlier conditions. The team can then revise the configuration or choose another arrangement with a clear account of what changed.

11. Real-World Case Study: Building a SaaS

Scope, realism and approved decisions

A useful case needs more than an appealing feature. It should expose decisions that different processes have to carry, boundaries that a positive-path demonstration can miss, and changes that test whether the original intent remains recoverable. Vendor-access approval supplies those conditions without requiring a live business deployment.

The service records a request for temporary vendor access, an eligible decision and its history. It does not grant access in an external system. There are two synthetic customer tenants, overlapping roles and a later policy change. The author selected TypeScript / Node.js and accepted this domain/design direction. No web framework, database, model or host has been selected as an executed common stack.

The complete design authority is the case specification. It distinguishes accepted direction from unresolved interfaces and future acceptance checks. This chapter explains how to walk that design through the selected source mechanisms. It does not turn a paper design into an implementation result.

The business model needs Tenant, User, Membership/Role, VendorAccessRequest, ApprovalDecision, PolicyVersion and AuditEvent concepts. These names describe responsibilities, not an imposed ORM schema or class hierarchy. Each method remains free to organize native planning artifacts and a future implementation within the agreed constraints.

A request begins pending after valid creation. An eligible different tenant member can approve or reject according to its bound policy. A rejection while pending is terminal. Approval becomes terminal when the policy's required distinct approvals are present. Public business operations cannot rewrite accepted decision history. Those are expectations to realize and check later.

Realistic constraints include tenant isolation, current membership, no self-approval, retries, concurrency and durability. Real identity federation, live provisioning, billing, email, production hosting and regulatory

certification are excluded. The narrow scope lets us reason about meaningful behavior while keeping the evidence claim local and explicit.

The missing application/framework execution layer is material. We cannot report SaaS build success, defect counts, elapsed times or framework effectiveness. A transparent qualitative walkthrough can still show which questions and artifact relationships a reader should inspect. It also identifies the concrete work required before a measured comparison would be credible.

Requirements, fixtures and acceptance oracle

Mira requests access in tenant A. Leon and Priya are eligible approvers in A. Ozan belongs to B. A role-overlap variant gives Mira approver and administrator roles. A revoked member tests current eligibility rather than a stale role assumption. These are proposed fixture roles; no database has been seeded.

Use a controllable clock and synthetic identifiers. A valid creation expiry is after the controlled creation time. Keep unrelated checks within a valid time range so an expiry failure does not hide the intended authorization or state-machine question. Behavior after expiry at decision time remains an interface/product detail to settle rather than infer silently.

Each negative check needs a meaningful positive control. Foreign-tenant access requires a record that actually exists in A and an A actor who can obtain the permitted view. Self-approval requires a pending request and a requester who otherwise holds sufficient roles. A completed or nonexistent request could deny the action for an irrelevant reason.

The proposed requirement/check inventory is:

ID	Business obligation	Proposed acceptance probe - all Not run
CS-01	Established identity and tenant membership	Valid member versus missing/invalid identity and non-member
CS-02	Complete valid creation	Future expiry and valid content versus invalid/missing fields without partial state
CS-03	Tenant-isolated reads, lists and searches	Existing A record read/list/search by A versus B, including direct identifiers

ID	Business obligation	Proposed acceptance probe - all Not run
CS-04	Current eligible approver	Same-tenant approver versus revoked/foreign/requester-only actor
CS-05	No self-approval with overlapping roles	Pending request, requester with approver/admin roles, and separate eligible-actor control
CS-06	One eligible approval under base policy	Valid v1 decision completes; invalid actions do not contribute
CS-07	Consistent terminal behavior	Pending rejection/approval, then prohibited conflicting terminal changes
CS-08	Idempotent retries	Same key/payload repetition and conflicting key reuse under the future frozen contract
CS-09	Concurrency preserves policy	Competing decisions and duplicate submissions with resulting state/history inspected
CS-10	Durable tenant-scoped audit	Restart, permitted history read, foreign-history read and mutation attempt
CS-11	Scoped role administration	Local changes, foreign membership denial and preserved self-approval exclusion
CS-12	Two distinct high-risk v2 approvers	First leaves pending, second distinct completes, repeated first does not advance
CS-13	Explicit version transition	New v2 versus existing v1 and standard-risk versus high-risk
CS-14	No prohibited data/secret leakage	Error/log inspection under the selected disclosure policy
CS-15	Predetermined boundary repair	Known failing baseline probe before repair and retained regression evidence afterward

The oracle is the reviewed decision procedure that relates these expectations to observations. It includes fixtures, assertions and adjudication rules for ambiguous results. It must be independent of the implementation's accidental behavior. A generated test that repeats a defect is not a valid external expectation.

Several details remain unfrozen: exact interface, response/disclosure policy, idempotency-key scope/retention, error representation and persistence/concurrency design. This chapter does not invent those decisions to make the checklist appear executable. A measured exercise must settle them before exposing each framework to the task.

Role administration also needs episode scope. E1's bounded feature can use prepared membership fixtures without implementing a complete admin interface. If administration is added to scored work, add it uniformly before runs. The common specification can cover more product behavior than one episode implements.

Design a fixture that answers one question

A fixture is part of the argument for a check. Its important properties are the ones that make the target rule distinguishable from other reasons the operation could succeed or fail. A convenient collection of users and requests is not sufficient if the expected outcome has several unrelated explanations.

For the requester exclusion, prepare two equivalent pending requests in A, with the same policy and valid creation conditions. In one independent check, Mira has requester, approver and administrator roles and attempts to approve her own item. In the positive control, Leon is a current eligible A approver who did not create the other item. The actor/request relationship is the material difference. Do not run the second case on the already completed output of the first.

For tenant isolation, start with a known A request that an A member can read. Use Ozan in B to attempt the corresponding foreign access. A denial is informative only alongside evidence that the target exists and the permitted reader works. For list/search checks, choose fixture values that would match the query if tenant scope were removed. A search term that matches nothing in either tenant would make a broken scope look correct.

For current membership, preserve the relevant role while marking the actor's membership inactive before the attempted action. Otherwise a denial could be explained by absence of the approver role. For a role-only check, keep membership active and remove the required role. A small fixture matrix makes those distinctions reviewable:

Probe family	Condition deliberately varied	Conditions held suitable
Self-approval	Actor is the requester	Pending state, same tenant, current membership, approver role
Tenant boundary	Actor's permitted tenant context differs from target	Existing target, active eligible role in the actor's own tenant
Revocation	Current membership is inactive	Existing pending target, otherwise relevant role
Role eligibility	Approver role is absent	Active same-tenant membership, different requester
Terminal behavior	Request is already approved or rejected	Otherwise eligible different actor
Distinct-count behavior	Actor already contributed an approval	Pending high-risk v2 request, current same-tenant eligibility

These are proposed controls. They have not been instantiated against an application. They are also not a rule that every check must vary exactly one input: integration behavior often involves several conditions. Their purpose is to make the claim and the possible confounders explicit.

Separate required semantics from unresolved policy

Do not let a larger fixture set create new product rules accidentally. Current membership at decision time is defined. The effect of revoking a *previous* approver after their valid contribution but before a second approval needs a separate policy decision. The existing requirement does not explicitly say that historical approvals become invalid retroactively. That scenario should remain outside scored expectations until the contract settles it.

Expiry creates a similar boundary. Creation needs a future expiry. The exercise has not settled every consequence of reaching expiry while a request is pending. Use a controlled clock to keep ordinary authorization checks away from that ambiguity, and record an explicit decision before testing expiry-at-decision behavior. A model should not infer it from the word temporary.

For each unresolved scenario, record the accepted surrounding rule, the ambiguity, the alternatives and which checks depend on the answer. That is a

concrete preparation task rather than a reason to ask the author the same broad case question again. The comparison can continue as a documentary walkthrough while the executable contract remains unfrozen.

An observation has more than a result label

A useful future observation record identifies the starting fixture, command, expected result set, resulting business state and relevant side effects. Add revision/environment identity and the actual command/output locator when execution exists. A field called Pass without that context is difficult to challenge or reproduce.

The expected result can be a set of legal histories, especially for concurrent work. It should be defined before observing an implementation. An operator must not select whichever outcome occurred and then call it the intended one. Conversely, requiring one arbitrary ordering can reject an implementation whose result is legal under the agreed model.

Keep an adjudication rule for observations that do not fit the expected shape. An environment failure may make a business assertion unassessed. A malformed fixture may invalidate the check. A genuine forbidden transition may establish a product defect. Retain the raw observation while the interpretation is reviewed; changing its category should not erase the original record.

Freeze profiles and per-episode baselines

A documentary pin identifies the instructions inspected. An experimental freeze requires more: actual package contents, host/model conditions, enabled modules, application stack, baseline code, fixtures, checks and allowed interventions. None of those additional controls should be inferred from a repository URL.

The working source identities are:

Subject	Documentary source identity	Execution disposition
BMAD	`bab9082c3e884e0d72c759bcd1a6d90bcc01be84`	Native path/module/host and installed artifact unfrozen

Subject	Documentary source identity	Execution disposition
GSD Core	`72c4e14dfa56ecc98c04 b3fd6736bea2dd3195dc`	Contemporary lineage scope; runtime conflict and package identity unresolved
AWS workflow	`3939a8b9182f2e4153b9 2028057a7be74e0684c2`	Profile, guard and local/operations route unfrozen
AIUP	`3ad51b0707e9bed6e54b 9de46ebc2f28d7afa960`	Core/Node candidate; common stack and coverage route unresolved

These are the sources used in chapters 4-7, not four completed installations. The source receipts preserve their provenance and inspection limits. All thirteen experiment readiness checks remain open.

Each episode needs a controlled starting state. E1 starts from one neutral scaffold. E2 starts from one externally prepared working baseline with v1 behavior. E3 starts from one prepared baseline with a predetermined tenant-boundary defect. No such code baseline exists here, so there is no baseline hash to report.

Matched starting state is different from longitudinal continuity. If every method maintains its own E1 output in E2, the episode includes earlier architectural and quality differences. That is a legitimate different study, but it does not isolate the change procedure on a common application. Keep the designs separate.

Neutrality also needs review. A scaffold with one method's domain layers, hidden helper implementation or preferred generated tests could advantage that arrangement. Define the interface and public obligations without embedding a solution. Record any unavoidable adapter or host asymmetry rather than silently call it controlled.

For actual runs, predeclare operator familiarization, order, replication, resource ceiling, stopping conditions, missing-data treatment and permitted manual intervention. Count worker calls and setup/adaptation effort. No fixed model access or spending authority has been established by this documentary chapter.

What neutrality requires of the scaffold

A neutral scaffold should make the common task possible without deciding its implementation. It can define how the exercise starts, how synthetic identities are supplied, how checks address the business interface and how a clean reset is performed. It should not secretly contain the completed approval state machine or one framework's generated architecture.

Some shared components are necessary for a bounded exercise. The evaluation must state which behavior they already supply. If identity establishment is prepared infrastructure, assess its integration and assumptions without attributing its implementation to each method. If an arm must implement it, add that work uniformly to the episode. Mixing those scopes would change both effort and defect opportunity.

A baseline acceptance packet should identify four things: what is already correct, what is deliberately missing, what is intentionally defective in E3 and what is excluded. The E2 baseline needs actual evidence of working v1 behavior before the policy change begins. The E3 baseline needs a reproducible failing probe before repair. A design promise is not that evidence.

Prepare separate working copies per attempt. A reset must restore code, persistence, fixtures, clock and workflow state that the task assumes. Clearing one framework's chat while leaving another's generated application records or previous decisions in place would provide unequal starting information. The procedure must identify the method-native state that is legitimately part of the selected intervention and the stale state that must not leak from an earlier attempt.

The local teaching model cannot simply become the hidden scaffold solution. It already embodies selected approval rules. Giving it to one arm while asking another to derive the behavior would alter the task. It may be a public illustrative reference for readers or a separately reviewed ingredient in an oracle design, but either use must be declared before a measured exercise.

Record deviations at the right level

Suppose a method's selected adapter needs a database not present in the common environment. A manual port, custom adapter or stack change is an intervention, not an invisible setup detail. Record who performed it and what changed. Its cost belongs somewhere in the accounting policy even if it occurs before scored execution.

Likewise, an operator may answer a material policy question after work starts. Preserve the question, answer, affected work and reason it was not settled earlier. The final implementation might still meet the contract. The record would then support a conclusion about the combined assisted arrangement, including the intervention, rather than the initial autonomous route alone.

These controls do not guarantee a causal experiment. They make its remaining confounders inspectable. The present chapter has no completed baseline/reset exercise and therefore makes no stronger claim.

Verify that two baselines agree before comparing their repairs

A future evaluator should inspect the prepared baseline through a small set of common observations before dispatching it to any arm. Confirm the expected initial records, the relevant policy behavior and the known defect for the repair episode. Record the source identity and fixture identity together. A matching source revision with different fixture populations can produce a different task.

Preserve a clean copy of the baseline and the preparation record. If an operator changes it while diagnosing setup, distinguish that attempt from the intended starting state. The evaluator can decide whether to restart from the clean baseline or report the deviation. Either choice needs a visible account before results are compared.

Use the same principle for the policy-change episode. The baseline needs a request that genuinely predates the switch, along with the current rule and the observation channel. A manually invented precondition that only one arm receives would change the information available for the task. Freeze that preparation before examining candidate output.

E1: a bounded feature

E1 asks each selected arrangement to address the same base behavior: valid request creation, eligible decision, tenant-local reads/history and the applicable audit/retry/concurrency obligations. Administrative fixtures can remain outside the scored feature scope when declared uniformly.

The first business review should make the requester exception and tenant boundary explicit. A plausible plan that treats administrator status as universal authority would contradict CS-05. A history endpoint scoped only by request identifier would need examination against CS-03/10. These are diagnostic questions, not defects observed in an implementation.

The source mechanisms suggest different documentary trails:

Arrangement	Native trail to inspect for E1	External question
BMAD	Intent-sized planning, named specification where used, linked Build record	Are accepted constraints present through the chosen small/epic route?
Core	Discussion context, bounded plans, summaries and verification	Do context and checks include constraints the implementation might omit?
AWS workflow	Intent, configured route/unit records, approval and boundary evidence	Which decisions were reviewed and which checks were actually active?
AIUP	Requirements/use case, selected construction inputs and acceptance trail	Are applicable constraints linked and realized through the chosen adapter?

The deep dives cite the exact instructions behind these trails. The table is an author-designed inspection plan; it does not report native files generated in four sessions.

For a future completion record, keep requirement-level states. A positive-path demo may establish one observation while leaving retries and foreign-tenant reads Not run. Do not promote an aggregate “done” label to full acceptance. The external inventory should identify every applicable unmet obligation.

A useful independent check also isolates its cause. The self-approval probe starts pending, with a role-overlap requester and a valid tenant relationship. The positive control uses a separate equivalent request and an eligible different actor. The foreign-read probe uses a known existing A record, not an arbitrary missing identifier.

No observation is supplied here. E1's documentary result is the inspection trail and the list of decisions that must be settled for execution. Its implementation acceptance, effort and defect outcomes remain Not assessed.

Four native handoff sketches

The following short records are original teaching sketches. They express the same accepted business intent in forms a reviewer could look for along each native trail. They are not copied prompts, official templates or outputs of installed workflows. The aim is to show a meaningful handoff without forcing identical artifacts onto all four methods.

BMAD sketch. A specification capability describes an eligible decision on a pending tenant-local request, with requester exclusion as a constraint. Its success condition points to the business state and durable history, not just an approval button. A linked story identifies the independently reviewable slice and the checkpoint chosen by the operator. The Build record should explain the relevant existing patterns, implementation changes, checks and deferred unrelated work. If the route is smaller, the reviewer still needs to find the accepted constraints in its current record.

The diagnostic question is whether decomposition preserved cross-cutting obligations. A story for the history reader cannot treat tenant isolation as another story's future responsibility. A record can link to the common requirement instead of repeating it, but the link must make the applicable behavior recoverable.

Core sketch. Phase context records the accepted actor/tenant policy and the open interface details. The bounded plan identifies its goal, dependencies and acceptance criteria. A worker should receive the relevant current context and plan, and its summary should distinguish changes from unexecuted checks. Verification should examine requirement and decision coverage, while the external inventory checks obligations omitted from a summary-derived list.

The diagnostic question is whether context selection changed meaning. If the phase goal survives but the requester exception disappears from the worker's input, the record is persistent yet incomplete. A fresh session does not cure that omission. The observation needed later is the actual handoff and resulting correction, not a claimed capacity number.

AWS workflow sketch. Intent records the business goal and exclusions. The selected route identifies its included stages, unit structure, approval duties and active guard configuration. A design decision makes actor/request/tenant relationships explicit. Construction and boundary records explain which artifacts and links were checked. Local substitutions and operational exclusions remain visible instead of being hidden behind a lifecycle completion status.

The diagnostic question is which authority advanced the work. Approval of a development plan must not be confused with an approval in the business application. A workflow audit can help reconstruct the former; the application's tenant-scoped audit must establish the latter. Their different purposes remain visible in the evidence packet.

AIUP sketch. Requirements identify tenant locality, current eligibility, no self-approval and durable decision history. A use case organizes the actor's goal, preconditions, permitted path and required alternatives, linking all applicable constraints. Construction uses the selected core/adapter inputs, and tests and acceptance records address the same business behavior. Any manual coverage route compensating for an adapter limitation is named rather than attributed to automatic core enforcement.

The diagnostic question is whether the behavioral links are complete and correct. A use case can reference a membership requirement while missing the requester exception. A code marker and test label can resolve while the assertion exercises the wrong state. Structural resolution is valuable, but semantic review and actual acceptance still have separate jobs.

Across all four sketches, preserve native identifiers and add the external CS mapping beside them. Do not rename a GSD phase as an AIUP use case or require every method to emit a BMAD story file. The comparison concerns how the method carries the common work, not whether it can imitate another method's paperwork.

E2: a policy change

The E2 change introduces two distinct eligible approvals for newly created high-risk requests. Standard-risk v2 requests still need one. Existing v1 requests retain the policy bound when they were created. This is accepted case direction, not a conclusion discovered by a run.

The proposed behavior matrix makes preserved and changed expectations visible:

Request fixture	Required interpretation
Pending v1 high-risk request	One eligible distinct approval can complete under v1
New v2 standard-risk request	One eligible distinct approval can complete
New v2 high-risk request, no approval	Remains pending until its required distinct approvals are obtained
New v2 high-risk request, one eligible approval	Remains pending; decision/history remain consistent
Same first approver repeats	Does not advance the distinct count; retry details follow the frozen contract
Second distinct eligible approver acts while pending	Can complete if no competing terminal outcome has already prevailed
Already terminal request	A later conflicting action cannot switch its accepted outcome

The distinction between a valid partial approval and terminal completion matters. A test that checks only the final approved state could miss an implementation that completes after the first approval. A test that checks only two actor identifiers could miss a revoked or foreign-tenant participant. Preserve both policy and eligibility.

The native change trail differs by method, but each should carry the same semantics. BMAD's named specification update path can preserve capability references. Core's new decisions/plans and fresh verification can carry the altered rule. AWS's configured route can record the intent and approved construction change. AIUP's requirements/use-case reconciliation can

identify downstream effects. Those are documentary paths, not evidence that the paths succeeded.

Concurrency adds valid outcome sets. Under v1, an approval and rejection racing on one pending request must not produce incompatible terminal states or mismatched history. Under high-risk v2, the order of partial approvals and rejection can matter while pending. The acceptance contract must identify legal histories under the chosen semantics rather than demand one arbitrary scheduler order.

Record actual interleavings or a controlled test strategy when execution exists. A sequential test with two calls does not establish a concurrency result. Likewise, a schema field called `policyVersion` does not prove correct binding; its use in every relevant decision path needs inspection.

E2 is designed to expose maintenance, context and traceability questions. It supplies no measured superiority. A long-lived benefit would require more than this one change and should not be inferred from a neatly updated artifact.

Read a proposed trace as an expectation

Here is an authored scenario for a newly created high-risk v2 request. Each row describes an expected transition or invariant to check later; it is not a log from an execution. The concrete fixture identities and interface still need a reviewed contract.

Proposed input sequence	Expected checkpoint
Create a valid request under v2	Pending, bound to v2, no counted approvals
Leon supplies an eligible approval	Pending, one distinct contribution, required history consistent
Leon repeats the contribution	Still one distinct contributor; retry/error details follow the future contract
Priya is inactive and attempts approval	No successful new contribution
A separately prepared eligible Priya attempts approval while pending	Two distinct eligible contributions can complete the request

Proposed input sequence	Expected checkpoint
Another action tries to change the completed outcome	Terminal outcome remains consistent with the accepted history

The inactivity and eligibility cases must be prepared deliberately. A test that mutates membership as part of its fixture should record when the change takes effect. If the test relies on a separate administration operation, that operation becomes a dependency whose correctness must be established for the observation to mean what it claims.

Repeat the relevant checks on a pending v1 high-risk request after v2 is introduced. Its one-approval expectation must come from the bound policy rather than a global current-policy lookup. A field named policy version is not sufficient: inspect how the decision path uses it and whether readers/history retain the same interpretation.

Distinct approval is not request idempotency

The local model demonstrates that Leon cannot count as two different approvers. It intentionally does not implement a client request-key contract. Those are different mechanisms. A second call can be a retry of the same command, a new command from the same actor or a command that reuses a key incorrectly.

Before an executable comparison, define the key's scope. Tenant-wide, actor-scoped and target/operation-scoped designs have different collision and information-disclosure consequences. The case does not yet choose one. Whichever design is selected, apply it uniformly and define which payload differences constitute conflicting reuse.

Also define which authority applies before a replayed result is returned. A stored result should not quietly create a route around the current identity/tenant policy. The exact interaction among authentication, permission changes and replay needs a reviewed contract; the model's internal duplicate-actor reason code cannot answer it.

The acceptance inventory should distinguish at least these cases:

Case	Question the future check must settle
Same authorized command/key/payload repeated	Is the business effect represented once, with consistent replay behavior?
Same scoped key with changed payload	Is conflicting reuse handled without a second unintended effect?
Same actor with a new key on high-risk v2	Does distinct-actor counting still prevent a second contribution?
Same visible key in another tenant/actor scope	Does the chosen scope prevent cross-boundary disclosure or collision?
Retry after terminal completion	Does replay preserve the agreed result without creating new history?
Retry after authority changes	Which current-access and replay rules govern the response?

These are contract-design questions and proposed probes, not newly asserted HTTP behavior. No response codes, retention duration or replay body has been frozen. That restraint prevents a method's preferred implementation from deciding what the oracle later calls correct.

Observe state and history together

An implementation could satisfy a final-state assertion while losing the corresponding history, or retain a history event for a business transition that never became durable. CS-09 and CS-10 require the observation plan to connect these facts. The database and atomicity mechanism remain open, so this chapter does not prescribe a transaction recipe.

For a future fault exercise, identify intended interruption points such as before persistence, between related writes and after a durable effect but before acknowledgement. Define the legal recovered states before injecting faults. A consistent completed effect and a consistent absence of the effect are different possibilities; an unmatched business/history state should not be accepted merely because one record looks correct.

Partial approval under high-risk v2 adds another legitimate state: pending with one accepted contribution and its consistent required history. Recovery must not confuse that state with a failed terminal transition. The observation

needs policy version and contribution identities as well as the overall request status.

This is why a sequential function check cannot establish persistence or race correctness. The model helps inspect selected policy logic. Application-level observation must address how that logic is integrated with actual identity, storage and concurrent requests.

Define legal histories before racing commands

Consider a pending v1 request with no approvals. An eligible approval and an eligible rejection compete, with membership and other policy conditions held constant. The acceptance design should permit either command to prevail when their relative order is unconstrained. It should not permit a final approved state alongside a successful terminal rejection in the same business history.

The observer therefore needs more than two response bodies. Preserve the submitted operations, their acknowledged outcomes, the resulting request and the required durable history. The frozen interface may distinguish a rejected competing operation from a transport failure whose outcome is initially unknown. Those cases need different reconciliation rules; the chapter does not assign response codes in advance.

For a high-risk v2 request, the first approval is a successful partial contribution. If a rejection becomes effective while the request remains pending, the resulting history can legitimately include that contribution followed by a terminal rejection. If two distinct eligible approvals complete before a rejection takes effect, the accepted terminal history must remain consistent with approval. The oracle must inspect the complete sequence under its declared concurrency model rather than simply prohibit every mixture of approval and rejection records.

This distinction also prevents an overly strict test from rejecting valid business history. A prior partial contribution is not a second terminal outcome. A separate security log of an unsuccessful attempt is not a successful business transition. Define which record types the assertions count and which additional diagnostic records are allowed.

None of these histories has been observed here. They are proposed constraints for independent review before a concurrency test exists. The

implementation's database isolation, synchronization and retry behavior still need a real design and exercise; a plausible order described in prose supplies none of those guarantees.

Check version binding through creation and change

The local policy model reads a supplied version. A stronger application check must establish where that version came from and whether later policy activation changes existing work. Merely constructing an in-memory record marked v1 cannot prove the creation/migration behavior required by CS-13.

A proposed acceptance card begins with a controlled v1 policy and creates a valid high-risk request through the agreed application boundary. Retain evidence of creation and leave the item pending. Activate the reviewed v2 policy through the exercise's defined setup procedure, then create new standard-risk and high-risk requests. Keep expiry, membership and other conditions suitable throughout.

The original high-risk request should still use its v1 threshold. The new standard-risk request should need one eligible contribution, and the new high-risk request should remain pending after its first and complete only after a second distinct eligible contribution. Observe the required history as well as state. A completed v1 request should also remain a consistent historical decision after policy activation.

The observation method must be agreed in advance. The contract may expose a version in the permitted response, or the reviewed harness may use a defined evidence view. The case does not require a newly invented public debugging endpoint. What matters is that the assertion establishes creation-time binding without trusting a field injected by the test itself.

This card is a future design, not an executable test or a completed migration exercise. It identifies the integration evidence missing from the narrow function example and gives every method the same preservation obligation.

E3: a boundary defect

E3 uses one predetermined defect in a shared prepared baseline. The proposed defect concerns tenant isolation on a defined query path. Its exact implementation and hidden acceptance probes must be fixed before runs; the current project has not created or exercised that seed.

A useful defect report should describe observable behavior without handing one method a more helpful diagnosis. For example, a synthetic report could identify that a B actor obtains data associated with an A request through a particular operation. That is a proposed report shape, not an incident that happened here.

The diagnostic procedure should first reproduce the boundary under controlled identities and a known existing record. Inspect the request/tenant relationship, current actor membership and relevant query/authorization path. An access denial obtained only by deleting the fixture would not be a repair.

Narrowness concerns intent, not the smallest line count. A correct repair may need a shared boundary check and multiple regression probes. An unrelated architectural rewrite would need a separate reason. Preserve failed attempts, especially when one closes the visible path but leaves a related route exposed.

The common acceptance inventory includes direct identifiers, lists and searches. The seeded defect may affect only one path, but regression scope should follow the shared boundary and the predefined contract. Do not expand hidden tests after observing which method performs well.

The four methods may legitimately take different small-change paths. Compare the documented selected route rather than force full greenfield planning on all of them. Record any departure, manual intervention or unavailable host capability. A profile change is part of the evaluated arrangement.

The expected execution packet would contain baseline evidence that the fixed probe fails before repair, actual change/diagnosis records and after-repair regression observations. None is present. CS-15 therefore remains Not run; the prose does not simulate a failed or successful trial to complete the narrative.

Distinguish a narrow repair from a narrow check

A small code change can close a shared boundary while a single happy-path check leaves it unexamined. Conversely, many new tests can all exercise an unrelated denial condition. Choose regression scope from the affected requirement and dependency path, not from the patch's line count or the size of the generated suite.

The predetermined failing probe should name the actual operation and fixture. If a search path leaks an A record to B, choose a term that would reveal that record and keep the permitted A search as a control. If a direct-ID reader is affected, establish the target's existence and the actor's valid membership in B. Preserve those conditions across the before/after observation.

Then inspect related paths according to the predeclared acceptance inventory. A shared query helper could affect list and history behavior as well as direct lookup. A framework may recommend a broader review, but the scored boundaries and hidden checks must not be adjusted after observing an arm's result. Additional discoveries can be retained as out-of-scope findings or a separately declared follow-up.

Keep a failed attempt as an artifact with its baseline, change and observation. A later accepted repair does not make the earlier investigation disappear. The record may explain a context omission, a wrong diagnosis or an oracle defect. Those explanations have different implications for process selection.

For a future reset, restore the same prepared defect rather than hand-edit a vaguely similar error. Verify the baseline hash and fixture state, then confirm that the predeclared probe still exposes the intended behavior. Otherwise later arms might receive a different challenge even when the report text is identical.

No fault injection, baseline reset or repair check has been performed in this manuscript package. The concrete design gives a reviewer something to inspect and a future operator something to implement, without manufacturing the missing observations.

Inspect the repair for permitted access as well as the leak

The proposed E3 repair starts from the same planted tenant-boundary defect for each future arm. One approach might add a tenant predicate at the reported endpoint. Another might change a shared lookup so callers must supply the tenant scope. Before choosing a design, inspect which paths use the affected helper and identify which ones should continue to retrieve the request. An overbroad fix that makes all reads fail would suppress the symptom and break the intended A path.

A proposed acceptance pair uses the same known A request. An authorized A actor exercises the reported path and sees the data permitted by the contract. An actor in B tries that identifier and receives the defined disclosure behavior. If the repair changes a shared lookup, repeat the observation for the applicable list, search or history paths. The exact interface and hidden regression inventory still need to be frozen by the evaluator.

Keep the planted defect and the acceptance observations outside the native repair notes used by an implementer. The issue should include the agreed reproduction and enough context for a legitimate investigation. The evaluator's additional probes should be prepared before seeing any repair. That division makes it possible to compare how the native work identifies affected paths without turning the hidden checks into a guide for one of the arms.

What the records support

The records currently support a business specification, source-specific process mechanisms and an original comparative inspection design. They support questions about how intent, context, authority and acceptance would be related. They do not establish implementation results or framework effectiveness.

Keep a claim/result distinction at the episode level:

Item	Current disposition
Accepted domain and language family	Recorded author direction
Business requirement inventory	Design definition; executable review/freeze pending
Native artifact trails	Documentary teaching maps, not generated run outputs
Separate author-written policy model	Ten local sequential checks passed; not application acceptance or independent validation
E1 implementation acceptance	Not run
E2 policy-change acceptance	Not run

Item	Current disposition
E3 failing baseline and repair regression	Not run
Timing, cost and defect comparison	Not assessed
Production/team/longitudinal outcomes	Not observed

An author-designed example can reveal an ambiguity without proving how often it occurs. The earlier pilot's self-approval fixture ambiguity is a real editorial finding in the book material, not an application bug or a failed framework trial. Preserve that distinction when using it diagnostically.

If later execution fills these gaps, retain adverse results and the original protocol. An installation problem should not become an application failure score. A successful manual correction should not disappear from the effort record. A changed requirement should trigger a task-identity review rather than be silently absorbed into a favorable result.

The qualitative route is the honest current deliverable because essential execution controls are unmet. It remains useful as a reproducible way to inspect primary instructions against a shared business problem. It cannot answer which method is faster or safer.

Separate what the walkthrough teaches from what a later run would establish

The walkthrough follows one set of business obligations through several representations. A reader can inspect whether the proposed artifacts preserve tenant scope, requester exclusion, version binding and distinct contributions. That inspection can reveal a contradiction in the design before any application exists. Record such a finding as a design issue with the owning rule and affected handoff.

An executable episode would add a different observation. It would identify the actual environment, input sequence and resulting durable state. A passing observation on one path supports that path under its fixture. A failing one can reveal a defect or an invalid fixture, and the record should preserve enough

information to tell which. Neither outcome should be backfilled into this walkthrough as though the proposed artifact had produced it.

A comparative run would add the configured method and its operator actions. The evaluator could inspect which decisions were available, which interventions were needed and what each candidate produced against the common boundary. The case design gives that exercise a starting point and shows where independence and baseline control are required.

The resulting evidence packet should preserve all three kinds of work. The accepted design explains the expected behavior. The execution record shows what happened. The comparison interpretation explains which claims follow from those observations. Keeping their relationship explicit lets a reviewer challenge a conclusion without losing the useful design or the underlying result.

Reproduce and delimit the case

To reproduce this documentary exercise, begin with the case specification and its CS inventory. Read the four pinned source trails identified in chapters 4-7. Map each requirement into the selected native records, distinguish prescribed from optional work, and record unresolved decisions. Compare the resulting maps using F01-F11 without assigning outcomes that were never observed.

A narrow executable companion is available in examples/approval-policy. It models only an approval transition over trusted in-memory records: tenant/current-role eligibility, self-approval exclusion, distinct counting, terminal rejection of later approvals and creation-bound v1/v2 thresholds. Ten author-written checks passed under the locally available Node.js v26.13.0; the actual output and file hashes are retained. No package was installed.

This observation does not fill the application acceptance table. The example has no authentication, database, audit, HTTP interface, idempotency-key handling, rejection operation or concurrent execution. Its internal duplicate-actor result is not the future retry contract. TypeScript is executed through Node's type stripping; `tsc` is unavailable on the session PATH, so static type-checking was not performed. Node's execution limits distinguish this from compiler validation. The model and checks were authored together and are not an independently reviewed oracle.

The repository's `python3 scripts/check_manuscript.py` command audits manuscript hashes/counts, section/title coverage, citation receipts and local-link paths. It checks document integrity. It does not install a methodology, build the SaaS or certify a requirement. The actual command/output record is linked from the manuscript report.

An executable reproduction requires the additional inputs defined in the run protocol: stack/host/model identities, installed artifacts, per-episode baseline hashes, fixtures, independent checks, reset commands, allowed interventions, resource limits and retained outputs. Missing values must stay missing until genuinely supplied.

Reproduce the documentary analysis

The paper exercise can be repeated without buying a model subscription or installing every workflow. Begin with one CS requirement and one episode. Read the cited pinned instructions and write down the artifact that should carry that requirement, who owns the relevant decision, and what evidence the selected route prescribes. Keep each statement's source locator next to it.

Next, test the interpretation against a counterexample. For CS-05, ask whether a plausible native record could omit requester identity while retaining an approver role. For CS-13, ask whether a global threshold change would satisfy the record while altering old requests. These are semantic probes of the documentary map; they are not attempts to predict a model's output.

Compare your map with the chapter's teaching sketches. Disagreement is useful when it identifies a source ambiguity, a version difference or a missing assumption. Record the two interpretations and the passage that would resolve them. Do not treat agreement with the book as the acceptance oracle for a future implementation.

Finally, complete a disposition for the chosen episode: documented mechanism, external dependency, unresolved contract, or observation needed. This produces a reviewable artifact even when execution is not yet possible. It also prevents an articulate walkthrough from quietly being relabeled a benchmark.

Package the evidence for a future rerun

A future case packet should keep the common task and native outputs in separate locations with explicit links. The common area contains requirement/contract versions, fixture definitions, baseline identities and oracle rules. Each attempt area contains the selected profile/configuration, native planning/implementation/review records, operator interventions and actual outputs. A results index points to observations rather than restating unsupported success claims.

The package also needs a boundary around unavailable information. A hosted model may not expose a fully reproducible internal build identity; disclose the identifier available and run date. A required service may be inaccessible; record the stopped scope. A native artifact may contain sensitive content; preserve a reviewed redaction and state what cannot be independently recovered. None of those limits should be hidden by a generic reproducible label.

The present repository preserves source receipts, manuscript identities and the separate policy-model run. It does not contain the future common application packet. Keeping those artifact classes distinct lets readers reproduce what exists today and identify exactly what is needed for the stronger experiment.

A concrete blank episode packet

An operator should be able to locate the common input and the evidence for one attempt without reconstructing the study from a chapter's narrative. The following packet design is original and proposed. Its fields are deliberately unfilled: no application baseline, installed profile or episode result exists to insert.

Record	Fields to supply before or during a future attempt
Common episode definition	Episode ID, requirement/contract revision, included and excluded work, fixture definition, controlled clock and expected-result rules
Baseline identity	Source revision/hash, dependency identity, persistent-state reset, seed procedure and baseline verification evidence
Selected arrangement	Framework/source/package identity, host/model identifiers, profile/modules, guard/review settings and declared substitutions

Record	Fields to supply before or during a future attempt
Attempt identity	Unique attempt reference, episode, operator, order, familiarization and resource/stopping rules
Native evidence index	Actual intent/context/plan/implementation/review records and links to the external CS obligations
Observation entry	Check ID, starting fixture, actual command, exit/output evidence, observed state/history and interpretation
Intervention entry	Time/order, responsible person, trigger, action, affected artifacts and whether it changed the task
Attempt disposition	Completed, stopped or invalid under the predeclared rules, with missing observations and retained outputs

The common definition and per-attempt identity have different jobs. A change to an attempt's native plan need not change the business task. A newly accepted rule or altered baseline may do so. Keep those revisions separate so a reviewer can decide whether two observations remain comparable.

Use stable references for observations rather than placing an unqualified Pass in the narrative. A reviewer should be able to move from CS-05 to the relevant fixture, then to the actual command/output and the state/history assertion. If a command never ran, the entry remains unexecuted. If an observation cannot be interpreted because setup failed, retain that failure and mark the business question unassessed.

The packet should also expose missing work. A completed local build would not substitute for a restart observation, and a passing policy test would not populate tenant-isolation fields. Do not fill empty cells with a nearby result simply to produce a complete table. Distinct evidence duties stay distinct even when the same command supplies several valid observations.

Review the packet from the perspective of a second operator. Can they identify the intended clean starting state? Can they tell which files are common inputs and which were produced by the attempt? Can they recover the policy decision used by a check? Can they distinguish a manual correction from the original output? A missing answer becomes a preparation defect to fix before claiming reproducibility.

The repository's existing manuscript manifest illustrates exact artifact identity, but it is a document manifest. Its chapter hashes and section checks cannot be reused as an application baseline record. The policy-model run preserves a real command/result, but only for that separate example. The future packet needs its own identities and observations for its own task.

The case can then produce bounded observations about this local synthetic service. Wider conclusions require different evidence: people and teams for coordination, operational exercises for recovery, longer maintenance for drift/cost, and replication for more stable estimates. No one local trial should stand in for all of them.

This chapter gives readers an inspection design they can challenge and a precise boundary around its present evidence. The next chapter develops the acceptance and operational duties needed before those missing results could support stronger claims.

Give another reader a reproducible design packet

For the walkthrough, a second reader should be able to reconstruct each starting state without seeing a model conversation. Give each episode a manifest containing the business obligation, policy version, initial actors and records, known decision owners and the proposed observation. If the case later becomes executable, add the actual baseline revision, schema or fixture version, installed tool profile and commands under that episode's record.

Keep creation and evaluation inputs distinct. For a future repair trial, the implementer receives the authorized issue and a known reproduction. The independent reviewer holds the fuller inventory of protected surfaces and preserved behavior. Each receives a different view of the same agreed requirement, for a different job. A third person should be able to inspect that division after the run and tell whether one arrangement received extra guidance.

An example of a reproducibility failure is a v1 probe that depends on a manually seeded record with no documented version. The reported transition might be repeatable for the original operator and ambiguous for another reader. The repair is to define the record's origin and expected behavior before drawing a conclusion. The same principle applies to the experimental baseline. A Git revision identifies source, while fixtures, configuration and installed packages identify the starting conditions in which it was used.

Use the packet to retain unresolved questions as well as decisions. If replay conflict semantics remain unsettled, label the affected checks as awaiting a contract. The rest of the episode can still be inspected. This makes the qualitative walkthrough useful as a design even when the corresponding execution evidence is unavailable.

12. Testing, Security & Production Readiness

Define acceptance and evidence ownership

Acceptance begins with an expected behavior and someone authorized to decide whether the evidence supports it. Generated code, a test report and a workflow status can contribute, but none can supply a missing product decision by itself. The same obligation applies regardless of which framework arranged the work.

For the SaaS, CS-05 forbids self-approval even when roles overlap. The expectation comes from the agreed case requirement. Implementation and acceptance checks must be assessed against that rule. If a test expects administrator self-approval, a passing result would preserve the wrong expectation.

Assign ownership at the decision level. The product owner resolves intended behavior. The implementing engineer explains the design and changes. The acceptance reviewer checks the expectation, fixture and observation. An operational owner decides which further conditions justify deployment. One person may hold several duties, but the record must expose that overlap.

Independent review requires a real reviewer arrangement with interests disclosed. Another model invocation can provide a useful perspective, but it is not independent human validation. A generated checklist can support a reviewer; it does not appoint one or establish that the review occurred.

NIST's SSDF 1.1 publication describes high-level secure-development practices that can be integrated into a lifecycle. We use it as a reminder that selecting an AI-oriented process does not exhaust secure-development duties. This is a fixed-version reference, not a claim of SSDF compliance. NIST SP 800-218, SSDF 1.1

The acceptance decision should identify the reviewed revision, relevant observations, unresolved risks and scope. “Accept the local synthetic increment with specified limits” differs from “release a supported service.” Keep those decisions separate even when one meeting discusses both.

Write the decision before collecting results

Suppose an engineer proposes releasing the v2 policy to a synthetic test environment. The release question is whether new requests can be bound to v2 while existing v1 requests can still be decided under v1. The reviewer needs evidence about both populations. A checklist headed “v2 tests passed” gives no way to tell whether the release could strand an existing request. Write the decision sentence first, then identify the observations that would change the decision.

One workable separation is to ask the product owner to name the protected behaviors, the acceptance reviewer to define the fixture and inspect the result, and the release owner to set the operating boundary. For this exercise, a release owner might allow a synthetic environment to be rebuilt after a failed migration. The same recovery decision would be inappropriate for an environment carrying real decisions and audit records. State the environment in the disposition so someone cannot lift the sentence out of the packet and apply it elsewhere.

When a check fails, distinguish the requirement from the failure mechanism. If the v1 request stays pending after an otherwise eligible approval, the reviewer can record the contradiction before anyone diagnoses it. If the fixture accidentally assigned v2 at creation, correct the fixture and retain both records. If v1 was assigned correctly and the runtime applied v2 anyway, the application is the repair target. This sequence gives reviewers a common observation even when they disagree about the cause.

Use generated tests without circular validation

Generated tests are useful when their assertions address intended behavior. Their existence and passing status do not establish that relation. An implementation can embody a mistaken assumption, and a test derived from it can repeat the assumption precisely.

Separate three inputs: the business expectation, the fixture that makes the expectation observable, and the implementation under examination. For the self-approval rule, begin with a pending request, valid same-tenant membership and a requester who otherwise holds sufficient roles. An invalid identity or terminal request could produce a denial without exercising the requester exclusion.

Use a separate positive control. An eligible different actor should be able to make the permitted decision on an equivalent pending request. If every action fails because the fixture is broken, the negative example has not established the intended boundary.

The same reasoning applies to tenant isolation. A B actor attempting an A identifier is informative only if the record exists and a permitted A view works. Lists and searches need fixtures that would expose a leak if scope were missing. A page containing no data by accident cannot certify isolation.

An independent acceptance inventory can coexist with native generated tests. Keep the generated suite for its useful implementation coverage, then map external expectations to actual observations. Do not discard a stronger native check merely because it was generated; inspect its basis and scope.

Where justified, a controlled fault or mutation can help test whether an acceptance check detects the intended defect. If removing an eligibility guard leaves the relevant check green, investigate the fixture/assertion. Such a diagnostic must be deliberately scoped and kept separate from production. No mutation exercise was performed here.

Record actual commands and outputs when checks run. Include the revision, environment, fixtures, selected tests, result and interpretation. A textual summary without a retrievable observation cannot support the same claim. Unexecuted checks remain Not run, including every application check proposed in this book.

The policy-model companion supplies a deliberately narrow example: ten locally executed checks of a sequential approval function passed, including a pending-state self-approval negative case and a separate positive control. Its run record preserves the command, runtime and file hashes. That is an observation about the authored in-memory model, not independent application acceptance. Database, audit, retries, concurrency and static type-checking remain outside that result.

Design an oracle around a plausible false pass

Take two requests in the same tenant, both pending and owned by Mira. On the first, submit a decision as Mira with the roles needed to pass the ordinary role check. On the second, submit an equivalent decision as Leon with the same applicable role. Establish that both actors can reach the operation under the fixture's authentication assumptions. Keep request state and policy version equal across the pair, and vary the relationship between actor and requester. The positive control needs its own record so it cannot be affected by the denial attempt.

Observe both the decision and durable state. If the self-approval action returns the expected denial but inserts an approval row before returning, an assertion on the response alone misses the defect. A later eligible decision can make the inconsistency harder to diagnose. Read state and history through an agreed observation path, compare them with the known initial values, and name the exact records that are forbidden to appear. Any denial event permitted by the contract needs a separate assertion from the prohibition on a success event.

A test author can diagnose the quality of this check by proposing an intentional fault and predicting which assertion would notice it. Imagine preserving the rejection response but moving the state write ahead of the actor check. A robust design would detect the write regardless of the apparent response. Imagine instead making every request fail before authorization. The positive control should detect the broken entry path. These are thought experiments for inspecting a proposed oracle. If a reviewer later executes a fault probe, restore the reference revision and record the faulted revision separately.

One suite can contain fast in-memory checks and slower tests across the actual persistence boundary. The first can answer whether the policy function computes a transition. The second can answer whether the service applies it to the right row under the agreed fixture. Give them distinct names and retain both. A failure in a fast check helps localize a policy regression, while a failure only at the boundary can reveal integration behavior the function was never asked to model.

Trace requirements through changes

Traceability is an inspection route. It should let a reviewer travel from requirement to decision, implementation and observation, and back from a material change to its justification. The quality of the links matters more than their count.

For E2, the changed high-risk threshold is only part of the requirement. Existing v1 requests retain their creation-time policy, standard-risk v2 remains one-approval, and tenant/self-approval constraints continue to apply. A narrow test of the final high-risk approved state leaves several obligations unresolved.

Use a requirement-impact record to distinguish changed, preserved and newly relevant checks. A stable requirement ID can retain its intent while new code locations depend on it. A test that was sufficient before the change may no longer address all affected paths.

The observed revision must also match the reviewed one. GSD Core's source describes fingerprint-based verification staleness and a legacy modification-time rule. That is one explicit freshness mechanism, not proof that all relevant inputs are covered. Inspect both freshness and semantic coverage. Core verification freshness

AIUP separates structural specification checks, semantic advice and implementation/test coverage, with adapter-specific limits. BMAD separates built-in generated QA from optional broader testing. AWS profiles alter checks and reviews. Their mechanisms can support the evidence packet, but the common acceptance question remains external to any favorable status. AIUP coverage scope, BMAD QA boundary, AWS profile differences.

A useful report therefore includes both the native verdict and its interpreted scope. If a requirement is linked but untested, say so. If a check failed for an unrelated fixture issue, preserve the failure and correction rather than call it a product defect without evidence.

Read the change from both directions

An impact review begins at the requirement and ends at something observable. For creation-time policy binding, the reviewer first locates where creation selects the version, then where decisions resolve that saved version, then where a check creates a request under one policy and decides it after the policy changes. A test that inserts a v1 marker directly can isolate the resolver, but leaves the creation path open. Pair it with a sequence that goes through creation and assert the stored version before attempting the decision.

Then reverse the walk. Start from each edited boundary and ask which obligation it affects. If a migration changes both the policy representation and the index used for tenant-scoped lookups, the change reaches beyond threshold behavior. A reviewer needs to know whether direct lookup, history and search still enforce the intended tenant scope. The code diff can reveal an affected path that the original ticket did not name. Update the impact record before using the earlier acceptance plan.

Changed surface	Preserved obligation	Observation to request
Request creation	Version fixed at creation	Requests created before and after policy switch retain their respective identifiers
Decision command	Distinct eligible actors	Repeat one actor on high-risk v2 and inspect count and terminal state
Migration	Existing v1 stays decidable	Use a pre-migration fixture, migrate it, then decide it through the normal entry point
Tenant lookup used by history	Foreign actors cannot obtain case history	A known existing record is visible within A and protected from B under the agreed contract

The table is an example of a proposed review, and every observation is still open. The useful evidence key is the combination of expected behavior, observation and identity of the checked artifact. If a developer edits the migration after the reviewer observed it, a passing log can remain useful history. The decision about the new revision needs either a fresh observation or an explicit explanation of why that specific evidence still applies.

Use a trace review to decide which checks need another run

A small change to a display label can leave policy-function evidence intact while requiring a visual check at the affected screen. A change to the resolver can leave the accepted policy record intact while requiring fresh behavior observations. Write the affected dependency next to the check so a reviewer can explain the difference.

For the case, a migration change can affect the creation-time version even if the decision function remains byte-for-byte identical. The preservation probe needs the migrated record as its input. A fresh function check alone would leave that path open. Conversely, if a documentation correction changes no executable input, a reviewer can retain the earlier run and inspect the corrected explanation.

The decision to reuse evidence should state its premise. Identify which inputs stayed the same, which changed and why the observation still addresses the claim. If the inputs cannot be recovered, mark the evidence as historical and schedule the observation needed for the current candidate. This is a bounded freshness review that a later contributor can repeat.

Examine the SaaS boundaries

The case's security-relevant boundaries are business-specific. Identify the actor, target resource, tenant, current membership, requested action and current state. An administrator role alone cannot override the requester exclusion. A difficult-to-guess identifier cannot substitute for an authorization rule.

OWASP's authorization guidance recommends denying by default and validating permissions on every request. Those are general principles; the concrete case expectation still comes from the defined tenant and actor rules. A local check against selected paths does not establish comprehensive authorization coverage. OWASP Authorization Cheat Sheet

An original boundary model for this exercise is:

Boundary	Case question	Proposed negative/consistency evidence
Identity to membership	Is this actor a current member of the selected tenant?	Invalid, foreign and revoked membership fixtures
Membership to request	Does the target request belong to the permitted tenant scope?	Direct-ID, list, search and history probes
Actor to decision	Is the actor eligible and different from the requester?	Pending-state overlapping-role control and permitted different actor
Decision to persistence	Can competing or repeated actions break the versioned policy?	Defined retry/concurrency tests with final state/history inspection
Persistence to history	Is the successful transition represented durably without foreign disclosure?	Restart, scoped read and history-mutation probes
Service to logs/errors	Does the defined response disclose only permitted information?	Contract/error/log review with synthetic sensitive markers

Concurrency requires a declared legal-outcome model. Two calls returning success is not necessarily sufficient evidence: inspect distinct approvals, terminal state and audit consistency. Conversely, one losing concurrent action may be expected. The chosen semantics and execution record determine the conclusion.

Durability also needs an actual boundary. An in-memory assertion before restart does not establish persistence. A database row can exist while a matching audit record is missing. The future design must define which changes must commit consistently and how its checks observe that obligation.

OWASP ASVS supplies a broader security-requirement framework; the inspected site identifies release 5.0.0. It can guide a separately scoped assessment. We have not applied every ASVS requirement or certified this case against the standard. ASVS overview and edition

The local exercise intentionally excludes production federation and real access provisioning. Those exclusions remove particular integrations from

the example, not from the duties of a service that later adopts them. Expand the assessment when the system boundary changes.

Use the access model to find paths the happy flow hides

Build a small surface inventory before selecting checks. A request can be reached by a direct identifier, a filtered list, a search result, an audit-history view and an action command. If a future UI adds counts or previews, include those outputs too. Each surface can reveal different information. The visible rows in a list could be correctly scoped while a total count includes foreign rows. A foreign direct request could conceal content while exposing a distinguishable error. The case owner has to settle the disclosure contract before the reviewer interprets those differences.

Construct the fixture so a missing filter could plausibly change the answer. Put matching search terms in both tenants. Use enough rows to exercise a cursor or page transition if that interface exists. Keep the permitted result visible while checking the prohibited result. Use synthetic markers with no production meaning so the reviewer can identify leakage across bodies, counts and history without copying personal data into the evidence.

A write check should distinguish lookup scope from authority over the transition. If B sends a decision request against A, inspect the externally permitted response and verify that A's state and success history remain unchanged. If a tester uses only an identifier that was never stored, the lookup can fail before tenant logic is involved. Establish existence through an authorized A path and use a different observation channel for the negative request. The evaluator can then explain exactly what the case exercises.

Turn competing writes into a finite set of legal outcomes

For a single-approval pending request, imagine an eligible approve and an eligible reject released from a coordinated starting point. The product contract has to say which actions are terminal and how a losing action is reported. If either decision may win, the oracle can accept either of two final histories. It should still reject a history with both successful terminal decisions, an unmatched success event, or a terminal state that contradicts its recorded winner.

A separate probe covers a high-risk v2 request receiving two distinct approvals. The contract may permit either actor to be first. The oracle should

reconstruct the sequence from durable evidence rather than assuming the client response order is the commit order. Both requests returning without transport error gives little information about the resulting state. The useful question is whether the durable outcome belongs to the set the contract permits.

A retry introduces another dimension. The same actor may repeat a successful request after losing the response. The case needs a frozen scope for the key and a defined result when a reused key arrives with a different payload. Keep that test separate from two distinct actors competing. The first tests command identity and replay behavior. The second tests simultaneous transitions and decision eligibility. Combining them in one fixture can obscure which rule prevented an unwanted second transition.

A deterministic schedule can make an exercise easier to diagnose, while repeated schedules can expose outcomes the first one misses. Record whatever coordination the harness actually provided. Launching requests together at the client only proves that the client issued them together. If the service serialized them before the contested boundary, describe that observation and qualify the concurrency claim. A useful report can show a specific prohibited state was absent in a specific exercise without generalizing to all schedules.

Inspect failure after a permitted action begins

Authorization is only the beginning of a successful decision. Suppose a permitted approval reaches persistence and the storage operation fails. The service needs a defined way to represent the action's outcome so a client can decide whether to retry. The contract should identify what proves a committed contribution and what a later retry can observe.

A proposed failure probe can begin with a pending request and one authorized command. Introduce a controlled failure at the boundary the architecture claims to handle, then inspect response, durable state and success history. The expected result depends on the selected design. A single transactional design and a separately delivered history design need different observation rules.

Keep this probe separate from an unauthorized attempt. Otherwise a denial can mask the persistence path the test was meant to exercise. A positive control should demonstrate that the same fixture can complete a permitted

action when the injected condition is absent. Record the actual failure point provided by the harness, since a failure before the write and one after commit ask different questions of the retry contract.

Review dependencies, configuration and secrets

An application can satisfy a business assertion while relying on an unsuitable dependency or configuration. Record exact runtime/compiler/library versions, lockfiles, build inputs and enabled options when execution exists. A framework's minimum version statement does not prove a common stack actually builds or behaves correctly.

The Core guide/manifest discrepancy and AIUP Node adapter assumptions are concrete examples from our evidence base. Resolve them through an installed-artifact and compatibility exercise before publishing a setup recipe as tested. Do not average conflicting prerequisites or transfer one adapter's tested status to another.

Build, lint, dependency and security scans supply scoped observations. A clean report means the selected checks found no reportable issue under their rules/database/configuration. It does not establish absence of every vulnerability. Retain tool versions, inputs, exclusions and findings requiring judgment.

Secrets have a lifecycle beyond keeping them out of a source file. OWASP discusses creation, rotation, revocation and expiration. An exposed credential may need revocation or replacement in its backing system; restarting an application is not that action. We have inspected guidance, not an actual credential exposure in this project. OWASP Secrets Management

Use synthetic data and controlled credentials in any future local example, and keep real secrets out of prompts, logs, screenshots and evidence packets. Record redaction and access rules without including the secret values. If a supporting service is unavailable, document the limitation rather than invent successful integration evidence.

There is also a development-agent boundary. The SaaS need not expose an LLM endpoint for repository text, issue content or tool output to carry adversarial instructions to a coding agent. Treat external content as data, constrain tool permissions and validate consequential actions at the execution boundary. OWASP's prompt-injection guidance describes layered

controls; it does not promise complete prevention. OWASP prompt-injection guidance

Keep agent and application assurance separate. An agent's permission to edit files does not authorize a production deployment. An application's tenant check does not protect the agent from following hostile repository instructions. Both boundaries need their own design and evidence.

Inspect the configuration that will actually run

Start with a candidate release that passed its local application checks. Suppose the test process used a fixture identity provider while the intended environment uses a real integration. The evidence packet should name the identity mode used in each environment and explain which behaviors the test mode bypasses. An engineer reviewing the release can then see whether the claimed check exercised membership, request scoping and policy decisions, and whether it exercised any part of the integration.

For dependencies, preserve the resolved input and the scan configuration together. A finding has an affected component, a version and a path into the build. The disposition should explain whether the component reaches the running service, what action was taken, and how the revised release was checked. If the team accepts a finding temporarily, name the owner and the condition that triggers a new decision. This is an illustrative review procedure. No scanner or dependency database was selected for the case.

The agent boundary deserves a concrete review as well. Imagine an issue attachment containing a command that asks the agent to fetch an environment file and send it to an external endpoint. The issue may provide useful reproduction details while carrying unrelated text. The review should trace the authorized task through permitted reads and edits, then check that external actions require authorization from the controlling workflow. Repository write permissions and application network permissions should be inspected separately. A synthetic exercise can record the attempted action and its disposition without including real credentials or sending data.

Separate the shipped dependency graph from the development environment

Suppose a local check passes using a developer's installed tooling and a lockfile that changed during experimentation. A release reviewer needs to know which inputs actually built the candidate. Record the package lock, build environment and resulting revision, along with the configuration consumed by the deployed service. When a local helper exists only on the developer's machine, describe that dependency before interpreting the command as reproducible.

A finding in an installation tool has different reach from a finding in code loaded by the running service, although both can matter. Inspect the path by which the affected component enters the build or runtime and identify what privileges it receives. A component that executes during the build may see build credentials even if it does not ship. A runtime library may handle request data without ever touching deployment credentials. These distinctions inform the scope of a proposed fix and the evidence needed to review it.

If the fix updates a package shared by test and production, rerun the checks tied to affected behavior and retain the resolved versions used by that run. A scan can contribute a new finding or confirm the scanner's revised view of the dependency graph. The decision about release also depends on the service behavior and on the disposition of any unresolved findings. In this book those steps describe a possible review packet for a future executable case.

From local evidence to operations

A local demonstration can establish useful observations about a selected revision and environment. Production readiness asks wider questions about service ownership, changes, failure and recovery. The word ready needs a declared scope rather than a universal status.

For the approval service, operational questions include detecting decision failures, preserving state/history through restart or migration, recognizing a bad policy rollout and recovering without changing already-bound request semantics accidentally. A rollback plan must account for data created under the newer policy, not only code reversal.

These are questions for a selected architecture and environment. No database, migration tool, deployment platform or backup system is frozen here. We therefore do not supply untested commands or claim that a particular transaction/isolation setting solves every case boundary.

Separate planned controls from exercised controls. A documented restore procedure is preparation. A retained restore exercise with checked business state is an observation. A successful local exercise still leaves scale, failure modes and operational responsibilities outside its conditions.

Assign an owner for each consequential decision: release, stop, recovery, incident communication and acceptance of unresolved risk. A generated runbook can help that person, but it cannot accept responsibility. A process that names an operations phase may still leave these duties to the organization.

The evidence packet should include failure cases as well as the happy path. What happens if a transition cannot persist its audit record? If a migration changes the representation of a policy, how is existing-request behavior checked? If identity or authorization support is unavailable, which operations stop? The exact responses belong in the chosen design and contract before their tests are interpreted.

Operational readiness remains open. The packet identifies the exercises and decisions still needed for the selected environment.

Rehearse a rollback with data already written

Design a recovery scenario around three records: a pending v1 request from before the rollout, a pending high-risk v2 request with one approval, and an approved v2 request with its complete history. The second record is particularly useful because v1 code that reads it as an ordinary pending request could finalize it under the wrong threshold. Before selecting a rollback mechanism, decide how the older application will treat each record shape and policy version.

One proposed response is to pause decision writes while preserving authorized reads and to repair forward with software that understands both versions. Another possible design is an older release that was deliberately made compatible before rollout. The service contract, deployment architecture and already-written data determine whether either response is

available. An operator needs a decision boundary and an observation that tells them which state they are in. Record these conditions before a rehearsal so the reviewer can tell whether the response followed the intended policy.

A rehearsal should check business state after recovery, with pre-change and post-change observations for the same named records. A process health signal can confirm that an endpoint responds. It cannot tell whether the high-risk pending request still has one approval and still requires a second eligible actor. Include an authorized history lookup and an attempted transition under each relevant version in the proposed observation plan. Define who will decide whether writes may resume.

A second rehearsal can address persistence failure during a decision. The release owner needs to know whether the service rejected the action, committed the decision with an authorized delivery state, or entered an inconsistency requiring intervention. Those alternatives depend on the selected architecture. If the case adopts a single durable transaction, the oracle can inspect whether both rows were committed together. If the design adopts separately delivered history, the contract must first define which durable record proves the outcome and how a delayed delivery is represented. Choose the observation to match the design.

Choose operational signals around the failure you can act on

A rollout dashboard could report accepted requests and decision failures. Add the business interpretation before relying on either. An increase in pending high-risk v2 requests may be normal if the second approver has not acted. The same increase among eligible v1 requests after one recorded approval would warrant a different investigation. The operator needs a way to distinguish policy version and risk under the service's disclosure and data-handling rules.

Design the signal with a response. An unmatched success event and terminal state may warrant pausing the affected action while a reviewer identifies the affected revisions and records. An ordinary rise in denials after an authorization change may call for a fixture-based check of authorized paths. Write the action and its owner alongside the proposed signal, so an alert has a useful disposition. For the book's case these are design candidates that would require actual exercises to support readiness claims.

Assemble a release evidence packet

A release evidence packet connects a decision to retrievable facts. Its size should follow the service and risk, but its contents need clear owners and limits. For this case, the proposed packet would contain:

1. Accepted requirements and interface/policy contract, including version binding and exclusions.
2. Exact source/package, application and environment identities, with enabled modules/configuration.
3. Requirement-to-change/check mapping and actual build/test/flow outputs.
4. Review findings, structural/semantic distinctions, human dispositions and interests.
5. Dependency/configuration/secrets assessment with actual scan scope and unresolved issues.
6. Migration, observability, rollback/recovery evidence appropriate to the operational scope.
7. Named acceptance/release authority, explicit remaining risks and stop conditions.

The case packet contains the design, documentary source work and proposed checks described here. No application item can be marked passed. Document integrity checks on the manuscript are a different kind of evidence and must not appear as SaaS acceptance results.

The packet should make disagreement actionable. A reader must be able to identify which requirement or observation blocks acceptance, who can resolve it and what additional evidence is needed. Vague reassurance makes the decision harder to review; an honest Not run makes the next task clear.

Walk a proposed release decision

Imagine the review packet arrives with three entries. A local check confirms that the policy function counts distinct actors under v2. A service-level probe reports that a foreign request receives the contract's permitted response. The migration rehearsal is marked Not run. The first entry supports the function behavior. The second supports a response observation on the specified path, and its reviewer still needs to confirm whether durable state and history were inspected. The third leaves the behavior of existing requests during rollout open.

A release owner could accept the packet for continued work in a disposable environment and leave authorization for any stateful rollout open. The disposition should identify the missing evidence and the next review trigger. For example, require a rehearsed migration and recovery of a pre-existing v1 request and an in-progress v2 request before the stateful decision returns. Use a named revision and keep the original packet available, so later evidence changes the disposition without erasing how the earlier decision was reached.

If the reviewer requests another check, give that request the same precision as a finding. Identify the requirement, expected observation, fixture and owner. “More security testing” is hard to finish. “Add a foreign-tenant write probe against an existing A request and inspect A's durable state and success history” makes an unresolved claim reviewable. Once the check has run, attach the result and update only the claims it addresses.

Maintain the disposition after an accepted change

A team can finish one release review and then discover that a dependency fix changed the runtime used in the acceptance environment. Open the packet at the decision that used the old observation. Identify which parts of the review depend on that environment and which remain supported by unchanged inputs. A specification sign-off can remain applicable while a boundary probe needs to be repeated.

Preserve the earlier approval and record the new condition that reopened the decision. For the synthetic service, the reviewer might repeat a tenant-isolation probe and a migration rehearsal if the dependency change affects database access. The release owner can then make a fresh disposition for the candidate that was actually built. The packet becomes a record of decisions over time, with each approval attached to what reviewers inspected.

The same practice helps an incident review. If a rollout exposes a problem despite a favorable packet, find the accepted observation and reconstruct its assumptions. The fixture might have omitted the affected data shape, or the deployed configuration might differ from the checked one. Revise the smallest part of the review that would have detected the gap and record the later exercise that tests that correction.

Chapter 13 uses these distinctions diagnostically. The aim is to recover from a concrete failure without replacing evidence with another status label, and to improve the process only where the observed problem justifies a change.

13. Common Failures and Anti-Patterns

A diagnostic pattern with evidence labels

A symptom is not a cause. An approval result that violates intent could arise from an omitted requirement, stale context, incorrect code, a wrong test expectation or a broken fixture. Starting with a favored explanation can lead to an elaborate repair of the wrong problem.

Use a diagnostic record with six entries: observable symptom, conditions, candidate mechanism, discriminating check, proposed recovery and recurrence-prevention question. Label the evidence type. A practitioner's report, an executed case observation and an invented teaching scenario are not interchangeable.

For example, “a requester can approve their own item” is an alleged business symptom. To investigate it, retain actor roles, tenant, request state, policy and the operation used. A screenshot of an approval button is weaker evidence than a recorded successful forbidden transition, but neither should be discarded without understanding its scope.

The following sections organize useful failure hypotheses across methods. They do not establish that a particular framework causes them or that they occur at a measured rate. The purpose is to help a reader ask a discriminating question before changing the process.

Keep a symptom distinct from its cause

An illustrative support ticket says two users saw the same approval message. That observation could reflect duplicate delivery, two distinct decisions or a presentation bug that rendered one event twice. It can be tempting to classify the incident immediately as broken idempotency. A diagnosis starts by establishing which layer produced the duplicate and which durable records exist.

Ask the engineer to preserve the two messages and recover their source identifiers under the agreed data-handling rules. If both messages refer to one decision, inspect delivery and display. If there are two successful

decisions for an action the contract treats as a replay, inspect command identity and persistence. If the available evidence lacks identifiers, record that as an observation gap and design a reproduction with synthetic markers.

This approach also protects the process review. A team can see one incident after adopting a methodology and still need evidence that a specific handoff contributed. If the intended replay rule never reached the implementer, inspect how the rule was recorded and passed forward. If the rule reached implementation but a dependency failed during delivery, the useful correction may be elsewhere. Keep the process explanation as a hypothesis until the records support it.

Ambiguous intent and invisible decisions

Hypothetical symptom: a policy change applies two approvals to every pending high-risk request, including v1 requests. The code and tests agree, but the accepted case binds policy at creation.

The first candidate cause is missing or misread intent. Inspect the request, decision record and relevant native artifacts. Was v1 preservation stated? Did a later summary omit it? Was retroactivity proposed and explicitly accepted as a task change? These possibilities call for different corrections.

If the requirement was missing, the product owner must settle the behavior before implementation is evaluated. If it was present but dropped from worker context, repair the handoff and inspect affected work. If it reached the worker and was implemented incorrectly, the technical correction belongs downstream. Do not automatically rewrite the specification to match a coherent output.

A useful recovery record would preserve the agreed rule, the location where meaning diverged, the bounded correction and relevant checks. The recurrence-prevention question is specific: which decision should have been visible at the point of implementation or review?

An additional approval ceremony may help only if it exposes that decision. A general “approve plan” prompt without the material policy can reproduce the same failure with more steps. The evidence should justify the process change.

Recognize when a repair actually needs a product decision

A support report says users sometimes see “pending” after a successful approval. The implementation team assumes this is a bug and asks an agent to mark every successful approval terminal. Under v1 that may look plausible. Under high-risk v2, pending is the intended state after the first eligible approval. The report needs the policy version, prior state and actor sequence before the team can infer a defect.

Start with one observed request and reconstruct its legal history from authorized evidence. Establish which policy was bound, which actors contributed and whether the observed response and durable state agree. If the request is high-risk v2 with one approval, the product decision may concern how the interface explains progress. If it is a v1 request incorrectly evaluated under v2, the repair belongs in policy resolution. If the evidence does not establish the starting state, record the missing fact before opening either change.

A narrow repair can proceed once the owner chooses a direction. Keep the support report linked as the original observation. Record the interpretation separately so that a later reviewer can see how the same symptom led to a particular action. This is especially useful when an agent proposes a broad fix before the team identifies the condition in which the reported behavior is legitimate.

Stale artifacts and lost context

Hypothetical symptom: a later session reintroduces an old threshold after E2. Candidate causes include an obsolete context file, a plan linked to the wrong requirement, a current specification ignored during implementation or a verification verdict reused after inputs changed.

Inspect what the session actually received where records exist. The repository containing the right requirement does not establish that the relevant worker saw it. A summary saying “context loaded” needs the identity and scope of those inputs before it supports a stronger claim.

GSD Core's documented freshness policy is one concrete mechanism for detecting changes to covered verification inputs. It does not prove coverage completeness or prevent every semantic omission. A current digest of an

incomplete input set can still support an incomplete verdict. Verification staleness policy

Recovery may require identifying the authoritative current decision, retiring or marking superseded material, refreshing the affected plan and rechecking the changed behavior. Loading every historical document is not an automatic cure; it can leave conflicting instructions competing for authority.

The prevention question is where current intent is owned and how a handoff selects it. A compact current record with a clear supersession link can be more useful than a larger archive with ambiguous status. Its usefulness still needs observation in the selected project.

Diagnose a stale handoff before regenerating the plan

Imagine a developer resumes E2 after the policy owner has approved a clarification. The plan still describes all pending requests as v2, while the decision record protects v1. First establish the revision and date of the two records. Then inspect which one the implementer received and whether any code or tests were generated from it. The first observable failure might be that a preserved v1 request stays pending after one eligible approval. This sequence is illustrative.

Repair the link at the point where the stale record entered the work. Update the task plan and any fixture that copied its assumption. Refresh affected verification against the revised code. Keep the earlier output with its revision so a reviewer can distinguish an obsolete result from a failed check on the repaired implementation. Regenerating all documentation would create more material to review while making the provenance of the contradiction harder to see.

After the repair, ask a contributor to locate the governing policy using the normal handoff. Observe whether they reach the active record and can distinguish its v1 and v2 examples. If the active record is easy to find and the wrong plan was a one-off, a narrow repair may suffice. Repeated failures at the same handoff give a reason to change where the workflow records and consumes decisions.

Diagnose a current document that carries an obsolete interpretation

A document may have a recent timestamp because someone changed formatting while leaving its behavioral assumption untouched. Inspect the accepted decision and the text that actually reached the work. Timestamp freshness alone cannot tell whether the interpretation is current.

Suppose a plan still treats every approval as terminal after the product owner introduced an intermediate pending state. The plan's revision may be newer than the policy decision. Compare the relevant sentences and trace which implementation and test expectations consumed them. Repair those dependencies and attach a fresh observation to the current candidate.

Then inspect the mechanism that selected context. If contributors chose the plan because it appeared first in search, a clear link to the accepted rule and an explicit supersession note can help. If the active work entry point copied the obsolete sentence, correct that entry point. The diagnosis should identify the path by which the assumption reached execution.

A later handoff exercise can test the repair. Ask another contributor to identify the expected result after the first eligible action on high-risk v2. Record the source they used and whether they found the preserved v1 rule as well. This gives the team an observation about the corrected context path.

Ceremony without useful decisions

Hypothetical symptom: a tiny display repair generates extensive product and architecture documents, while the only needed choice is already clear. Another project has the opposite symptom: a short path proceeds despite unresolved policy binding. Both concern process sizing, not the intrinsic virtue of fewer or more files.

For each activity, ask which decision it makes, which uncertainty it reduces or which acceptance obligation it supports. If the answer is a duplicate of current accepted context, reuse or link that context. If the activity exposes a material uncertainty, retain it even when the diff is small.

BMAD and Core document small-work allowances in the inspected sources. Their presence is counterevidence to forcing every task through the largest route, but it does not establish the actual cost of a reduced path. BMAD, Core.

Recovery is a process decision: remove redundant work, clarify ownership or enlarge a path for a newly discovered risk. Record why. A future evaluation should include preparation, review and relevant repair rather than report only initial code generation time.

The prevention question is whether the selected profile has a recognizable task-sizing rule and a way to revisit it. An informal shortcut that silently omits a required gate is a deviation, not automatically a legitimate minimal path.

Remove work only after finding the duty it serves

A team sees three review records generated for every repair. Each repeats the same file list and none names the product decision. Before eliminating the records, inspect who reads them and what decision each is meant to enable. The apparent duplication may conceal a real approval or evidence requirement. If one report is the only retained record of the release review, removing it without moving that duty would create a gap.

Try a narrower change. Preserve the release disposition and the evidence it relies on. Combine the duplicate preparation notes into one record that names the authorized repair, affected path and required observation. On the next repair, inspect whether a reviewer can still find the decision and reproduce the relevant check. The outcome tells the team whether the simplification preserved the duty under the selected conditions.

For a high-risk change, the team may need a separate decision before implementation and another before release. Those decisions can use different evidence even if they refer to one requirement. Keep both when they constrain different actions. The goal of a lighter workflow is to reduce the work required to make and inspect decisions, while keeping those decisions visible.

Circular checks and false completion

Hypothetical symptom: a suite passes while the agreed requester exclusion is absent. A candidate mechanism is circular validation: tests reproduce the implementation's assumption or exercise a different denial condition. Inspect expected behavior and fixtures before adding more tests of the same mistaken interpretation.

The book's earlier pilot contained a real editorial ambiguity: its case table could be read as completing a request before testing self-approval. The v0.2 correction made checks independent and required a pending-state fixture. This was a flaw in the teaching/test design, not an observed application or framework failure. The revision record preserves that distinction.

A separate first-hand account reports a collaborative AI-DLC sample marking empty stages successful after a credential-related error returned a zero exit code. Its author identifies an AWS role and states limits on what was observed. This is evidence about that reported platform incident, not an independent trial or a general result about AI-DLC. Agius's field account

The diagnostic question is what the status actually measures. Did a process launch, did it produce the expected artifact, did a check assert the intended condition, or did a reviewer accept the behavior? Those events can diverge. Inspect the relevant output and its interpretation rather than assume green means the business goal was met.

Recovery may involve fixing a check, distinguishing empty output from completed work, invalidating a verdict or rerunning the affected activity. Preserve the initial result and reason for correction. A rewritten success summary alone would remove the evidence needed to understand the failure.

Investigate an all-green suite

Suppose a report claims tenant isolation because a test attempted to read an A request as a B actor and received a denial. Inspect the fixture before examining the authorization branch. If the identifier never existed, a lookup without any tenant scope could produce the same result. A useful repair starts by creating the protected record and demonstrating that an authorized A actor can retrieve it. Repeat the B probe and check that the observed behavior matches the disclosure contract.

For a write path, check state and history as well as the response. A command can return a denial after doing work that should have been forbidden. Separately verify that an eligible action can succeed and produce the intended history on a clean fixture. These controls let a later reviewer explain what kind of failure a passing suite would detect. Chapter 12 develops the corresponding oracle design.

Record the original claim as unsupported by the old fixture, even if the repaired check subsequently passes. A corrected check establishes a new observation. It does not change what was exercised in the earlier run. The distinction matters when the original report was used to approve work or was cited as evidence for a larger process claim.

Review a generated diagnosis against the actual failing observation

An agent may explain a failed probe with a plausible race condition. Before changing concurrency controls, ask which observation requires that explanation. A repeated response might come from transport retry, a duplicate display event or two durable contributions. Recover the identifiers and state necessary to distinguish them.

If the proposed cause predicts two approval rows but only one exists, revise the hypothesis. If two rows exist, inspect whether both are successful contributions under the contract and which commands created them. The next repair should follow the observed boundary. Preserve the rejected explanation only where it helps a future reviewer understand the investigation.

A diagnosis can be useful even when the first hypothesis fails. Record the observation that changed the interpretation and the narrower action it enabled. This gives the team a concrete improvement to its debugging practice and prevents a confident generated explanation from becoming accepted incident history without supporting evidence.

Repair narrowly and retain the record

For a tenant-boundary defect, narrow repair means a justified change boundary with meaningful regression evidence. It does not mean changing the fewest possible characters. The same boundary may apply to direct identifiers, lists, searches and history reads.

Begin with a known existing foreign record, controlled actors and the operation identified in the report. Record the pre-repair observation when an executable baseline exists. Inspect the relevant authorization/data-access path and propose the correction against the accepted requirement. Do not invent that observation when the baseline has not been prepared.

Preserve attempted repairs separately from accepted ones. A patch that blocks one route but leaves the shared query unscoped may be useful diagnostic evidence. A failed assertion caused by a bad fixture needs a different disposition. Both should remain visible with their conditions.

The proposed repair record contains:

Entry	Purpose
Baseline and symptom	Makes the alleged failure recoverable
Fixture and expected result	Establishes which business boundary is under examination
Candidate causes and checks	Distinguishes diagnosis from speculation
Change and rationale	Explains why the scope is sufficient and proportionate
Actual regression observations	Relates repaired and preserved behavior to evidence
Open limits	Prevents a narrow repair from becoming a broad assurance claim

No such application repair packet exists for our case. Chapter 11's E3 remains a design. The record here gives a future operator a structure for honest recovery rather than a simulated success narrative.

Write a repair note that can be used next week

An illustrative record starts with the claim under review: the repair prevents foreign access to an existing request on the specified path. It then records the original observation and why its fixture could pass without exercising tenant scope. The revised fixture establishes that A can see the record and checks the defined B response. The remaining surfaces retain their own status. A later reviewer can tell which part of the earlier claim changed and why.

Link the evidence to the revision that produced it. If the repair later moves scope enforcement into a common helper, revisit the callers affected by that move. A prior result at one endpoint can remain useful, but the broader change creates additional questions. A small note with the original boundary, corrected fixture and affected callers can support that follow-up without reconstructing a long conversation.

Repair the workflow at the first point of loss

A worked diagnosis can follow a decision from ownership through use. The owner resolves a disclosure rule in a review thread. The planner copies a short summary into an issue. The implementer sees the issue and treats the summary as permission to use a different error for a known foreign record. The acceptance reviewer checks only that the response body omits the record. The workflow has lost a constraint before implementation and selected an observation that cannot reveal the loss.

The team can repair both causes without rebuilding its process. Place the accepted rule where the work record links to it. Update the fixture to distinguish a known foreign record from a nonexistent identifier under the agreed response contract. Review the repair and record the earlier finding separately. Another run can then tell the team whether the corrected handoff and observation address the diagnosed failure.

If the next change still loses the rule, investigate how contributors select context. A document can be current and still never reach the actual work. Ask a contributor to show their inputs and how they determined the governing record. A small change to the work entry point may solve the problem more directly than another checklist at the end.

Use failure to update process choices

An incident should change a process choice when evidence identifies a weakness in the arrangement. A lost policy exception may justify better context ownership. Repeated stale verdicts may justify freshness checks. Excessive work on tiny repairs may justify a smaller profile. The remedy should address the mechanism the record supports.

Avoid universal conclusions from one event. A field report can reveal a concrete defect without establishing prevalence. A synthetic example can test reasoning without predicting production frequency. A failed host integration may concern the adapter rather than the method's core design.

Return to chapter 10's selection record. Update the evidence, affected constraint, cost/uncertainty and revisit trigger. Preserve the earlier reasoning so readers can see why the decision changed. An unsupported failure hypothesis should create an investigation task, not an intuitive penalty score.

The final chapter asks how this learning procedure survives changes in models, hosts and frameworks. Its starting point is the same discipline used here: identify the claim, inspect the conditions, preserve disagreement and make the next commitment proportionate to the evidence.

Look for a repeated failure at the same boundary

A team might observe that two different changes passed native verification but each missed a preserved behavior. Before adding a new review phase, compare the two incidents. Did the owner fail to record the obligation, did the plan omit it, or did the fixture never exercise it? If both failures started in the fixture, a review rule that checks fixture preconditions may be more targeted than adding another plan approval.

Record the resulting process change as an experiment. Name the failure it should catch and choose an observation on the next applicable change. For the approval service, a reviewer could inspect whether the v1 preservation probe actually starts with a request created before the switch. If the process change catches a broken fixture, retain that observation and describe the conditions. If it finds nothing, distinguish absence of a defect from the test's ability to detect one.

Treat a counterexample as a reason to revise the rule. If a small repair produces clear evidence through an existing regression test, a new checklist may repeat information without exposing a new risk. Keep the review duty and adapt its form to the type of work. A repeatable process is easier to sustain when each required action has a visible purpose.

Retire a corrective rule when its purpose disappears

A team may add a manual handoff check after repeatedly losing one product constraint. Later, a maintained work entry point can carry that constraint and expose stale evidence directly. Review whether the manual action still detects something the new arrangement misses. Use an actual bounded exercise and record the result.

If the manual check repeats an observation already available at the decision, the team can propose removing it and name where the duty now lives. If it still catches missing context, retain the action and explain the boundary it protects. This keeps process changes connected to the failures that justified them.

Review the retirement after the next applicable change. A simplified process can be accepted with a concrete observation, just as an added control can. The team can preserve the earlier incident and correction as history while keeping the active workflow understandable.

14. The Future of AI-Native Engineering

What this edition established

This chapter has examined four source-specific arrangements for decisions, artifacts, context, implementation and verification. It has supplied a common business case and a procedure for conditional selection. It has not established an effectiveness winner.

The distinction is substantive. BMAD's planning/Build paths, Core's persistent phase context, AWS's configurable lifecycle and AIUP's maintained behavioral artifacts give us mechanisms to inspect. Documentation supports claims about those designs. The present case remains a qualitative walkthrough without installation, application or timing observations.

A separate author-written policy model has ten passing local checks. That adds a narrow executable example, not independent acceptance or a method-comparison result. Its exclusions remain part of what the edition established.

SDD adds a lifecycle question across the four: what role do specifications play over time, who owns them and how is disagreement resolved? It does not add a fifth experimental arm or a maturity score. Tool and adapter capabilities remain part of the operating conditions rather than competing methodology chapters.

The evidence gaps are also part of the edition. Independent technical review, full-book originality comparison, exact executed profiles, actual common-stack compatibility and all case readiness controls remain open. The manuscript and its evidence records preserve these limits for review.

The useful result is an inspectable decision system. Readers can identify a source/profile, state constraints, distinguish prescribed work from observation, design a reversible evaluation and record what would change the choice. That procedure can remain useful even when the implementations change.

Stable questions, changing implementations

The name of a framework can persist while its vocabulary, package contents and default route change. A host can add worker dispatch or alter permissions. A plugin can acquire a coverage check that an older snapshot lacked. Each change may affect the intervention without changing the business requirement.

Keep the stable questions close: who can decide, what information reaches implementation, what constrains change, what evidence supports acceptance and who maintains the result? They concern responsibilities rather than one interface's command names.

Versioning makes the relationship inspectable. Preserve the original documentary snapshot and record a new assessment when material conditions change. Silently rewriting an old comparison to describe new defaults would destroy the reader's ability to reconstruct the original claim.

The approval service illustrates the separation. A newer model or host may change how a plan is produced. It does not itself decide whether v1 requests retain their policy. That remains an explicit product commitment until an authorized change revises it. Reevaluate the implementation arrangement against the commitment.

The same applies to assurance. A stronger tool does not eliminate the need to identify what was checked. An improved automated review may contribute more useful evidence, but its scope, failures and human disposition still need an account.

Keep the evaluation question stable across tool changes

A team can retain a useful evaluation question while replacing the tools that help answer it. “Can a new contributor recover the accepted behavior and complete the next authorized step?” applies across hosts, models and record formats. The actual mechanism and evidence change with the environment.

Record each new arrangement's inputs and repeat the bounded exercise. Compare the observed handoff, the interventions required and the resulting acceptance evidence. A new capability can then earn its place through a concrete improvement under stated conditions. If the improvement does not appear, the team still has a reproducible account of what it tried.

Scenarios and their assumptions

The following scenarios are possibilities, not a roadmap or probability estimate. Each identifies what would need to be true and which engineering duties would remain.

Scenario	Assumption that would need evidence	Consequence to evaluate
More capable bounded workers	Workers handle a wider task set under the selected tools/budget	Whether context handoffs and independent acceptance still cover the added scope
Stronger execution controls	Configured gates reliably constrain specified consequential actions	Which bypasses, overrides and operator duties remain
Better artifact exchange	Native records can move between environments without losing meaning	Whether ownership, versioning and semantic links survive migration
More useful automated review	Checks detect relevant omissions with acceptable error/cost	Which findings still need judgment and how false confidence is avoided
Lower preparation cost	Setup/context/spec maintenance takes less effort in the actual project	Whether a larger process becomes worthwhile for smaller changes

Several scenarios can occur together or fail independently. A capable worker can still receive the wrong policy. Strong execution controls can faithfully enforce an incorrect plan. Portable files can lose the meaning of their ownership conventions. Better generation does not settle every uncertainty in acceptance.

An appropriate response is a new bounded evaluation. State the changed assumption and keep other conditions visible. If the host's reviewer capabilities change, inspect that difference rather than reattribute a result to an unchanged framework name.

Avoid turning a hypothetical possibility into a current claim by removing the word if. The book's selection procedure can incorporate new evidence without pretending we know the direction or timing of every capability change.

Evaluate progress when the agent can do more work between reviews

Imagine a team grants its agent access to a local build, a fixture database and a browser pointed at a disposable environment. The agent can now assemble a richer record than a code-only assistant. It can also exercise the wrong environment, use a fixture that bypasses identity or mistake a browser success message for a durable transition. The team's review should follow the action boundary and the evidence it can produce.

An adoption exercise can compare two changes in the same team. On the first, inspect whether the agent can run the agreed acceptance sequence and retain the actual output. On the second, alter a precondition and see whether the workflow identifies the missing input before reporting success. State the authority, fixtures and observation channels for both. A better result would support a decision about this environment and the workflow's handling of that boundary.

The next decision may concern what information is worth retaining. A full interaction log can help diagnose a failure and overwhelm an ordinary reviewer. A brief acceptance record can be easy to inspect and omit the context needed to reproduce an unusual result. Preserve the observations that supported the decision, keep the detailed history accessible under the team's data policy and identify the version of the environment that produced them. Readers can then ask what richer tooling actually improved.

An agenda for evidence

The largest open questions concern effects over tasks, people and time. Does a maintained specification help the fifth change enough to justify its upkeep? Does a context strategy recover material decisions after interruption? Does a team distribute authority clearly or accumulate approval waiting? These require observations that a single documentary walkthrough cannot supply.

Study design should match the question. Matched baselines help compare a bounded change. A longitudinal design follows the consequences of earlier outputs. Real human teams are needed for coordination claims. Operational exercises are needed for recovery claims. Combining those goals into one small demo would obscure each.

Independent assessment also needs attention. The prior comparative preprint acknowledges single-rater scoring limitations. Future documentary instruments should examine reviewer agreement and preserve disagreements rather than treat a rubric as validated by its existence. Prior comparison and limitations

Productivity measurement needs a defined denominator and honest missing-data treatment. METR's 2026 update describes selection and concurrent-agent timing problems in its later study. That is a concrete reason to revisit methods of measurement as work practices change, not a clean estimate to import into this comparison. METR's design update

Publish useful failures as well as favorable outcomes. A profile that cannot be installed under stated conditions, an omitted invariant and a stopped run all inform the decision system. Explain the level at which the failure occurred and avoid turning every missing result into a common score.

The case protocol provides a starting structure for this evidence agenda. It needs actual environments, independent checks, operators and resources before producing measured results. Its current incompleteness should remain explicit.

Compare two useful kinds of follow-up

A team evaluating its own workflow can run a short case with a frozen repository and check whether an operator preserves a versioned policy under interruption. The result can help that team decide where to keep decisions and which checks need a fresh run. Its report should name the starting state, the authority given to the operator and the observations that led to the decision.

A broader study could follow teams through several changes, recording artifact updates, review disputes, late defects and changes in configuration. That design would address questions a single task cannot answer, such as whether a maintained rule stays useful after multiple contributors edit its dependents. The study would need a way to describe differences between teams and a policy for handling abandoned attempts. The local exercise and the longer study support different decisions and can share an explicit vocabulary for their evidence.

The case in this book illustrates where to start: isolate a decision with an observable consequence. When a future team claims an approach improves maintenance, ask which change the team could make more reliably, what was observed and how a different explanation was considered. A report grounded at that level gives others a way to try the practice and report where it breaks.

Publish enough conditions for another reader to disagree precisely

A future report on an AI-oriented workflow should let a reader identify the task, selected configuration, starting state and observation behind a conclusion. Retain failures, manual interventions and unresolved outcomes. When a reader challenges the conclusion, they can then name the premise or observation at issue.

That level of reporting also helps the original team. A later change in environment can be compared with the recorded conditions. A new result can revise a specific claim while preserving observations that still apply. Progress becomes inspectable through a series of bounded decisions and the evidence that changed them.

When to evaluate again

Reevaluation is warranted when a material premise changes. A new release alone may be a research lead; a changed mandatory dependency, gate default, adapter or business requirement can alter the selection itself.

Use a short trigger record:

Trigger	Question before retaining the choice
Source/package change	Which instructions or defaults differ from the reviewed snapshot?
Host/model change	Which capabilities, costs, permissions and unavailable identifiers alter the intervention?
Application constraint change	Does the selected adapter/stack still fit without hidden adaptation?
New business policy	Which maintained artifacts, implementations and checks need reconciliation?

Trigger	Question before retaining the choice
Team/ownership change	Are decision rights and review capacity still explicit?
New operational duty	What additional environment and recovery evidence is required?
Repeated defect or overhead evidence	Which mechanism is implicated, and which smaller or different path should be tested?

The response need not be a complete re-adoption exercise. A narrow changed assumption can justify a narrow compatibility or acceptance check. A broad change in business/operational scope may require a larger evaluation. Preserve the reason for that sizing decision.

Update chapter 10's selection record with the new evidence, uncertainty and next commitment. Retain the previous record so another engineer can understand why the choice changed. A living decision record is different from an endorsement that never expires.

Revisit a choice when its conditions change

Suppose a team chose a workflow for its documented ability to resume a multi-session change from persistent context. A later upgrade changes the structure of its state records and the way it dispatches workers. The team's original selection record should say which handoff mattered and which observation supported it. That gives the team a short reevaluation task: resume a partially completed policy change under the new configuration and check that the next operator can find the accepted version rule and the stale verification record.

A different trigger comes from the work. A workflow that suited one developer making small changes may have to support a second team that owns deployment and recovery. Revisit the decision rights before evaluating speed. Ask who can stop the release, where the recovery evidence lives and how an approval is tied to the checked revision. If the old workflow cannot express one of these duties, the team can integrate a control it already uses and record the resulting arrangement.

Keep reevaluation criteria specific enough to decide an outcome. “The model got better” gives no reason to rerun every check. A change in the agent's write

permissions, a new release boundary or repeated loss of context creates a specific behavior to inspect. Preserve the earlier record and add the evidence that changed the choice. The team can then distinguish an outdated assumption from a choice that still works under new conditions.

Engineering judgment beyond this edition

Return to the small approval change that opened the book. A convincing interface is an output. Accepting responsibility for it requires knowing the intended behavior, the decisions that shaped it and the evidence that supports it. AI can participate throughout that work, but the relationship among those items must remain inspectable.

Begin the next project with a bounded question. Describe its conditions, identify the decision owners and choose a process path proportionate to unresolved work. Preserve native artifacts where they carry useful information. Check behavior against an agreed expectation and state what the observations leave open.

If evidence favors a lightweight repair, use it. If a durable behavioral record earns its maintenance cost, maintain it. If a profile's dependencies or controls do not fit the task, investigate another arrangement. The conclusion belongs to the constraints and evidence, not the popularity of a name.

Preserve a decision that a future team can revise

Suppose a team decides to keep a human-owned policy and use an agent to propose implementation and evidence. Write down the conditions under which the decision made sense. The policy changes often enough to need a durable record. One person can resolve disagreements. A second person can inspect the resulting behavior through a controlled fixture. These conditions give a future maintainer a way to reconsider the arrangement.

If ownership moves between teams, the later maintainer can revisit the authority boundary. If verification becomes easier to automate, the team can move more evidence collection into the workflow and keep the interpretation explicit. If an action grows more consequential, the team can add a decision point at that action. The earlier record gives the team a comparison point and preserves why the workflow was chosen.

The discipline applies to the book's own conclusions. Each method account names an inspected scope, the comparison states which outcomes remain open, and the case exposes the evidence a future run would need. A reader can repeat one of the proposed exercises with their own toolchain and explain which decision the observation changed. That is a concrete way to extend the edition without requiring agreement on a universal method.

A clear next commitment, a retrievable reason for it and an observation that can change the decision give the next contributor a place to begin.

Source notes

Prepared 2026-11-01. This is a source index, not a systematic literature review or a list of independent studies. Several records belong to the same repository/publisher. Exact passages and limitations remain in the receipt files and near chapter claims. No external book, website, prompt collection or figure is reproduced here.

GitHub source pins fix the documentary files discussed; they do not prove installed-package equivalence. Web URLs can change. The canonical Node TypeScript page displayed v26.13.0 when inspected; a version-specific URL failed and was not used as evidence. ASVS is cited only for its overview/edition, and SSDF 1.1 only for the stated high-level integration purpose. These are not compliance assessments.

Development recheck **2026-11-02**: M2-A02 supplementary lines 18-43 cover the AWS profile chooser and its static-count qualification. This extends the inspected passages of an existing pinned source; original access dates below are retained and the distinct record count remains 35.

Subsequent four-method rechecks retrieved BMAD's raw Build file and Core's rendered verification guide. BMAD locators were corrected in current prose; historical receipts are preserved. AWS/AIUP fresh retrievals failed in this pass and are marked **NOT VERIFIED IN THIS SESSION**; their earlier inspections are not relabeled as new visits. No new distinct source was added. Broader semantic locator review remains open.

M2-B02 - choose-a-planning-path.md

bmad-code-org/BMAD-METHOD maintainers. Source version: bab9082c3e884e0d72c759bcd1a6d90bcc01be84. Accessed 2026-11-01.

https://github.com/bmad-code-org/BMAD-METHOD/blob/bab9082c3e884e0d72c759bcd1a6d90bcc01be84/docs/plan/choose-a-planning-path.md

M2-B03 - build-a-change.md

bmad-code-org/BMAD-METHOD maintainers. Source version: bab9082c3e884e0d72c759bcd1a6d90bcc01be84. Accessed 2026-11-01.

https://github.com/bmad-code-org/BMAD-METHOD/blob/bab9082c3e884e0d72c759bcd1a6d90bcc01be84/docs/build/build-a-change.md

M2-B04 - start-in-an-existing-codebase.md

bmad-code-org/BMAD-METHOD maintainers. Source version: bab9082c3e884e0d72c759bcd1a6d90bcc01be84. Accessed 2026-11-01.

https://github.com/bmad-code-org/BMAD-METHOD/blob/bab9082c3e884e0d72c759bcd1a6d90bcc01be84/docs/existing-codebases/start-in-an-existing-codebase.md

M2-B05 - define-requirements-and-a-specification.md

bmad-code-org/BMAD-METHOD maintainers. Source version: bab9082c3e884e0d72c759bcd1a6d90bcc01be84. Accessed 2026-11-01.

https://github.com/bmad-code-org/BMAD-METHOD/blob/bab9082c3e884e0d72c759bcd1a6d90bcc01be84/docs/plan/define-requirements-and-a-specification.md

M2-B07 - test-completed-work.md

bmad-code-org/BMAD-METHOD maintainers. Source version: bab9082c3e884e0d72c759bcd1a6d90bcc01be84. Accessed 2026-11-01.

https://github.com/bmad-code-org/BMAD-METHOD/blob/bab9082c3e884e0d72c759bcd1a6d90bcc01be84/docs/build/test-completed-work.md

M2-B10 - package.json

bmad-code-org/BMAD-METHOD maintainers. Source version: bab9082c3e884e0d72c759bcd1a6d90bcc01be84. Accessed 2026-11-01.

https://github.com/bmad-code-org/BMAD-METHOD/blob/bab9082c3e884e0d72c759bcd1a6d90bcc01be84/package.json

M2-G03 - install-on-your-runtime.md

open-gsd/gsd-core maintainers. Source version: 72c4e14dfa56ecc98c04b3fd6736bea2dd3195dc. Accessed 2026-11-01.

https://github.com/open-gsd/gsd-core/blob/72c4e14dfa56ecc98c04b3fd6736bea2dd3195dc/docs/how-to/install-on-your-runtime.md

M2-G06 - package.json

open-gsd/gsd-core maintainers. Source version: 72c4e14dfa56ecc98c04b3fd6736bea2dd3195dc. Accessed 2026-11-01.

https://github.com/open-gsd/gsd-core/blob/72c4e14dfa56ecc98c04b3fd6736bea2dd3195dc/package.json

M2-A02 - workflow-profiles.md

awslabs/aidlc-workflows maintainers. Source version: 3939a8b9182f2e4153b92028057a7be74e0684c2. Accessed 2026-11-01.

https://github.com/awslabs/aidlc-workflows/blob/3939a8b9182f2e4153b92028057a7be74e0684c2/docs/guide/workflow-profiles.md

M2-U02 - README.md

AI-Unified-Process/marketplace maintainers. Source version: 3ad51b0707e9bed6e54b9de46ebc2f28d7afa960. Accessed 2026-11-01.

https://github.com/AI-Unified-Process/marketplace/blob/3ad51b0707e9bed6e54b9de46ebc2f28d7afa960/aiup-nestjs-nextjs/README.md

M2-U01 - workflow.md

AI-Unified-Process/marketplace maintainers. Source version: 3ad51b0707e9bed6e54b9de46ebc2f28d7afa960. Accessed 2026-11-01.

https://github.com/AI-Unified-Process/marketplace/blob/3ad51b0707e9bed6e54b9de46ebc2f28d7afa960/docs/workflow.md

M2-W11 - Introducing GitHub Copilot: your AI pair programmer

GitHub / Nat Friedman. Source version: Web page; version as inspected. Accessed 2026-11-01.

https://github.blog/news-insights/product-news/introducing-github-copilot-ai-pair-programmer/

M2-W01 - AI-Driven Development Life Cycle: Reimagining Software Engineering

AWS / Raja SP. Source version: Web page; version as inspected. Accessed 2026-11-01.

https://aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/

M3-H01 - ReAct: Synergizing Reasoning and Acting in Language Models

Yao et al., author-submitted paper on arXiv. Source version: arXiv:2210.03629v1. Accessed 2026-11-01.

https://arxiv.org/abs/2210.03629v1

M3-H02 - SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering

Yang et al., author-submitted paper on arXiv. Source version: arXiv:2405.15793v1. Accessed 2026-11-01.

https://arxiv.org/abs/2405.15793v1

M2-G01 - the-phase-loop.md

open-gsd/gsd-core maintainers. Source version: 72c4e14dfa56ecc98c04b3fd6736bea2dd3195dc. Accessed 2026-11-01.

https://github.com/open-gsd/gsd-core/blob/72c4e14dfa56ecc98c04b3fd6736bea2dd3195dc/docs/explanation/the-phase-loop.md

M2-A01 - 04-phases-and-stages.md

awslabs/aidlc-workflows maintainers. Source version: 3939a8b9182f2e4153b92028057a7be74e0684c2. Accessed 2026-11-01.

https://github.com/awslabs/aidlc-workflows/blob/3939a8b9182f2e4153b92028057a7be74e0684c2/docs/guide/04-phases-and-stages.md

M2-W03 - Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl

Birgitta Böckeler / Martin Fowler site. Source version: Web page; version as inspected. Accessed 2026-11-01.

https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html

M2-W05 - Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity

METR / Becker, Rush, Barnes, Rein. Source version: Web page; version as inspected. Accessed 2026-11-01.

https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/

M2-W06 - We are Changing our Developer Productivity Experiment Design

METR / Becker, Rush, Cunningham, Rein, Mahamud. Source version: Web page; version as inspected. Accessed 2026-11-01.

https://metr.org/blog/2026-02-24-uplift-update/

M2-G07 - README.md

gsd-build/get-shit-done maintainers. Source version: bdcaab2c752d9a33a1a1ca9acf3a3c81fb991815. Accessed 2026-11-01.

https://github.com/gsd-build/get-shit-done/blob/bdcaab2c752d9a33a1a1ca9acf3a3c81fb991815/README.md

M2-G04 - verify-and-ship.md

open-gsd/gsd-core maintainers. Source version: 72c4e14dfa56ecc98c04b3fd6736bea2dd3195dc. Accessed 2026-11-01.

https://github.com/open-gsd/gsd-core/blob/72c4e14dfa56ecc98c04b3fd6736bea2dd3195dc/docs/how-to/verify-and-ship.md

M2-A03 - aidlc-plan-approval-guard.ts

awslabs/aidlc-workflows maintainers. Source version: 3939a8b9182f2e4153b92028057a7be74e0684c2. Accessed 2026-11-01.

https://github.com/awslabs/aidlc-workflows/blob/3939a8b9182f2e4153b92028057a7be74e0684c2/core/hooks/aidlc-plan-approval-guard.ts

M2-A04 - 14-artifacts-reference.md

awslabs/aidlc-workflows maintainers. Source version: 3939a8b9182f2e4153b92028057a7be74e0684c2. Accessed 2026-11-01.

https://github.com/awslabs/aidlc-workflows/blob/3939a8b9182f2e4153b92028057a7be74e0684c2/docs/guide/14-artifacts-reference.md

M2-U03 - SKILL.md

AI-Unified-Process/marketplace maintainers. Source version: 3ad51b0707e9bed6e54b9de46ebc2f28d7afa960. Accessed 2026-11-01.

https://github.com/AI-Unified-Process/marketplace/blob/3ad51b0707e9bed6e54b9de46ebc2f28d7afa960/aiup-core/skills/spec-review/SKILL.md

M2-U07 - SKILL.md

AI-Unified-Process/marketplace maintainers. Source version: 3ad51b0707e9bed6e54b9de46ebc2f28d7afa960. Accessed 2026-11-01.

https://github.com/AI-Unified-Process/marketplace/blob/3ad51b0707e9bed6e54b9de46ebc2f28d7afa960/aiup-nestjs-nextjs/skills/nest-test/SKILL.md

M2-U09 - SKILL.md

AI-Unified-Process/marketplace maintainers. Source version: 3ad51b0707e9bed6e54b9de46ebc2f28d7afa960. Accessed 2026-11-01.

https://github.com/AI-Unified-Process/marketplace/blob/3ad51b0707e9bed6e54b9de46ebc2f28d7afa960/aiup-nestjs-nextjs/skills/implement/SKILL.md

M2-W04 - From Prompt to Process: a Process Taxonomy and Comparative Assessment of Frameworks Supporting AI Software Development Agents

Sanderson Oliveira de Macedo. Source version: Web page; version as inspected. Accessed 2026-11-01.

https://arxiv.org/html/2606.04967v1

M2-W07 - AI-DLC in a Team: the Agent Is No Longer the Bottleneck

Alexandre Agius (bio identifies AWS affiliation). Source version: Web page; version as inspected. Accessed 2026-11-01.

https://www.agiusalexandre.com/blog/2026-09-26-ai-dlc-collaborative-team-bottleneck/

M2-W16 - Node.js TypeScript documentation

Node.js project. Source version: Node.js v26.13.0; canonical URL can change. Accessed 2026-11-01.

https://nodejs.org/api/typescript.html

M5-SEC01 - NIST SP 800-218 / SSDF 1.1

NIST. Source version: Web page; version as inspected. Accessed 2026-11-01.

https://csrc.nist.gov/pubs/sp/800/218/final

M5-SEC02 - ASVS overview and release identification

OWASP. Source version: Web page; version as inspected. Accessed 2026-11-01.

https://asvs.owasp.org/

M5-SEC03 - Authorization Cheat Sheet

OWASP. Source version: Web page; version as inspected. Accessed 2026-11-01.

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

M5-SEC04 - Secrets Management Cheat Sheet

OWASP. Source version: Web page; version as inspected. Accessed 2026-11-01.

https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

M5-SEC05 - LLM Prompt Injection Prevention Cheat Sheet

OWASP. Source version: Web page; version as inspected. Accessed 2026-11-01.

https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html

The current manuscript has no controlled methodology/application benchmark. External study findings and first-hand reports remain attributed and scoped; documentary design is not effectiveness evidence. The local policy example has its own command/output record and is not an independent acceptance oracle.

Full-book originality comparison, per-asset clearance and independent technical review remain pending. Repository licence inspection does not itself clear marks or every outside asset. Working bibliography formatting and copyediting remain production tasks.

Editorial source recheck - 2026-11-05

Selected passages of the pinned AWS profile guide and AIUP workflow guide were retrieved through the GitHub connector after earlier web retrieval failures. The AWS recheck covers profile routing, test depth, review dispatch and guard-policy defaults. The AIUP recheck covers construction inputs, decision rights, the adapter-specific coverage-auditor limit and change reconciliation. The inspection record identifies the exact URLs and line ranges. Broader semantic review and independent technical review remain open.

Publication review - 2026-11-05

The publication pass retrieved all 20 cited GitHub files at their pinned revisions and reopened all 15 cited web sources. All 95 GitHub citation occurrences resolved to existing ranges in those retrieved files. Semantic spot checks covered version and runtime requirements, QA boundaries, configured authority, adapter-specific audit limits, SDD terminology and the attributed research and field findings. The publication audit records the scope.

A bounded exact-phrase comparison found no contiguous 12-word match between the chapter prose and the 20 retrieved GitHub files under its normalization rules. This comparison excludes proprietary similarity databases and complete competing books. Independent human review and complete rights/originality clearance remain open.